# Tartalomjegyzék

**1. Az alapvető adatbázis-ismeretek áttekintése** **3**
Adatmodellezés . . . 4
Lekérdezés . . . 6
Programozás . . . 7
Kurzorok . . . 10
Tranzakciókezelés . . . 11

**2. Laza csatolás triggerek és ütemezett feladatok alapján** **13**
Problémaforgatókönyv . . . 13
Megoldás . . . 13
A triggerek rövid áttekintése . . . 14
Szoros csatolás . . . 15
A lazán csatolt rendszer . . . 16

**3. Replikáció, naplószállítás és feladatátvétel** **19**
Replikációs fogalmak és architektúra . . . 19
Pillanatkép-replikáció . . . 22
Tranzakciós replikáció . . . 27
Replikáció különálló szerverek között . . . 31
Peer-to-Peer tranzakciós replikáció . . . 34
Összefésülő replikáció . . . 35
Naplószállítás . . . 41
Feladatátvételi fürtök . . . 45

**4. Adatminőség és törzsadat-menedzsment** **47**
Adatprofil-elemzés . . . 49
SQL Server Data Quality Services . . . 49

**5. Oszloptár és particionálás** **53**
Oszloptár (Columnstore) . . . 53
Particionálás . . . 55

**6. Adatbázis-migráció** **59**
Migráció relációs technológiák között . . . 60
Migráció relációsból dokumentumtárolóba . . . 60
Dokumentumtárolók: A Cloud Firestore áttekintése . . . 62
Az adatintegritás érvényesítésének összehasonlítása . . . 64
Tranzakciók Firestore-ban és Postgres-ben . . . 65

**7. Felhőalapú adatbázis-technológiák** **67**
Áttekintés . . . 67
Új GCP-projekt indítása . . . 68
A Postgres adatbázis migrálása GCP-re . . . 68
A demóalkalmazás megvalósítása GCP-n → Cloud programozás MSc kurzus . . . 69
BigQuery áttekintés és bemutató . . . 70
További olvasnivaló . . . 73

**8. Gráfmodellek és BLOB-tárolás** **73**
SQL Server gráftáblák . . . 74
GraphQL interfészek . . . 77
Neo4j natív gráfadatbázis . . . 78
Adatok importálása Neo4j-be . . . 80
Képek és BLOB-ok tárolása . . . 81

**9. Memóriaoptimalizált táblák az SQL Serverben** **82**
Jellemzők . . . 82
Követelmények és korlátozások . . . 83
Ajánlott alkalmazási területek . . . 83
GYAKORLAT — Teljesítménymérés . . . 84
Migráció tervezése In-Memory megoldásra . . . 84
GYAKORLAT — Reális forgatókönyv párhuzamos munkamenetekkel . . . 84

**10. Köszönetnyilvánítás** **85**

**11. A függelék: SQL DML-példák önálló tanuláshoz** **85**
Alapvető SELECT lekérdezések . . . 85
Aggregáló függvények és beágyazott lekérdezések . . . 85
JOIN-ok . . . 86
Csoportosítás (GROUP BY) . . . 86
Önillesztés (hierarchia lekérdezése) . . . 87
Aritmetika, STR() formázás . . . 87
HAVING, ISNULL, a legjobb értékesítő . . . 88
TOP — a legnépszerűbb termék . . . 88
Dátumfüggvények . . . 89
CASE kifejezés . . . 89
Ideiglenes táblák . . . 90
Házi feladatok . . . 90

**12. B függelék: Adatbázis-adminisztráció és -karbantartás** **90**
Adatbázisfájlok . . . 91
Adatbázis-teljesítmény . . . 92
Biztonsági mentések . . . 93
Karbantartási tervek . . . 94
Riasztások (Alerts) . . . 95
Rendszerbiztonsági tagok, jogosultságok, sémák, szerepkörök . . . 97
Adatbiztonság az SQL Serverben . . . 99
SSL konfigurálása . . . 101
Mentési titkosítás (Backup Encryption) . . . 102
Always Encrypted . . . 103

# 1. Az alapvető adatbázis-ismeretek áttekintése

Ebben a kurzusban egyes bemutatókhoz a Northwind minta relációs adatbázist és a Microsoft SQL Server 2022 technológiát fogjuk használni. A Northwind adatbázist egy kis, fogyasztási cikkekkel kereskedő vállalat kiszolgálására tervezték. Tartalmaz egy készletnyilvántartást és táblákat a rendelések adminisztrációjához. A táblák és fájlok nevei önmagukért beszélnek.

*(Az adatbázis sémáját lásd a PDF 5. oldalán: Suppliers, Products, Categories, Orders, Order Details, Customers, Employees, EmployeeTerritories, Territories, Region táblák.)*

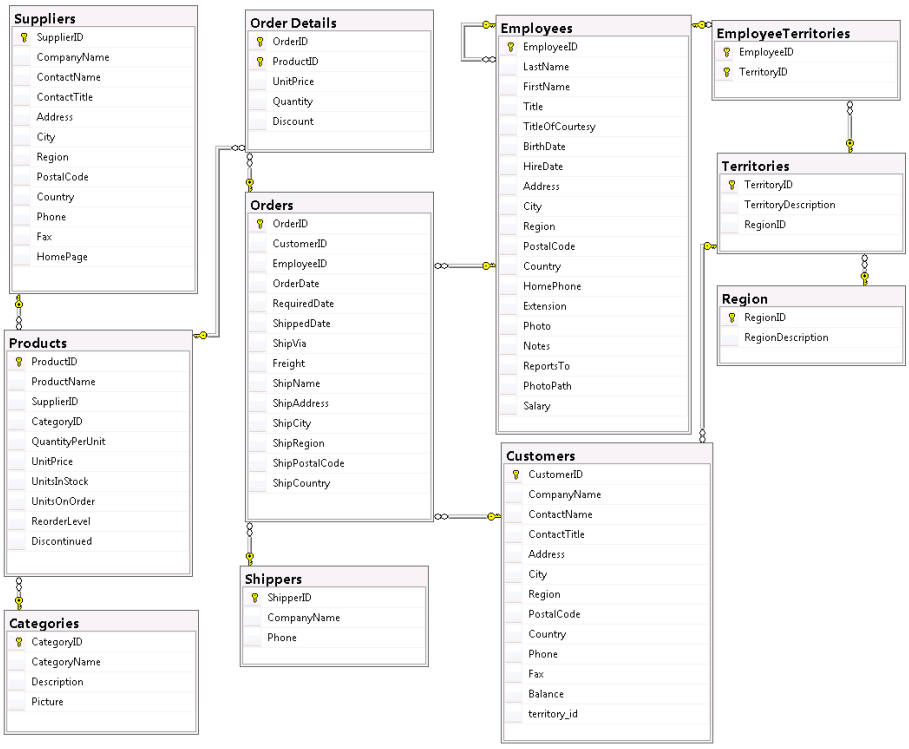
Az adatbázis-mentés (dump) letölthető innen: https://www.microsoft.com/en-us/download/details.aspx?id=23654
A kurzushoz az eredeti adatbázist módosítottuk: a Customers táblához hozzáadtuk

a `territory_id` idegen kulcsot, az Employees táblához pedig a `Salary` mezőt.

---

## Adatmodellezés

- Először az On-Line Transaction Processing (OLTP) adatbázisok alapvető relációs modellezési fogalmait tekintjük át a Northwind adatbázis példáján: Customers, Employees, Orders, OrderDetails, Products, Categories, Territories táblák.




- A kiindulópontunk a **fogalmi modell** (tartománymodell vagy egyed-kapcsolat modell), amelyet a használati esetekből (use case) vezetünk le; célunk a logikai adatbázis-modell kidolgozása.
- A **relációs modell** a legelterjedtebb paradigma a hagyományos üzleti folyamatok modellezésére, egyszerűsége miatt.
- Az entitásokat, attribútumokat, példányokat és azonosítókat a relációs modellben táblák, mezők, rekordok és elsődleges kulcsok valósítják meg. A kulcsok több mezőből is állhatnak.
- Minden cellában csak egyetlen érték szerepelhet; a harmadik normálforma (3NF) célja a redundancia és az inkonzisztencia minimalizálása. A 3NF jellemzői:
  - Minden táblának van egy elsődleges kulcsa, amely több mezőből is állhat, és amelytől az összes többi mező funkcionálisan függ.
  - Összetett (többmezős) kulcsok esetén az összes nem-kulcs mező a teljes kulcstól függ, nem csupán annak egy részétől — vagyis nincsenek **részleges függések**.
  - A nem-kulcs mezők kizárólag a kulcstól függnek, más mezőtől nem — vagyis nincs **tranzitív függés** a táblán belül.
- A táblák kapcsolatokon keresztül kapcsolódnak egymáshoz.

- **1:N** (egy-a-sokhoz) kapcsolatokat idegen kulcsok valósítanak meg (pl. `Orders.EmployeeID`).
- **N:M** (sok-a-sokhoz) kapcsolatokat kapcsolótáblák valósítanak meg (pl. `EmployeeTerritories`).
- **1:1** (egy-az-egyhez) kapcsolatra a Northwind adatbázisban nincs példa.
  - Normál 1:1 kapcsolat lehet például egy `CompanyCar` tábla, ahol egy alkalmazotthoz legfeljebb egy céges autó tartozhat.
  - Specializációs 1:1 kapcsolat lehet például egy `ExciseProducts` (Jövedéki termékek) tábla, `ExciseDutyAmount`, `RegBarCode` stb. extra mezőkkel.
- A kapcsolótáblákban általában összetett kulcsuk van. Mesterséges (generált) kulcsot csak akkor alkalmazunk, ha külső hivatkozás szükséges.
- Az OLTP séma kapcsolatstruktúrája feltárja az adatbázist használó alkalmazás kulcstranzakcióit.
  - **Hópehely- (snowflake) vagy snowball struktúra**: minden hópehely egy vagy több tranzakciót szolgál ki.
  - Az **alaptáblák** a leveleken helyezkednek el (pl. `Region`, `Customers`, `Categories`).
  - A **tranzakciós táblák** (eseménytáblák) középen találhatók (`Order Details`, `EmployeeTerritories`). Ezek a táblák az információs rendszer „lüktető szívét” alkotják.

| Jellemző | Alaptáblák | Tranzakciós táblák |
|---|---|---|
| Pozíció a sémában | Levélelem. Nem hivatkozik más táblára. | Középpont. Közvetlenül vagy közvetve hivatkozik az összes táblára. |
| Méret | Kicsi | Nagy |
| Változás sebessége | Lassú. Hideg (offline) mentés is elegendő lehet. | Gyors. Online mentés szükséges. |

- Megfeleltetés egy jól tervezett grafikus felhasználói felület (GUI) és a relációs séma között:
  - Rejtett vagy csak olvasható felirat: **kulcs**
  - Szerkeszthető szövegmezők: a kulcstól függő attribútumok (mezők)
  - Legördülő / kombinált listák: hivatkozások alaptáblákra
  - Jelölőnégyzet kiegészítő szövegmezővel: specializáció
  - Legördülő táblák vagy listák: 1:N kapcsolatok
- További olvasnivaló az adatmodellezésről:
  - https://www.safaribooksonline.com/library/view/relational-theory-for/9781449365431/ch01.html
  - http://www.blackwasp.co.uk/RelationalDBConcepts.aspx
  - https://www.tutorialspoint.com/ms_sql_server/index.htm
- **GYAKORLAT:** Hozd létre és bővítsd ki a mintaadatbázist!
  - Telepítsd az MS SQL Server 2016-os vagy újabb verzióját, indítsd el az adatbázisszolgáltatást, és csatlakozz hozzá az MS Management Studio segítségével.
  - Futtasd a Northwind adatbázis-létrehozó szkriptjét (dump), és tekintsd át a táblákat a grafikus felhasználói felület (GUI) eszközeinek segítségével.
  - Rajzolj a fenti diagramhoz hasonló logikai adatbázis-modell diagramot.
  - Add hozzá az `Employees.Salary` és a `Customers.territory_id` mezőket.
  - Tervezd meg és valósítsd meg az adatbázis alábbi forgatókönyv modellezéséhez szükséges bővítményét:

*Az alkalmazottainkat rendszeres képzésekre küldjük, ahol különböző készségeket sajátítanak el. A képzéseket szerződött harmadik fél cégek szervezik. Minden foglalkoztatási kategóriához (pl. „értékesítési vezető", lásd `Employees.Title`) van egy listánk az elvárt készségekről (pl. „B szintű üzleti prezentáció" vagy „számviteli alapismeretek"), amelyeket a munkaviszony megkezdésétől számított 10 éven belül kell elsajátítani. Minden képzéshez tároljuk az időtartamot (kezdési és befejezési dátum), a helyszínt, a szervező céget, az oktatott készségeket, a résztvevőket, a képzési állapotukat (pl. „beiratkozott", „elkezdett", „befejezett", „megszakított") és vizsgaeredményeiket az egyes készségek vonatkozásában. A képzéseket szervező cégek tekintetében tároljuk a cégünk által az egyes képzési foglalkozásokért évente fizetett díjakat.*

- Add hozzá az új táblákat az adatbázis-diagramhoz, és vigyél fel néhány tesztadatot.
- MEGOLDÁS: `train_tables.sql`[2]

---

## Lekérdezés

- Az SQL lekérdezés alapjait tekintjük át: adat-kiválasztás (select), csoportosítás, összekapcsolás (`join`). Példalekérdezések:
  - Az egyes rendelések értéke
  - Az egyes termékekből évenként eladott minimális és maximális mennyiség
  - Melyik alkalmazott adta el a legtöbb darabot a legnépszerűbb termékből 1998-ban?

```sql
-- Az egyes rendelések értéke
select o.orderid, o.orderdate,
  str(sum((1-discount)*unitprice*quantity), 15, 2) as order_value,
  sum(quantity) as no_of_pieces,
  count(d.orderid) as no_of_items
from orders o inner join [order details] d on o.orderid=d.orderid
group by o.orderid, o.orderdate
order by sum((1-discount)*unitprice*quantity) desc

-- Évenként eladott mennyiség termékenkénti bontásban
select p.ProductID, p.ProductName, year(o.orderdate), SUM(quantity) as quantity
from orders o inner join [order details] d on o.orderid=d.orderid
inner join Products p on p.ProductID=d.ProductID
group by p.ProductID, p.ProductName, year(o.orderdate)
order by p.ProductName

-- Melyik alkalmazott adta el a legtöbb darabot a legnépszerűbb termékből 1998-ban?
select top 1 u.titleofcourtesy+' '+u.lastname+' '+u.firstname+' ('+u.title+')' as name,
  sum(quantity) as pieces_sold,
  pr.productname as productname
from orders o inner join [order details] d on o.orderid=d.orderid
  inner join employees u on u.employeeid=o.employeeid
  inner join products pr on pr.productid=d.productid
where year(o.orderdate)=1998 and d.productid =
  (select top 1 p.productid
  from products p left outer join [order details] d on p.productid=d.productid
  group by p.productid
  order by count(*) desc)
group by u.employeeid, u.titleofcourtesy, u.title, u.lastname, u.firstname,
pr.ProductID, pr.productname
order by sum(quantity) desc
```

- További példákért és az SQL lekérdezések szisztematikus áttekintéséért lásd a Mellékletet.
- További olvasnivaló a lekérdezésekről:
  - https://docs.microsoft.com/en-us/sql/t-sql/queries/queries

- **GYAKORLAT:** Az első gyakorlatban megvalósított táblák segítségével valósítsd meg a következő lekérdezéseket:
  - Mik a hiányzó készségek Mrs. Peacock számára?
  - Vannak-e olyan jövőbeli képzések, amelyeken Peacocknak még részt kell vennie?
  - Mi az első és az utolsó képzési dátum, és mekkora a képzések átlagos időtartama napokban?
  - Melyik alkalmazott rendelkezik a legtöbb olyan készséggel, amelynek vizsgaeredménye jobb, mint „nem felelt meg” (fail)?
  - Mennyi az összes díj, amelyet azokért a képzési foglalkozásokért fizettünk, amelyeken a legképzettebb alkalmazottunk (lásd fent) részt vett?
  - Melyik elvárt készség(ek)et nem fedte le még egyetlen képzési foglalkozás sem?
  - MEGOLDÁS: `train_solution.sql`

---

## Programozás

- A deklaratív SQL lekérdezések mellett a procedurális üzleti logika a T-SQL szkriptnyelvben is megvalósítható, és a szerveroldalon futtatható és tárolható.
  - A szerveroldali üzleti logika előnyei és hátrányai:
    - ✓ Egyszerű architektúra
    - ✓ Platformfüggetlenség
    - ✓ Adatbiztonság
    - ✓ Kezelhetőség
    - ✓ Hatékonyság
    - ✓ Jól olvasható kód
    - ✗ Alacsony szintű
    - ✗ Gyenge szoftvertechnológiai támogatás
    - ✗ Drága skálázhatóság
  - Összefoglalás: azokat az üzletilogika-elemeket, amelyek *nagy mennyiségű*, *strukturált* adaton végzett egyszerű, halmazalapú műveleteket igényelnek, a legjobb az adatbázis-szerveren tárolt eljárások, függvények, triggerek és ütemezett feladatok formájában megvalósítani. A procedurálisan összetett részeket, amelyekhez magas szintű, objektumorientált programozási környezet szükséges, alkalmazásszerveren kell futtatni.
  - A szerveroldali programozhatóság elemei:
    - Speciális SQL vezérlőutasítások: `DECLARE`, `SET`, `BEGIN/END`, `IF/ELSE`, `WHILE/BREAK/CONTINUE`, `RETURN`, `WAITFOR/DELAY/TIME`, `GOTO`
    - Hibakezelés: `TRY/CATCH/THROW/RAISERROR`
    - Programozhatóságot támogató objektumok: `CREATE PROCEDURE/FUNCTION/TRIGGER`
    - Tranzakciókezelés: `BEGIN/COMMIT/ROLLBACK TRANSACTION`
  - Az alábbiakban egy egyszerű T-SQL szkript és tárolt eljárásként megvalósított megfelelője látható. A hasonló, felhasználó által definiált függvény `SELECT` utasításban is hívható.

```
--egyszerű szkript: alkalmazott keresése, és ha egyetlen egyező rekordot találunk,
--a fizetés emelése 10%-kal
set nocount on
declare @name nvarchar(20), @address nvarchar(max), @res_no int, @emp_id int
set @name='Fuller'
select @res_no=count(*) from Employees where LastName like @name + '%'
if @res_no=0 print 'No matching record.'
else if @res_no>1 print 'More than one matching record.'
```

```sql
else begin --egyetlen találat
    select @address=Country+', '+City+' '+Address, @emp_id=EmployeeID
        from Employees where LastName like @name
    print 'Employee ID: ' + cast(@emp_id as varchar(10)) + ', address: ' + @address
    update Employees set salary=1.1*salary where EmployeeID=@emp_id
    print 'Salary increased.'
end
go

--tárolt eljárásba csomagolva
create procedure sp_increase_salary @name nvarchar(40)
as
set nocount on
declare @address nvarchar(max), @res_no int, @emp_id int
select @res_no=count(*) from Employees where LastName like @name + '%'
if @res_no=0 print 'No matching record.'
else if @res_no>1 print 'More than one matching record.'
else begin --egyetlen találat
    select @address=Country+', '+City+' '+Address, @emp_id=EmployeeID
        from Employees where LastName like @name
    print 'Employee ID: ' + cast(@emp_id as varchar(10)) + ', address: ' + @address
    update Employees set salary=1.1*salary where EmployeeID=@emp_id
    print 'Salary increased.'
end
go
--teszt
select Salary from Employees where LastName like 'Fuller%'
exec sp_increase_salary 'Fuller'
select Salary from Employees where LastName like 'Fuller%'

--skaláris értékű függvény: visszaadja egy személy fizetését, vagy 0-t, ha nem található
go
create function fn_salary (@name nvarchar(40)) returns money as
begin
    declare @salary money, @res_no int
    select @res_no=count(*) from Employees where LastName like @name + '%'
    if @res_no <> 1 set @salary=0
    else select @salary=Salary from Employees where LastName like @name + '%'
    return @salary
end
go
--teszt
select [your user name].fn_salary('Fuller') as salary
```

- Megjegyzés: a **tárolt eljárás** több rekordhalmazt is visszaadhat, ha változóhozzárendelés nélkül több SELECT utasítást tartalmaz. A fenti példában szereplő paraméterek INPUT típusú paraméterek. A tárolt eljárások OUTPUT paraméterekkel skaláris értékeket is visszaadhatnak (nem szerepel a példában). Mivel más tárolt eljárásokat és függvényeket is meghívhatnak, összetett üzleti logika is megvalósítható velük az adatbázis-szerveren.
- A **felhasználó által definiált függvény** abban különbözik a tárolt eljárástól, hogy egyetlen visszatérési értéke van — típusa skaláris (pl. `money`) vagy tábla lehet. A függvény utolsó utasítása RETURN kell legyen. A felhasználó által definiált függvények előnye, hogy SELECT utasításból is meghívhatók, akárcsak a beépített SQL függvények (pl. `DATEDIFF`), így jelentősen növelik a statikus SQL lekérdezések rugalmasságát.
- **GYAKORLAT:**
  - A képzési lekérdezések segítségével hozz létre egy tárolt eljárást, amely visszaadja a hiányzó készségeket egy paraméterként átadott alkalmazott neve alapján. Az eljárás egyetlen mezőt tartalmazó táblát adjon vissza. Ha az alkalmazott nem azonosítható egyértelműen, adj vissza hibaüzenetet, és ne adj vissza táblát.
  - A képzési lekérdezések segítségével hozz létre egy tábla értékű függvényt, amely egy `employeeID` alapján adja vissza a hiányzó készségeket. Tipp: használd a `returns table` kulcsszót a függvény specifikációjában.

- Egy reálisabb üzleti folyamat bemutatásához lássunk egy példaszkriptet, amely egy új Northwind rendelést hoz létre egyetlen rendelési tétellel. A forgatókönyv: a vállalat irodája telefonon sürgős rendelést kap egy értékes vevőtől — ez egy tipikus üzleti tranzakció.

```
--változók
declare @prod_name varchar(20), @quantity int, @cust_id nchar(5)
declare @status_message nvarchar(100), @status int
declare @res_no int
declare @prod_id int, @order_id int
declare @stock int
declare @cust_balance money
declare @unitprice money

-- paraméterek
set @prod_name = 'boston'
set @quantity = 10
set @cust_id = 'AROUT'

begin try
  select @res_no = count(*) from products where productname like '%' + @prod_name + '%'
  if @res_no <> 1 begin
      set @status = 1
      set @status_message = 'ERROR: Ambiguous Product name.';
  end else begin
      select @prod_id = productID, @stock = unitsInStock from products where
productName like '%' + @prod_name + '%'
      if @stock < @quantity begin
          set @status = 2
          set @status_message = 'ERROR: Stock is insufficient.'
      end else begin
          select @cust_balance = balance from customers where customerid = @cust_id
          select @unitprice = unitPrice from products where productID = @prod_id
          if @cust_balance < @quantity*@unitprice or @cust_balance is null
          begin
              set @status = 3
              set @status_message = 'ERROR: Customer not found or balance insufficient.'
          end else begin
              -- nincs több ellenőrzés, kezdjük a tranzakciót (3 lépés)
              -- 1. egyenleg csökkentése
              update customers set balance = balance-(@quantity*@unitprice) where
customerid=@cust_id
              -- 2. új rekord az Orders és Order Details táblákban
              insert into orders (customerID, orderdate) values (@cust_id, getdate())
              set @order_id = @@identity
              insert [order details] (orderid, productid, quantity,
UnitPrice, Discount) values(@order_id, @prod_id, @quantity, @unitprice, 0)
              -- 3. készlet frissítése
              update products set unitsInStock = unitsInStock - @quantity
where productid = @prod_id
              set @status = 0
              set @status_message = 'Order No. ' + cast(@order_id as varchar(20)) + ' processed
               ↪  successfully.'
          end
      end
  end
  print @status
  print @status_message
end try
begin catch
  print 'OTHER ERROR: '+ ERROR_MESSAGE() + ' (' + cast(ERROR_NUMBER() as varchar(20)) + ')'
end catch
go
```

- Teszteld a fenti tárolt eljárást különböző hibákra: programozási hibák és logikai hibák (pl. elégtelen készlet). Ellenőrizd az adatbázis integritását. Győződj meg arról, hogy a tranzakciós támogatás megakadályozza a súlyos hibákat.
- Az elszigeteltség biztosítása érdekében a szerver zárolást alkalmaz sorokon (rekordokon), tartományokon vagy táblákon. Az **izolációs szint** (Isolation Level) a szerver által kénysze-

rített zárolási stratégia. Az MS SQL Server fő zártípusai: megosztott (shared), kizárólagos (exclusive) és frissítési (update). A négy ANSI szabványos izolációs szint:

- READ UNCOMMITTED: nincs zárolás
- READ COMMITTED: a zárak az SQL utasítás végrehajtása után felszabadulnak
- REPEATABLE READ: a tranzakcióhoz kiosztott zárak a tranzakció végéig megmaradnak
- SERIALIZABLE: más tranzakciók nem szúrhatnak be rekordokat olyan táblába, amelyen a tranzakciónak sor- vagy tartományzárja van — fantomolvasás nem lehetséges

```
--egyszerű izolációs szint bemutató: webáruház eset
create table test_product(id int primary key, prod_name varchar(50) not null, sold
varchar(50), buyer varchar(50))
insert test_product(id, prod_name, sold) values (1, 'car', 'for sale')
insert test_product(id, prod_name, sold) values (2, 'horse', 'for sale')
go
set tran isolation level read committed --az alapértelmezett
go
begin tran
declare @sold varchar(50)
select @sold=sold from test_product where id=2
if @sold='for sale' begin
  waitfor delay '00:00:10' --bankátutalás szimulálása
  update test_product set sold='sold', buyer='My name' where id=2
  print 'sold successfully'
end else print 'product not available'
commit tran
go
--Következtetés: körültekintően válaszd meg a megfelelő izolációs szintet.
```

- **GYAKORLAT:** Az előző gyakorlatokban megvalósított táblák és szkriptek segítségével:
  - Írj egy szkriptet, amely ellenőrzi, hogy egy alkalmazottnak szüksége van-e valamelyik képzési foglalkozáson kínált készségre, és ha igen, iratkoztasd be az alkalmazottat az összes ilyen foglalkozásra.
  - Futtasd a szkriptet tárolt eljárásként.
  - MEGOLDÁS: `train_solution.sql`

---

## Kurzorok

A kurzorok olyan problémáknál alkalmazhatók, ahol a procedurális, rekordonkénti feldolgozás megfelelőbb, mint a halmazalapú lekérdezési megközelítés.

**PÉLDA** kurzor szintaxisra:

```
declare @emp_id int, @emp_name nvarchar(50), @i int, @address nvarchar(60)
declare cursor_emp cursor for
  select employeeid, lastname, address from employees order by lastname
set @i=1
open cursor_emp
fetch next from cursor_emp into @emp_id, @emp_name, @address
while @@fetch_status = 0
begin
  print cast(@i as varchar(5)) + ' EMPLOYEE:'
  print 'ID: ' + cast(@emp_id as varchar(5)) + ', LASTNAME: ' + @emp_name + ', ADDRESS: ' + @address
  set @i=@i+1
  fetch next from cursor_emp into @emp_id, @emp_name, @address
end
close cursor_emp
deallocate cursor_emp
go
--ezzel egyenértékű SELECT megoldás
select 'ID: ' + cast(employeeid as varchar(5)) + isnull(', LASTNAME: ' + lastname, '') +
isnull(', ADDRESS: ' + address, '')
from employees order by lastname
--vagy sorszámmal
```

```
select cast(row_number() over(order by lastname) as varchar(50))+
'. ügynök: ID: ' + cast(employeeid as varchar(5)) + isnull(', LASTNAME: ' + lastname, '') +
isnull(', ADDRESS: ' + address, '')
from employees
```

**GYAKORLAT:** Valósíts meg egy kurzort, amely végigiterál az USA-beli vevőkön, és soronként kiírja a hozzájuk tartozó rendelések számát!

---

## Tranzakciókezelés

- Az alapvető tranzakciókezelési fogalmak:
  - A **tranzakciót** egy üzleti folyamat logikailag koherens műveleti sorozataként definiáljuk. A „logikailag koherens” azt jelenti, hogy a műveletek szemantikai egységet alkotnak. A tranzakciók **egymásba ágyazhatók** — például a helikoptervásárlás tranzakciója magában foglalja a vevő azonosításának és a banki átutalással történő fizetésnek a tranzakcióját.
  - Az **atomicitás**, a **konzisztencia**, az **elszigeteltség** és a **tartósság** (ACID) a tranzakciókat megvalósító rendszerekkel szemben támasztott követelmények. A rendelésfeldolgozási példánkban az atomicitás és az elszigeteltség követelményét sértettük meg.
  - Léteznek **implicit** és **explicit** (programozott) **tranzakciók**. Minden SQL DML utasítás implicit tranzakciónak minősül.
  - A T-SQL-ben a tranzakciókat a `BEGIN/COMMIT/ROLLBACK TRANSACTION` utasításokkal programozzuk. A tranzakció a `BEGIN TRANSACTION` és a `COMMIT TRANSACTION` vagy `ROLLBACK TRANSACTION` utasítás közötti összes utasításból áll. A `COMMIT` lezárja a tranzakciót, és felszabadítja az összes erőforrást (pl. táblazárakat), amelyeket a szerver a tranzakciókezeléshez használt. A `ROLLBACK` ugyanezt teszi, de előtte visszavonja a tranzakció összes módosítását. Ehhez a szerver egy kifinomult naplózási mechanizmust alkalmaz: **előreíró naplózás** (*Write-Ahead Log*, WAL). Ha nem csonkítják vagy mentik le, a tranzakciós napló nagyobb is lehet, mint maga az adatbázis.
  - Az MS SQL Serverben, ha az `XACT_ABORT` be van kapcsolva (`ON`), és a tranzakció valamelyik utasítása hibát okoz, a szerver leállítja a tranzakció végrehajtását, és automatikus `ROLLBACK`-et hajt végre.

```
--atomicitás bemutatója xact_abort nélkül
set xact_abort off
delete t2
go
begin tran
    insert t2 (id, t1_id) values (10, 1)
    insert t2 (id, t1_id) values (11, 2) --idegen kulcs megszorítás megsértése
    insert t2 (id, t1_id) values (12, 3)
commit tran
go
select * from t2
-- 10   1
-- 12   3
-- atomicitás NEM teljesült

set xact_abort on
delete t2
go
begin tran
    insert t2 (id, t1_id) values (10, 1)
    insert t2 (id, t1_id) values (11, 2) --idegen kulcs megszorítás megsértése
    insert t2 (id, t1_id) values (12, 3)
commit tran
```

```
go
select * from t2
-- (üres)
-- atomicitás teljesült
```

- Az **egymásba ágyazott tranzakciók** technikailag több `BEGIN TRANSACTION` utasítást jelentenek. Egyetlen `ROLLBACK` az összes megkezdett tranzakciót visszavonja:

```
begin tran
    print @@trancount  --1
    begin tran
        print @@trancount  --2
    commit tran
    print @@trancount  --1
commit tran
print @@trancount  --0

begin tran
    print @@trancount  --1
    begin tran
        print @@trancount  --2
rollback tran
print @@trancount  --0
```

- Súlyos **programozási hiba**, ha egy tranzakciót nem zárunk le sem `COMMIT`-tal, sem `ROLLBACK`-kel. A lezáratlan tranzakció folyamatosan fogyasztja a szerver erőforrásait, és végül megbénítja a rendszert. A `@@TRANCOUNT` globális változóval ellenőrizhető, hogy az aktuális kapcsolatban van-e lezáratlan tranzakció.

**PÉLDA:** A rendelésfeldolgozó szkript hiányosságainak orvoslásához csomagoljuk tárolt eljárásba, és adjunk hozzá `TRY/CATCH` hibakezelést és tranzakciós támogatást:

```
go
create procedure sp_example (@emp_id int)
as
set xact_abort on --automatikus visszagörgetés bármilyen hiba esetén
begin tran
begin try
    declare @i int
    select @i=count(*) from employees where EmployeeID=@emp_id
    if @i>0 print 'Employee found: ' + cast(@emp_id as varchar(50))
    else print 'Not found: ' + cast(@emp_id as varchar(50))
    if @i>0 begin
        update employees set salary=salary*1.1 where EmployeeID=@emp_id
        commit tran
        print 'Salary successfully increased'
    end else begin
        print 'Rolling back transaction'
        rollback tran
    end
end try
begin catch
    print 'OTHER ERROR: '+ ERROR_MESSAGE() + ' (' + cast(ERROR_NUMBER() as varchar(20)) + ')'
    print 'Rolling back transaction'
    rollback tran
end catch
go
--teszt
exec sp_example 12  --Not found: 12 Rolling back transaction
exec sp_example 11  --Employee found: 11 Salary successfully increased
```

- További olvasnivaló a tranzakciókezelésről:
  - https://docs.microsoft.com/en-us/sql/t-sql/language-elements/control-of-flow
  - https://learn.microsoft.com/en-us/sql/t-sql/language-elements/transactions-transact-sql?view=sql-server-ver16
  - https://www.sqlshack.com/transactions-in-sql-server-for-beginners/

- **GYAKORLAT:** Adj tranzakciós támogatást a saját képzéskezelő tárolt eljárásodhoz, és teszteld különböző hibákra!

---

[2] A hallgatók feladatainak megoldásaiért keresd meg a szerzőt.

# 2. Laza csatolás triggerek és ütemezett feladatok alapján

## Problémaforgatókönyv

Az új rendeléseket egy harmadik fél kereskedelmi és vállalatirányítási alkalmazás tárolja az Orders és az Orderitems táblákban. Ez az alkalmazás nem rendelkezik nyílt API-val, vagy más okból nem generál szolgáltatási szintű eseményeket. Ezért a Northwind Traders Kft. jelenlegi rendelésfeldolgozási munkafolyamatában a kereskedelmi osztály e-mailben (vagy más manuális üzenetváltási módon) tájékoztatja a Szállítási és Logisztikai (SzL) osztályt az új vagy módosult rendelésekről. Cégünk felelős az SzL-menedzsment informatikai támogatásáért. Az SzL-osztály vezetője minden reggel részletes napi munkaterveket készít az egyes részlegek számára a kereskedelmi osztálytól érkezett e-mailek alapján, ehhez a mi szoftverünket használja. Mind a kereskedelmi osztály, mind az SzL ugyanazt a Northwind SQL Server adatbázist használja.

Feladatunk: mentesítsük a kereskedelmi és SzL-munkatársakat az e-mailek kézi megírásától és az adatok kézi bevitelétől azáltal, hogy a rendelésfeldolgozási munkafolyamatot a lehető legnagyobb mértékben automatizáljuk.

---

## Megoldás

Mivel a kereskedelmi rendszer „fekete doboz”, adatbázisszintű eseményekre kell támaszkodnunk. Minden rendelés létrehozásakor vagy módosításakor futtatni kell a szükséges (meglehetősen összetett) logikát az adatbázison, amely létrehozza vagy módosítja a szükséges rekordokat az SzL-táblákban (pl. Products). Így az e-mailek írása és feldolgozása szükségtelenné válik.

Ugyanakkor létfontosságú, hogy a megoldásunk semmilyen módon ne avatkozzon be a kereskedelmi rendszer működésébe. Nem lassíthatja le jelentősen a rendelésmentési folyamatot, és az SzL-oldalon esetlegesen előforduló feldolgozási hibák nem gyűrűzhetnek vissza a kereskedelmi rendszerbe.

Ennek érdekében laza csatolást alkalmazunk. A rendelések INSERT és UPDATE eseményeit csak egy speciális táblába naplózzuk egy trigger segítségével, majd ezeket az eseményeket egy ütemezett feladat (job) dolgozza fel kötegesen. A feladat nyomon követi az egyes események feldolgozásának állapotát és eredményét is. Mivel az esemény feldolgozása aszinkron módon, háttérfolyamatként történik, a feldolgozási hiba nem jelenik meg hibaként a kereskedelmi rendszerben.

> **Megjegyzés:** A *trigger* egy speciális tárolt eljárás, amelyet az adatbázis-kezelő rendszer automatikusan meghív adatbázis-események — például tábla-INSERT, UPDATE vagy DELETE — hatására.

**A rendszer áttekintése:**

```
[Orders tábla] → [INSERT trigger] → [eseménynapló tábla] → [rendelésfeldolgozó feladat] → [gyártási táblák]
```

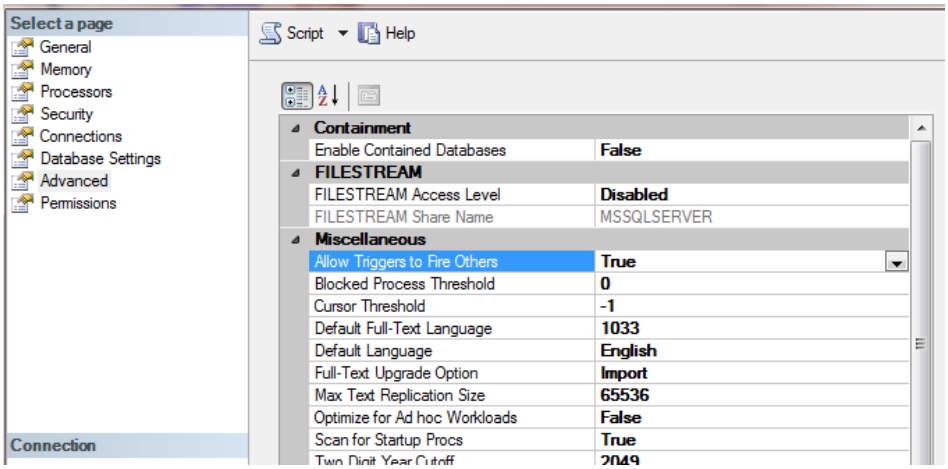
---

## A triggerek rövid áttekintése

A triggerek a szerveren tárolt speciális eljárások, amelyek automatikusan futnak, ha egy előre meghatározott feltétel teljesül. Az SQL Server a trigger eseménye szerint a következő triggertípusokat támogatja:

- **DML triggerek** (tábla szintű triggerek): DELETE, INSERT vagy UPDATE művelet végrehajtásakor aktiválódnak.
- **DDL triggerek** (adatbázisszintű triggerek): az adatbázis sémájának változásakor aktiválódnak, pl. tábla létrehozásakor.
- **Logon (bejelentkezési) triggerek** (szerverszintű triggerek): a bejelentkezési folyamat hitelesítési fázisának befejeztével aktiválódnak.

Most a DML triggerekre összpontosítunk. A trigger definíciója tartalmazza a céltáblát, a trigger eseményét (DELETE, INSERT vagy UPDATE) és a végrehajtási módot. Az SQL Server a következő végrehajtási módokat támogatja:




- **AFTER:** a megadott SQL utasítás sikeres végrehajtása *után* aktiválódik. Ez azt jelenti, hogy az összes esetleges ellenőrző (CHECK) megszorítás, más megszorítás, valamint a DML utasításhoz kapcsolódó kaszkádos frissítés/törlés sikeresen lefutott. Ugyanazon objektumra több trigger is elhelyezhető, akár azonos típusú is — például két INSERT trigger. Ebben az esetben a végrehajtási sorrend a trigger tulajdonságainak beállításával befolyásolható.
- **INSTEAD OF:** a DML utasítás egyáltalán nem hajtódik végre, csak a trigger.

A DML utasítás által módosított rekordok speciális virtuális táblákon keresztül érhetők el a trigger kódjából. Az SQL Server a következő két virtuális táblát biztosítja:

- **`deleted`:** DELETE trigger esetén a törölt rekordokat, UPDATE trigger esetén az *eredeti* (régi) rekordokat tartalmazza. Az UPDATE logikailag egy törlés és egy beszúrás sorozataként értelmezhető. INSERT trigger esetén a `deleted` tábla üres.
- **`inserted`:** INSERT utasítással beszúrt rekordokat, illetve UPDATE trigger esetén az *új* rekordokat tartalmazza. DELETE trigger esetén az `inserted` tábla üres.

Az SQL Server az INSERT vagy UPDATE triggerekben elérhető **`update`**`(mezőnév)` függvényt is támogatja, amely `true` értéket ad vissza, ha a DML utasítás módosította a megadott mezőt. A mező nem lehet számított oszlop.

Ha a trigger hibát dob, a DML utasítás visszagörgetődik (ROLLBACK).

Egy trigger olyan kódot is futtathat, amely más triggereket vagy akár önmagát hívja meg rekurzív módon, MS SQL Serveren legfeljebb 32 szintig. Ezt a funkciót a szerver `nested triggers` (egymásba ágyazott triggerek) beállítása szabályozza.

**Mikor ajánlott DML triggert alkalmazni?**

- **Adminisztrációs célok:** napló vezetése, vagy a módosított rekordok régi értékeinek megőrzése archív (történeti) táblákban.
- **Adatintegritási szabályok érvényesítése**, amelyek az üzleti logikából következnek, és meghaladják az egyszerű elsődleges kulcs-, idegen kulcs- vagy CHECK-megszorítások hatókörét. Példa a Northwind adatbázisban:
  - Nem küldünk nehéz csomagokat külföldre. Ezért visszautasítjuk azokat a rendeléseket, amelyeknél a csomag tömege meghaladja a 200 kg-ot, és a ShipCountry nem USA. Ez megvalósítható egy INSERT AFTER vagy INSERT INSTEAD OF triggerrel az Orders táblán. Az ilyen ellenőrzéseket természetesen az ügyfélalkalmazásba is be kell építeni, azonban az adatbázisszintű integritásvédelem megelőzheti az alkalmazáshibákat vagy a hackelést.
- **Üzleti munkafolyamatok automatizálása.** Példák a Northwind adatbázisban:
  - Automatikus e-mail küldése a vevőnek, amikor a szállítás dátuma meghatározásra kerül, azaz amikor egy rendelés ShippedDate mezője beállításra kerül (UPDATE trigger).
  - Automatikus megrendelés küldése a nagykereskedelmi szállítónak, amikor egy termék UnitsInStock értéke a ReorderLevel alá esik (UPDATE vagy INSERT trigger).
  - A Products tábla UnitsInStock mezőjének automatikus frissítése, amikor egy kapcsolódó Order Details rekord quantity mezője megváltozik (UPDATE trigger).

**GYAKORLAT:** Írj UPDATE triggert az Order Details táblára! Amikor a mennyiség megváltozik, frissítsd a termék UnitsInStock értékét! Feltételezheted, hogy egyszerre csak egy Order Details rekord frissül.

**GYAKORLAT:** Feltételezd, hogy a fenti feladatban egyszerre több Order Details rekord is frissülhet!

> **FIGYELMEZTETÉS:** A triggerek „csendben” működnek, és komoly problémák adódhatnak, ha megfeledkeznek róluk. Például ha a rendszergazda az Order Items táblákat egy biztonsági másolatból egy UPDATE utasítással állítja vissza anélkül, hogy először letiltaná a triggert...

További példák SQL Server triggerekre: http://sqlhints.com/2016/02/28/inserted-and-deleted-logical-tables-in-sql-server/

---

## Szoros csatolás

Az alábbi példában egy olyan új INSERT triggert hozunk létre az Orders táblán, amely sokáig fut és kivételt dob — ezzel megbénítja a rendelésmentési folyamatot.

```
drop trigger tr_demo_bad
go
create trigger tr_demo_bad on orders for insert as
declare @orderid int
select @orderid=OrderID from inserted
print 'New order ID: ' + cast(@orderid as varchar(50))
waitfor delay '00:00:10'  --10 s
select 1/0  --hibát generálunk
go
--1. teszt: az utolsó két sorral kikommentezve
insert Orders (CustomerID, OrderDate) values ('AROUT', GETDATE())
--tábla visszaállítása
delete Orders where CustomerID='AROUT' and EmployeeID is null
--2. teszt: a trigger újbóli létrehozása, az utolsó sorokkal kikommentezve
insert Orders (CustomerID, OrderDate) values ('AROUT', GETDATE())
--sokáig kell várnunk, de nincs hiba
--3. teszt: a trigger újbóli létrehozása, minden sorral
insert Orders (CustomerID, OrderDate) values ('AROUT', GETDATE())
```

```
--sokáig kell várnunk, majd megkapjuk az üzenetet:
-- 'New order ID: 11094
-- Msg 8134, Level 16, State 1, Procedure tr_demo_bad, Line 6 [...]
-- Divide by zero error encountered. The statement has been terminated.'
select * from Orders where CustomerID='AROUT' and EmployeeID is null
--nincs ilyen rekord, mert
--az insert utasítás visszagörgetésre került -> a kereskedelmi rendszer működése megszakadt
```

Ez pontosan az, amit el kell kerülnünk. A *szoros csatolás* helyett *laza csatolást* valósítunk meg.

---

## A lazán csatolt rendszer

Az alapötlet: a trigger csak az eseményeket menti egy naplótáblába. A táblát ezután egy tárolt eljárás dolgozza fel.

### A naplótábla és a trigger

A trigger az `inserted` és `deleted` virtuális táblákat használja. Ez a trigger több rekordot érintő INSERT és UPDATE műveleteket is kezelni tud.

```
--a naplótábla
go
--drop table order_log
go
create table order_log (
    event_id int IDENTITY (1, 1) primary key ,
    event_type varchar(50) NOT NULL ,
    order_id int NOT NULL ,
    orderitem_id int NULL ,
    status int NOT NULL default(0),
    time_created datetime NOT NULL default(getdate()) ,
    time_process_begin datetime NULL ,
    time_process_end datetime NULL ,
    process_duration as datediff(second, time_process_begin, time_process_end)
)
go
drop trigger tr_log_order
go
create trigger tr_log_order ON Orders for insert, update as
declare @orderid int
select @orderid=orderid from inserted  --FIGYELEM: ha több rekord szerepel az inserted táblában,
↪  csak az utolsó értékét kapjuk meg – ez hibás működéshez vezethet
print 'OrderID of the LAST record: ' + cast(@orderid as varchar(50))
if update(orderid) begin  --ha az orderid megváltozott, akkor ez egy INSERT
    print 'Warning: new order'
    insert order_log (event_type, order_id)  --a status és time_created az alapértelmezett értéket
     ↪  kapja
        select 'new order', orderid from inserted
end else if update(shipaddress) or update(shipcity) begin  --a szállítási cím megváltozott
    print 'Warning: address changed'
    insert order_log (event_type, order_id)
        select 'address changed', orderid from inserted
end else begin  --egyéb változás
    print 'Warning: other change'
    insert order_log (event_type, order_id)
        select 'other change', orderid from inserted
end
go

--1. teszt
insert Orders (CustomerID, OrderDate) values ('AROUT', GETDATE())
select * from order_log
--egy új rekordot kaptunk a naplótáblában

--2. teszt
```

```sql
insert Orders (CustomerID, OrderDate) values ('AROUT', GETDATE()), ('HANAR', GETDATE())
select * from order_log
--két új rekordot kaptunk a naplótáblában

--3. teszt
update Orders set ShipVia = 3 where OrderID in (11097, 11096)  --a 2. teszt rendelésazonosítói
select * from order_log
--két új 'other change' típusú rekordot kaptunk

--táblák visszaállítása
delete Orders where CustomerID in ('AROUT', 'HANAR') and EmployeeID is null
delete order_log
```

### A tárolt eljárás az új rendelések feldolgozásához

Tegyük fel, hogy az új rendelés tételeinek rögzítése a rendelési rekord létrehozása után következik.

```sql
--egyszerű tárolt eljárás, amely feldolgoz egy új rendelést
--és 0-t ad vissza, ha az összes tétel hibamentesen rögzíthető a készletbe
--egyben bemutatja az OUTPUT paraméterek használatát
drop proc sp_commit_new_order_to_inventory
go
create procedure sp_commit_new_order_to_inventory
@orderid int,
@result int output
as
begin try
    update products set unitsInStock = unitsInStock - od.quantity
    from products p inner join [Order Details] od on od.ProductID=p.ProductID
    where od.OrderID=@orderid
    set @result=0
end try
begin catch
    print ' Inventory error: '+ ERROR_MESSAGE() + ' (' + cast(ERROR_NUMBER() as varchar(20)) + ')'
    set @result=1
end catch
go

--teszt
select * from order_log  --11097
select * from Products where ProductID=10  --unitsinstock =31
select * from Products where ProductID=9   --unitsinstock =29
insert [Order Details] (orderid, productid, quantity, UnitPrice, Discount)
values (11097, 9, 10, 30, 0),(11097, 10, 40, 30, 0)  --a második tétel hibát fog okozni
go
declare @res int
exec sp_commit_new_order_to_inventory 11097, @res output
print @res
exec sp_commit_new_order_to_inventory 11096, @res output
print @res
go
--ellenőrzés: az unitsinstock értéke nem változott (OK)
select * from Products where ProductID=10  --unitsinstock =31
select * from Products where ProductID=9   --unitsinstock =29
```

### A tárolt eljárás az eseménynapló feldolgozásához

Mivel az esemény típusától függően teljesen eltérő műveleteket kell elvégezni, kurzorral iterálunk az eseménynapló rekordjain.

```sql
--tárolt eljárás az order_log feldolgozásához
--drop proc sp_order_process
go
create proc sp_order_process as
declare @event_id int, @event_type varchar(50), @order_id int, @result int
declare cursor_events cursor forward_only static
```

```sql
    for
    select  event_id, event_type, order_id
    from order_log where status=0  --csak a feldolgozatlan eseményeket kezeljük

set xact_abort on
set nocount on
open cursor_events
fetch next from cursor_events into @event_id, @event_type, @order_id
while @@fetch_status = 0
begin
    print 'Processing event ID=' + cast(@event_id as varchar(10)) + ', Order ID=' +
cast(@order_id as varchar(10))
    update order_log set time_process_begin=getdate() where event_id=@event_id
    begin tran
    set @result = null
    if @event_type = 'new order' begin
        print '  Processing new order...'
        exec sp_commit_new_order_to_inventory @order_id, @result output
    end else if @event_type = 'address changed' begin
        print '  Processing address changed...'
        waitfor delay '00:00:01'  --csak szimuláljuk a többi eseménytípus feldolgozását
        set @result=0
    end else if @event_type = 'other change' begin
        print '  Processing other change...'
        waitfor delay '00:00:01'
        set @result=0
    end else begin
        print '  Unknown event type...'
        waitfor delay '00:00:01'
        set @result=1
    end

    if @result=0 begin
        print 'Event processing OK'
        commit tran
    end else begin
        print 'Event processing failed'
        rollback tran
    end
    print ''
    update order_log set time_process_end=getdate(),
        status=case when @result=0 then 2 else 1 end
        where event_id=@event_id
    fetch next from cursor_events into @event_id, @event_type, @order_id
end
close cursor_events deallocate cursor_events
go

--teszt
update order_log set status=0
select *from orders where EmployeeID is null
select * from order_log
exec dbo.sp_order_process
select * from order_log
```

### Várható kimenet:

```
Processing event ID=5, Order ID=11097
 Processing new order...
 Inventory error: The UPDATE statement conflicted with the CHECK constraint etc.
Event processing failed

Processing event ID=6, Order ID=11096
 Processing new order...
Event processing OK
```

**Az eseménynaplót feldolgozó ütemezett feladat**

Az ütemezett feladatot (job) az SSMS grafikus felületével valósítjuk meg, és a Job Activity Monitor segítségével ellenőrizzük a működését.

---

**GYAKORLAT:** Hozz létre egy lazán csatolt megoldást, amely figyeli a Products táblát, és új készletutánpótlást rendel a kapcsolódó Supplier-től (Szállítótól), amikor az `UnitsInStock` értéke a `ReorderLevel` mezőben meghatározott érték alá esik!

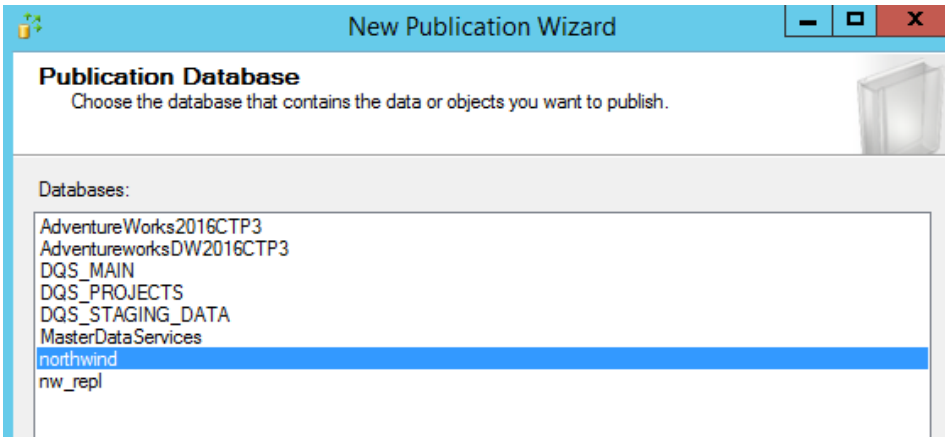
# 3. Replikáció, naplószállítás és feladatátvétel

A laza csatolásos esettanulmányban valójában a *replikáció* egy speciális formáját valósítottuk meg.

## Replikációs fogalmak és architektúra

A *replika* az eredeti másolatát jelenti. Az adatbázis-technológiában a replikációt arra használják, hogy egy vagy több forrásból automatikusan másolják és szinkronizálják az adatokat. A replikációs metafora összetevői a következők:

- A **közzétevő** (publisher) az az entitás (adatbázis-szerver), amelynek adatait meg kell osztani. Az adatok **kiadványokba** (publications) vannak szervezve. Minden kiadvány egy vagy több **cikket** (articles) tartalmaz. A cikkek lehetnek táblák, táblarészletek, tárolt eljárások vagy más adatbázis-objektumok.

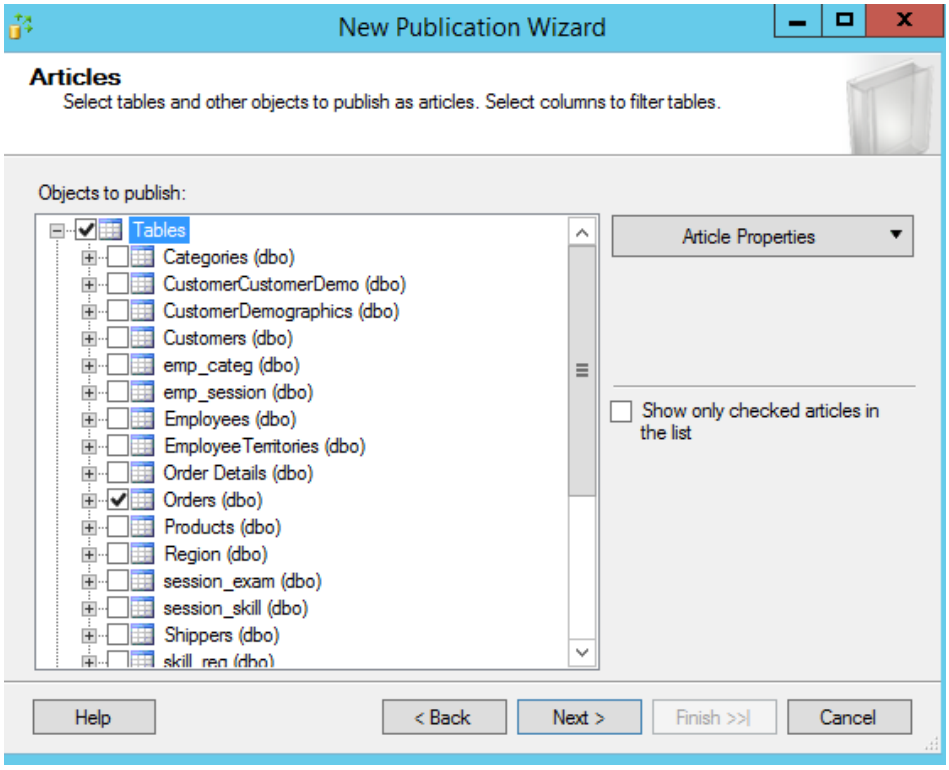



- Az **előfizető** (subscriber) az az entitás, amely feliratkozik a kiadványokra. Lehet ugyanaz az adatbázis-szerver, mint a közzétevő, vagy egy másik szerver. Több előfizető — esetleg különböző szervereken — is feliratkozhat ugyanarra a kiadványra.
- A **feliratkozásnak** (subscription) különböző módozatai lehetnek a másolás módjával és ütemezésével kapcsolatban. Tartalmazhat adatszűrőket vagy menet közbeni adatátalakítási lépéseket. Lehet **push** (tolt, azaz a közzétevő indítja) vagy **pull** (húzott, azaz az előfizető indítja) feliratkozás. A pull feliratkozásokat az előfizető hozza létre és ütemezi.

A replikáció fő típusai és alkalmazási esetei a következők:

- **Pillanatkép-replikáció** (Snapshot replication). A cikkek kezdeti pillanatfelvétele után a másolt objektumokat az előfizetőnél minden adatfrissítéskor töröljük és újra létrehozzuk — függetlenül attól, hogy volt-e változás a kiadványban. Akkor alkalmazható, ha egy

OLTP adatbázis részeit másolni kell egy adattárházba vagy egy riportszerverbe, munkaidőn kívüli ütemezéssel. Példa: a nap folyamán keletkezett adatok éjszakai küldése ('adott időpontra vonatkozó riportok'). Mivel több feliratkozás is mutathat ugyanarra a céladatbázisra, a replikáció ETL (extract–transform–load, azaz kinyerés–átalakítás–betöltés) mechanizmusként is használható, ha az összes közzétevő SQL Server technológiát alkalmaz. **FIGYELEM:** a törlés/újralétrehozás mechanizmusa miatt az előfizetőnél lévő objektumok ideiglenesen nem lesznek elérhetők, ami rövid szolgáltatáskimaradást okozhat. Az adatok késleltetését is el kell viselni. *Minden más alábbi replikációs típust pillanatfelvétellel inicializálnak.*




- **Tranzakciós replikáció** (Transactional replication). Csak a megváltozott adatokat másolja, és közel valós idejű adatszinkronizációra konfigurálható. Szükség van elsődleges kulcsra a replikált táblákon. Akkor alkalmazható, ha a nagy késleltetés problémát jelent, és nem akarjuk a változatlan adatokat is mozgatni. Példa: egy vállalat külső telephelyei saját helyi szerverekkel rendelkeznek, amelyek a központi adatbázis csak a működésükhöz szükséges részeit tárolják. Ez az architektúra javítja a helyszíni autonómiát és az információs rendszer megbízhatóságát.
- **Összefésülő replikáció** (Merge replication). Ebben a sémában az előfizetők maguk is módosíthatják az adatokat, és egy mechanizmus gondoskodik arról, hogy ezek a változások minden félhez eljussanak, és konzisztens adatbázissá olvadjanak össze. Az összefésülési folyamat konfliktusfeloldást is magában foglalhat. A rekordok több szerveren keresztüli azonosítása érdekében a tábláknak UNIQUEIDENTIFIER típusú, ROWGUIDCOL tulajdonságú mezővel kell rendelkezniük.[1] Tipikus eset az utazó üzletkötők esete, akik nem folyamatosan kapcsolódnak a központi adatbázishoz. A helyi adatbázisukban végzett módosítások automatikusan összefésülődnek a többi módosítással.

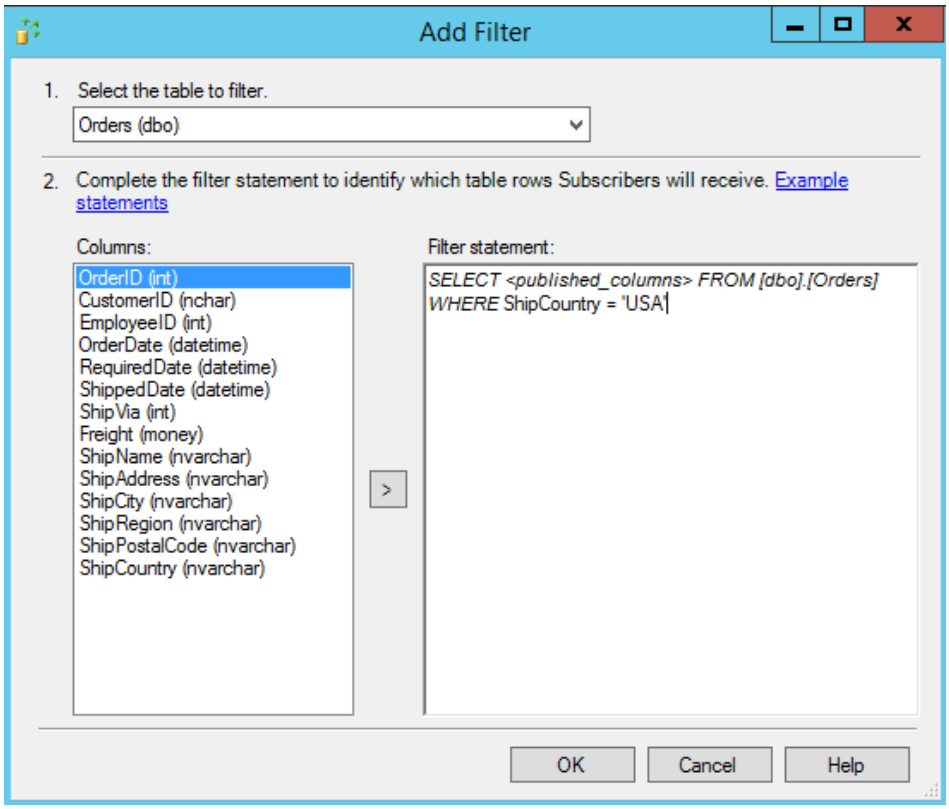
A replikáció típusát mindig a kiadvány határozza meg.

[1] Úgy működik, mint egy identity oszlop, de kezdőérték nélkül és globálisan egyedi értékekkel. Használat: `CREATE TABLE test (my_guid uniqueidentifier DEFAULT NEWSEQUENTIALID() ROWGUIDCOL, …)`

A replikációs technológia ütemezett feladatokon alapul, összefésülő replikáció esetén pedig a közzétett cikkek triggerein is.

A replikáció **nem ajánlott**, ha egy teljes adatbázis pontos másolatát kell karbantartani egy távoli szerveren a rendelkezésre állás és megbízhatóság javítása érdekében — erre a célra a naplószállítás és az SQL Server 2012-től elérhető Always On technológia egyszerűbb és robusztusabb megoldást kínál.

**FIGYELEM:** bár a replikáció az adatok több példányát tartja karban, nem helyettesíti a biztonsági mentést és a katasztrófa utáni helyreállítás tervezését.

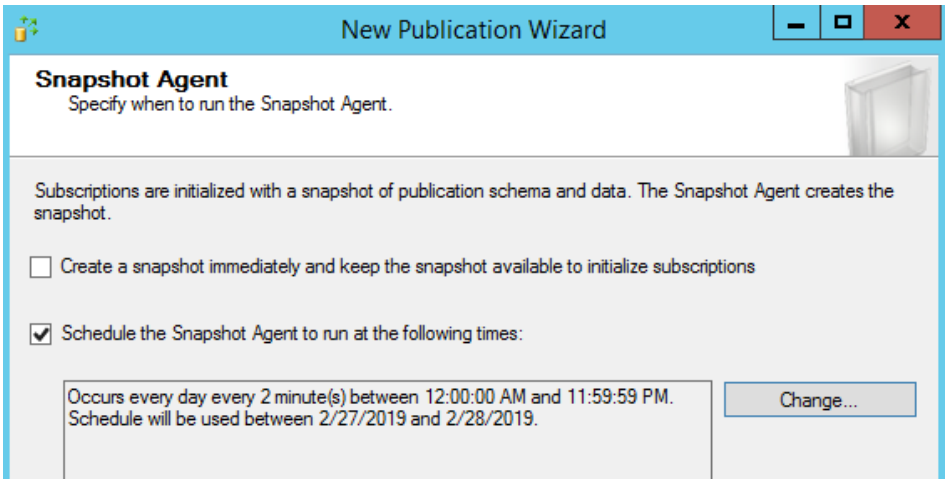



A replikációban három **szerverszerepkör** van: a *közzétevő*, a *terjesztő* (distributor) és az *előfizető*. Mindhárom szerepet ugyanaz a szerverpéldány is betöltheti, ha egy helyi adatbázist egy másik helyi adatbázisba replikálnak, vagy különböző példányok tölthetik be őket. Reálisabb konfigurációkban a terjesztő szerepét egy másik szerver veszi át, hogy tehermentesítse a közzétevőt. **A terjesztő** felelős a megváltozott adatok tárolásáért egy megosztott mappában vagy egy terjesztési adatbázisban, és az adatok továbbításáért az előfizetőknek. Egy közzétevőhöz csak egy terjesztő tartozhat, de egy terjesztő több előfizetőt is kiszolgálhat.

**Kétirányú** vagy **frissíthető** replikáció esetén az előfizető is módosíthatja a közzétevőn lévő adatokat.

Az SQL Server különféle **ügynökökkel** (agents) valósítja meg a replikációs funkcionalitást. Ezek az ügynökök az SQL Server Agent által felügyelt feladatokként futnak.

- **A pillanatkép-ügynök** (Snapshot agent) előállítja a pillanatfelvételt, és a terjesztőnél lévő **pillanatfelvétel-mappában** tárolja. Az ügynök a bcp (bulk copy) segédprogramot használja a kiadvány cikkeinek másolásához.




- **A terjesztési ügynök** (Distribution agent). Pillanatkép-replikációban ez az ügynök alkalmazza a pillanatfelvételt az előfizetőre; tranzakciós replikációban futtatja a **terjesztési adatbázisban** tárolt tranzakciókat az előfizetőn. A terjesztési adatbázis egy rendszer-adatbázis a terjesztőn, ezért a System Databases csoportban találod meg. Ez az ügynök pull feliratkozásnál az előfizetőn, push feliratkozásnál a közzétevőn fut.
- **A naplóolvasó ügynök** (Log reader agent) olvassa a tranzakciós naplót a közzétevőnél, és a releváns tranzakciókat átmásolja a terjesztési adatbázisba. Csak tranzakciós replikációban használják. Minden közzétett adatbázishoz külön ügynök tartozik.
- **A sorkezelő ügynök** (Queue reader agent) az előfizetők által végzett módosításokat másolja a közzétevőre *frissíthető* vagy *kétirányú* tranzakciós replikáció esetén.
- **Az összefésülési ügynök** (Merge agent) összefésüli az előfizetőnél és a közzétevőnél egyaránt keletkező növekményes változásokat összefésülő replikációban. A változások felismerése triggereken alapul. Az összefésülési ügynököt tranzakciós replikációban nem használják.

Pull feliratkozás kivételével az összes ügynök a terjesztőn fut.

---

## Pillanatkép-replikáció

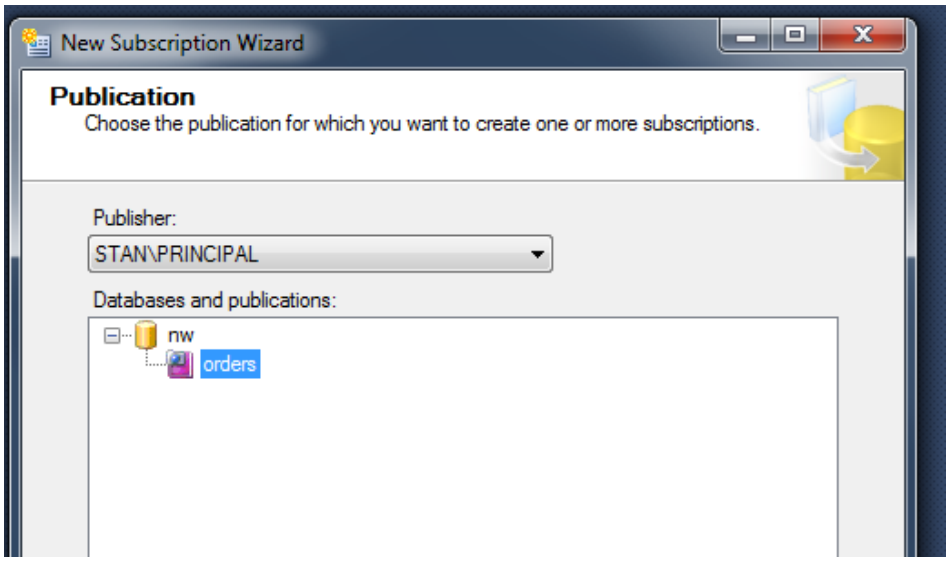
Először állítsd be a tesztkörnyezetet. A replikációs példák megfelelő működéséhez három 'nevesített' MS SQL Server példányra van szükség, amelyek ugyanazon a szervergépen vannak telepítve. Ezeket PRIM, SECOND és THIRD névvel kell ellátni.

**Forgatókönyv:** az amerikai ügyfelek rendeléseit szeretnénk átreplikálni ugyanazon a szerveren (Principal) lévő másik adatbázisba, hogy éjszaka frissítsük a riportadattárházat. Pillanatkép-replikációt választunk.

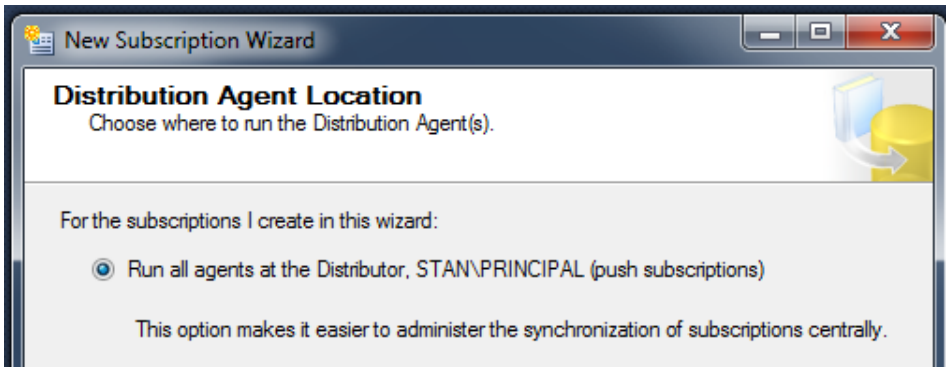
### A kiadvány létrehozása

1. Csatlakozz a Principal szerverhez, és hozz létre egy új adatbázist Northwind névvel, ha még nem létezik. Futtasd a Northwind adatbázis-létrehozó szkriptjét (dump).
2. Az alábbi hiba elkerülése érdekében: *„Cannot execute as the database principal because the principal "dbo" does not exist, …"* — amely a Northwind adatbázis hiányos visszaállításából ered — futtasd a következő szkriptet a Northwind adatbázison:

```
EXEC sp_changedbowner 'sa';
ALTER AUTHORIZATION ON DATABASE::northwind TO sa;
```

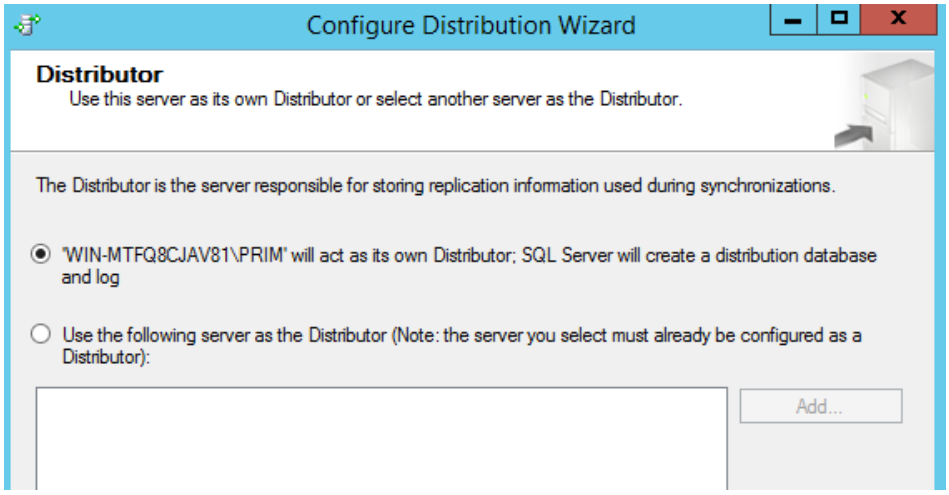



3. Hozz létre egy másik, üres adatbázist `nw_repl` névvel, szintén a PRIM szerveren.




4. Indítsd el az SQL Server Agent-et, ha nem fut.
5. A Replikáció → Terjesztés konfigurálása menüpont kiválasztásával állítsd be a PRIM-et saját terjesztőjeként. A pillanatfelvétel-mappa a következő lesz:

   `C:\Program Files\Microsoft SQL Server\MSSQL14.PRIM\MSSQL\ReplData`

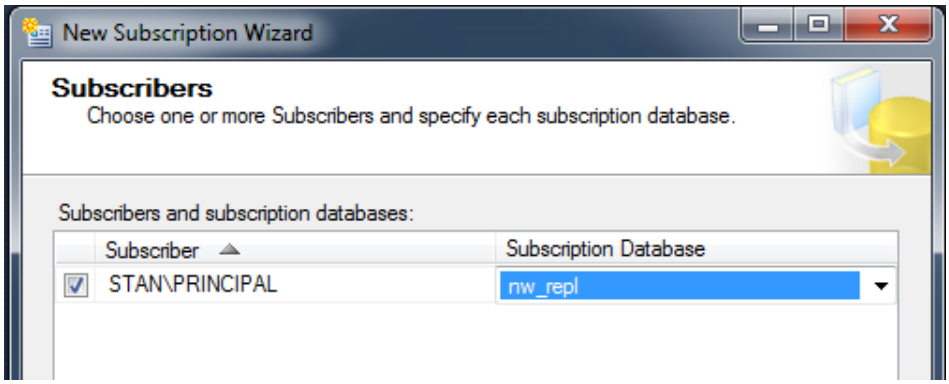



6. Indítsd el az Új kiadvány varázslót, és válaszd ki a northwind adatbázist kiadványi adatbá-

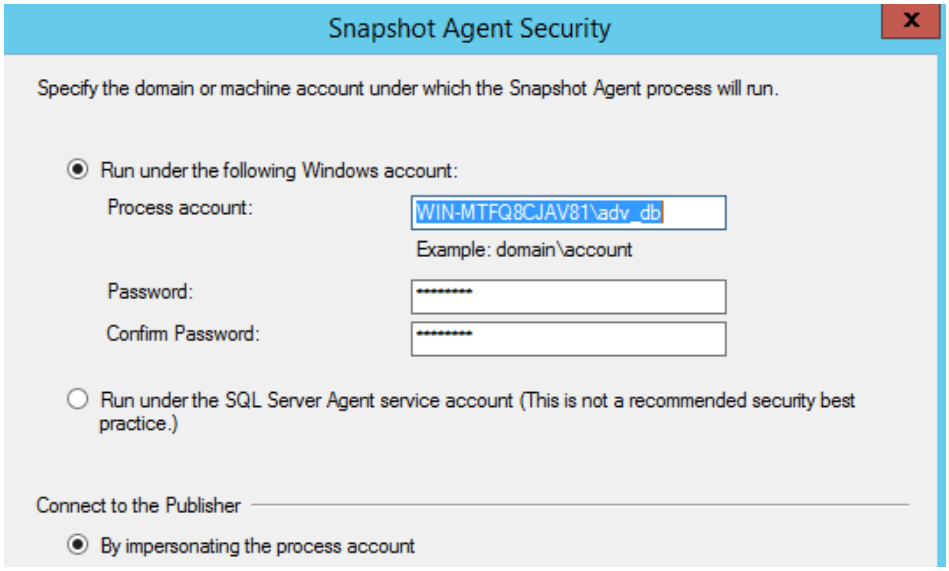
zisként.

7. A következő panelen válaszd a Pillanatkép-kiadvány típust, majd válaszd ki az Orders táblát a kiadvány egyetlen cikkeként.
8. A Táblaszűrő párbeszédpanelen kattints a Hozzáadás gombra.
9. Egészítsd ki a szűrési feltételt az amerikai ügyfelek szűréséhez:

```
SELECT <published_columns> FROM [dbo].[Orders]
WHERE ShipCountry = 'USA'
```

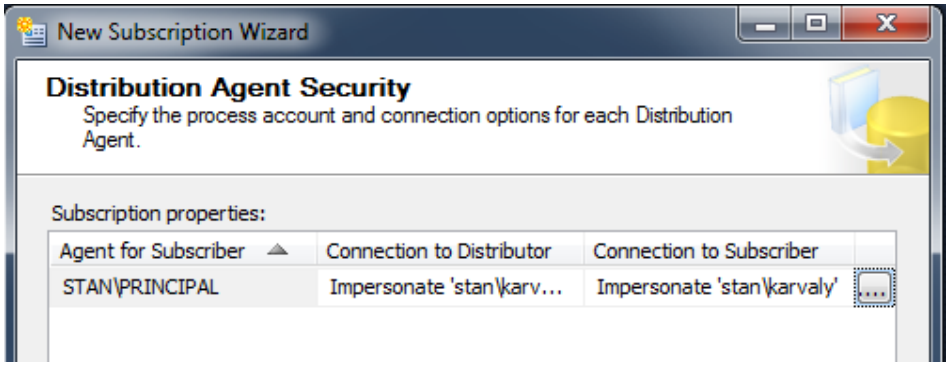



10. Add meg, hogy a pillanatkép-ügynök kétpercenként fusson — ehhez kattints a Módosítás gombra a következő panelen. **Megjegyzés:** *ezt a rövid időintervallumot kizárólag bemutató célból alkalmazzuk. Az ügynök a bcp segédprogramot használja, amely az egész táblát zárolja a másolás ideje alatt, hogy garantálja az adatkonzisztenciát. Ez azt jelenti, hogy minden más tranzakció blokkolódik, amely módosítani szeretné a táblát. Éles rendszerekben a pillanatfelvétel-generálást a teljesítményre tekintettel kell ütemezni.*
11. Meg kell adni az ügynök biztonsági beállításait. A Security settings fülön add meg a saját felhasználói azonosítódat, és állítsd be a folyamatfiók megszemélyesítését. Ez a legegyszerűbb módja annak, hogy a pillanatkép-ügynök írási jogosultsággal rendelkezzen a pillanatfelvétel-mappára. **Megjegyzés:** gondolhatod, hogy miért futtatja az SQL Server Agent szolgáltatás vagy a SQLSERVER szolgáltatás alacsony jogosultságú, nem rendszergazdai Windows-fiókkal. Ennek az az oka, hogy így egy sikeres DBMS-feltörés esetén a támadónak kevesebb esélye van a teljes szerver kompromittálására.



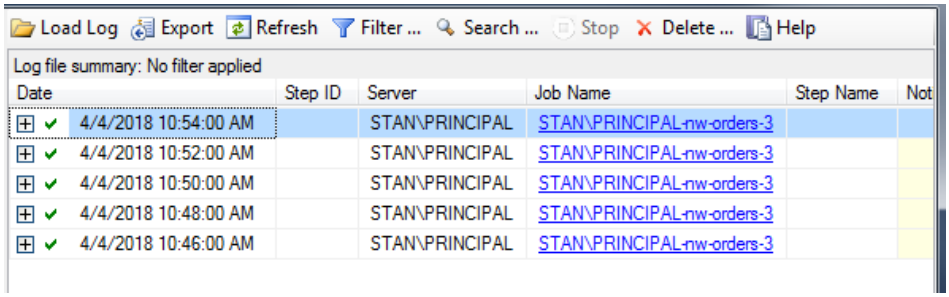
12. A következő panelen válaszd a kiadvány létrehozását, és nevezd el ‘orders’-nek.

### A kiadvány ellenőrzése




Az új kiadvány megjelenik a Helyi kiadványok csoportban. A pillanatfelvétel-mappák itt jönnek létre:

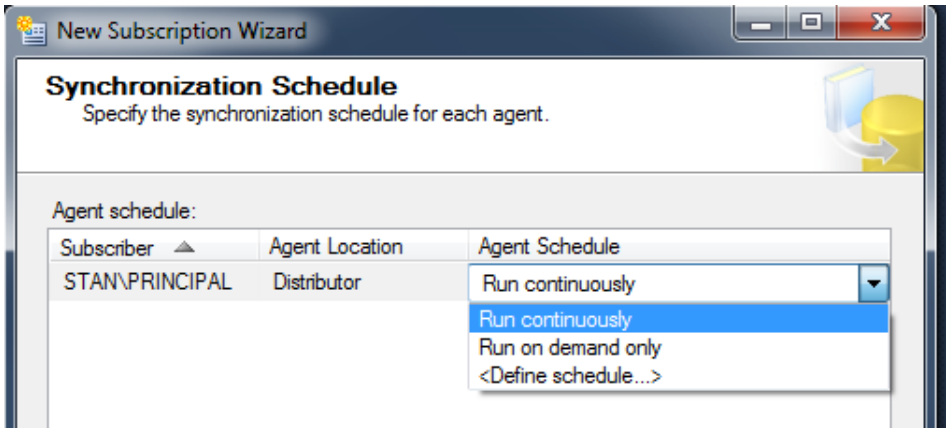
`C:\Program Files\Microsoft SQL Server\MSSQL14.PRIM\MSSQL\repldata\unc\WIN-MTFQ8CJAV81$PRIM_NORTHWIND_TEST`




Tényleges pillanatfelvétel azonban még nem keletkezett, mert még nincs feliratkozás, amelyet inicializálni kellene.

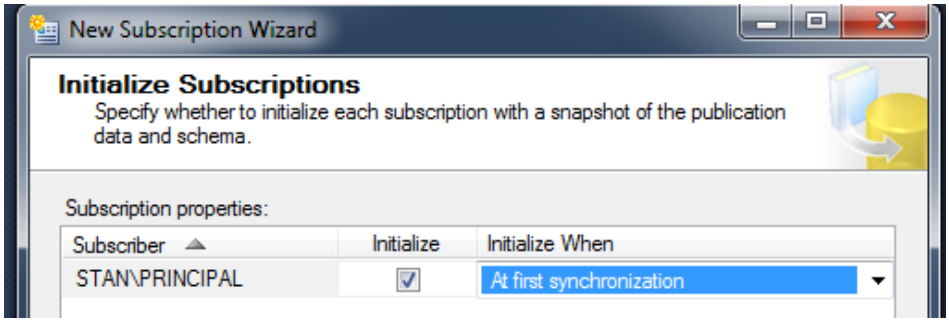
Ellenőrizd az SQL Server Agent feladatok között megjelenő új feladatot. A feladatelőzmények (job history) mutatja, hogy az ügynök rendszeresen lefut.

### Push feliratkozás létrehozása




1. Indítsd el az Új feliratkozás varázslót az orders kiadvány helyi menüjéből. Válaszd ki forrásként az orders kiadványt.

2. A következő panelen válaszd az összes ügynök futtatását a terjesztőn (push feliratkozás).
3. Add meg ugyanazt a szervert előfizetőként, és az `nw_repl` adatbázist feliratkozási adatbázisként.
4. A terjesztési ügynök biztonságát állítsd be ugyanúgy, mint a pillanatkép-ügynöknél.
5. Az ütemezésnél válaszd a Folyamatos futtatás lehetőséget a minimális késleltetés érdekében.

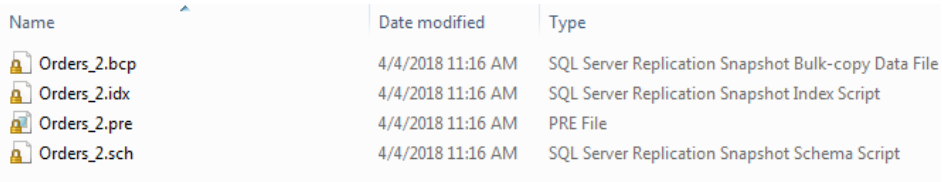



6. A következő panelen válaszd az inicializálást. Ez elkészíti az első pillanatfelvételt a terjesztési mappában.
7. Fejezd be az új feliratkozás létrehozását.

**Megjegyzés:** a feliratkozások és kiadványok tulajdonságait később is módosíthatod, ha a helyi menüből a Tulajdonságok pontot választod.

### A feliratkozás ellenőrzése

1. Keresd meg az Orders táblát az `nw_repl` adatbázisban, és ellenőrizd, hogy tartalmazza-e az USA-s rendeléseket.



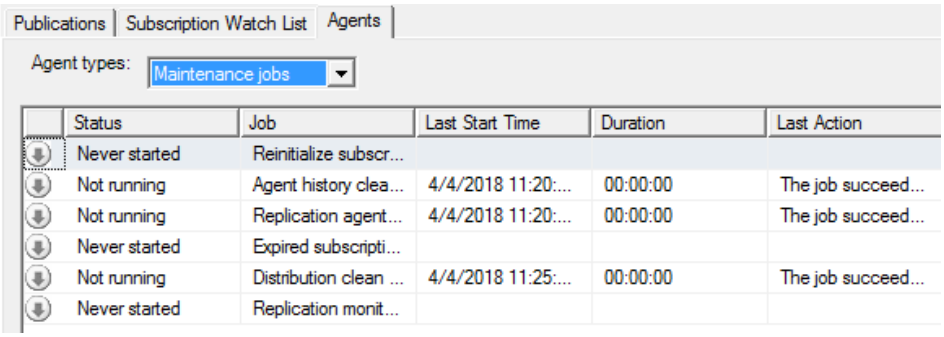
| Name | Date modified | Type |
|---|---|---|
| Orders_2.bcp | 4/4/2018 11:16 AM | SQL Server Replication Snapshot Bulk-copy Data File |
| Orders_2.idx | 4/4/2018 11:16 AM | SQL Server Replication Snapshot Index Script |
| Orders_2.pre | 4/4/2018 11:16 AM | PRE File |
| Orders_2.sch | 4/4/2018 11:16 AM | SQL Server Replication Snapshot Schema Script |

2. Ellenőrizd a pillanatfelvétel-mappa tartalmát. A **Bulk copy** egy gyors módszer, amellyel az SQL Server adatokat szúr be közvetlenül az adatbázisfájlokba. A mappában a következő fájlokat fogod látni:

| Fájlnév | Típus |
|---|---|
| Orders_2.bcp | SQL Server Replication Snapshot Bulk-copy Data File |
| Orders_2.idx | SQL Server Replication Snapshot Index Script |
| Orders_2.pre | PRE File |
| Orders_2.sch | SQL Server Replication Snapshot Schema Script |

3. Indítsd el a replikációs monitort az új feliratkozás helyi menüjéből, és ellenőrizd a kiadvány és a feliratkozás állapotát. Az aktív replikációs ügynököket is itt tekintheted meg.
4. Nyisd meg a Feladat-aktivitás monitort. A terjesztési ügynök új feladatként jelenik meg a listában, amely folyamatosan fut.

5. Módosítsd UPDATE utasítással az Orders tábla első USA-s rekordját a közzétevőnél. A változás rövid időn belül (kb. 30 másodperccel) megjelenik a replikált táblában, miután a pillanatkép-ügynök legközelebb lefut.




6. Végül töröld a feliratkozást és a kiadványt. Ehhez válaszd a Replikáció csoport helyi menüjéből a Szkriptek generálása lehetőséget, add meg a 'Törléshez...' opciót, majd futtasd a szkriptet egy szerkesztőben. Alternatívaként egyenként is törölheted az objektumokat.

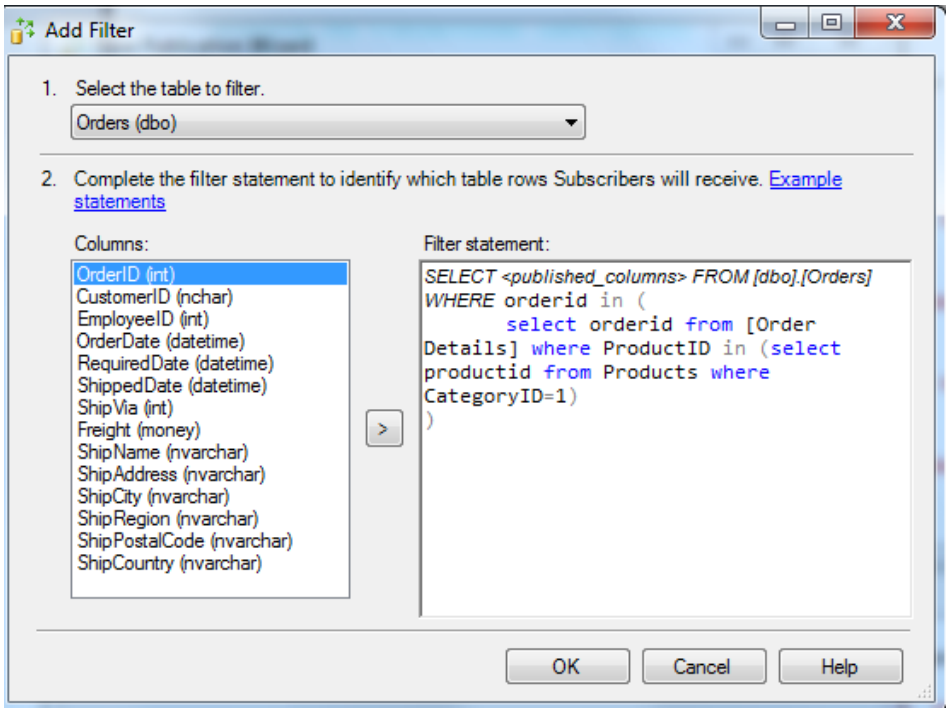
7. Az előfizetőnél lévő replikált táblák nem törlődnek a feliratkozás törlésekor, ezért az Orders táblát kézzel töröld az `nw_repl` adatbázisból.

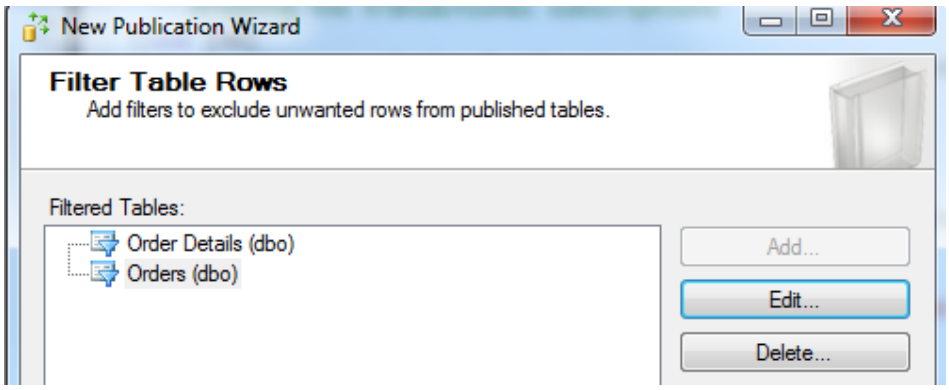
   **GYAKORLAT:** hozz létre push pillanatkép-kiadványt az `nw_repl` táblába, amely azokat az alkalmazottakat másolja az Employees táblából, akiknek a beosztása „Sales Representative". Ellenőrizd a helyes működést, majd töröld az összes kapcsolódó objektumot.

---

## Tranzakciós replikáció

**Forgatókönyv:** közel valós idejű (ütemezett) laza csatolást szeretnénk létrehozni a központi Northwind adatbázis és egy külső telephely között, amely csak az 'Italok' (Beverages, CategoryID=1) kategóriájú termékekkel foglalkozik. Csak az italos rendeléseket és rendeléstételeket replikáljuk tranzakciós replikáción keresztül.




Ezt a demót először egyetlen szerveren (Principal) valósítjuk meg. A fenti szűrési feltétel a következőképpen definiálható:

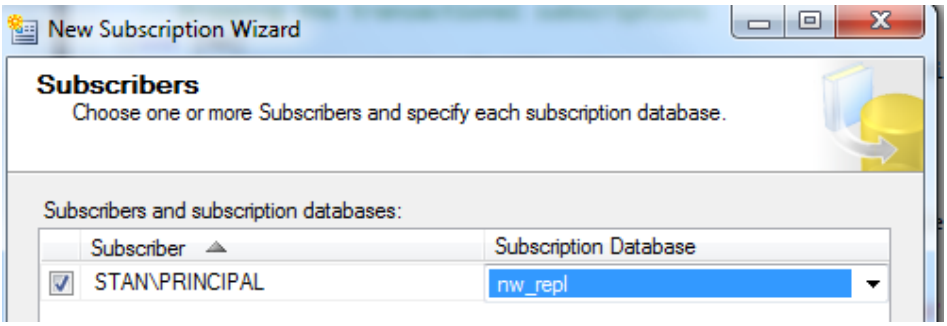
```
select * from [Order Details] where ProductID in (select productid from Products where
CategoryID=1)

select * from orders where orderid in (
   select orderid from [Order Details] where ProductID in (select productid from
Products where CategoryID=1)
)
```

1. A replikációs konfigurációt visszaállíthatod a Terjesztés és közzététel letiltása lehetőség kiválasztásával a Replikáció menüből.
2. A kiadványtípus párbeszédpanelen add meg a kiadvány típusaként a Tranzakciós típust.
3. A Cikkek panelen válaszd ki az Orders és az Order Details táblákat.
4. A Táblaszűrő panelen add hozzá a szűrőt mindkét táblához egyenként, a fenti lekérdezések WHERE részének bemásolásával. Az Orders táblához:

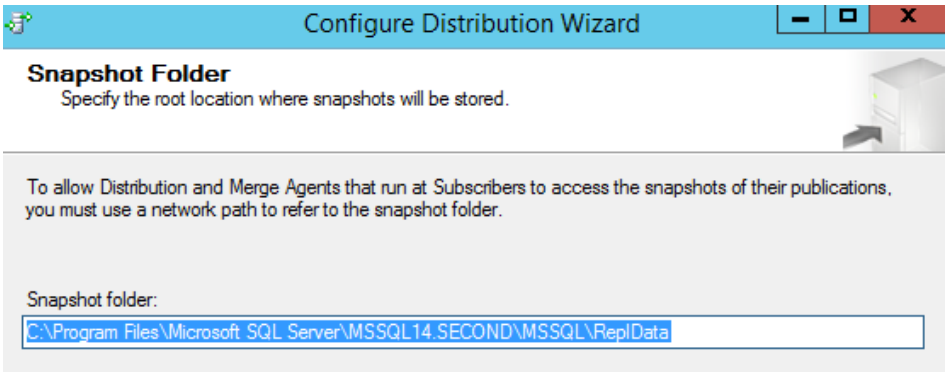
```
SELECT <published_columns> FROM [dbo].[Orders]
WHERE orderid in (
      select orderid from [Order Details] where ProductID in (select productid from
```

```
Products where CategoryID=1)
)
```

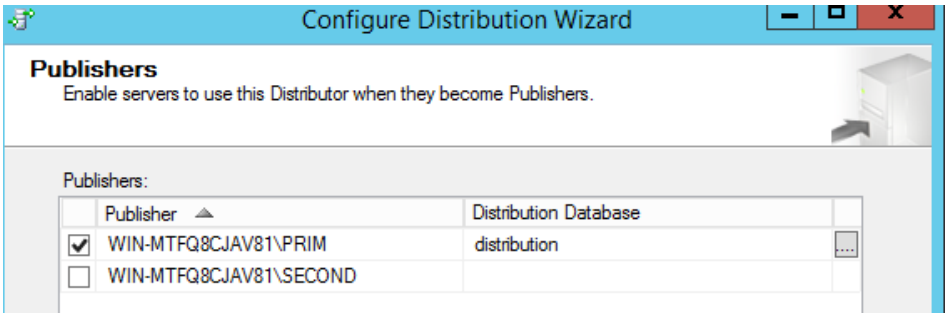
5. Mindkét táblára legyen definiálva a szűrő.
6. A következő panelen válaszd a Pillanatfelvétel azonnali létrehozása lehetőséget.
7. A következő paneleken állítsd be az ügynökök biztonságát az előző demóhoz hasonlóan.
8. Nevezd el a kiadványt `nw_trans` névvel, és fejezd be a kiadvány létrehozását.



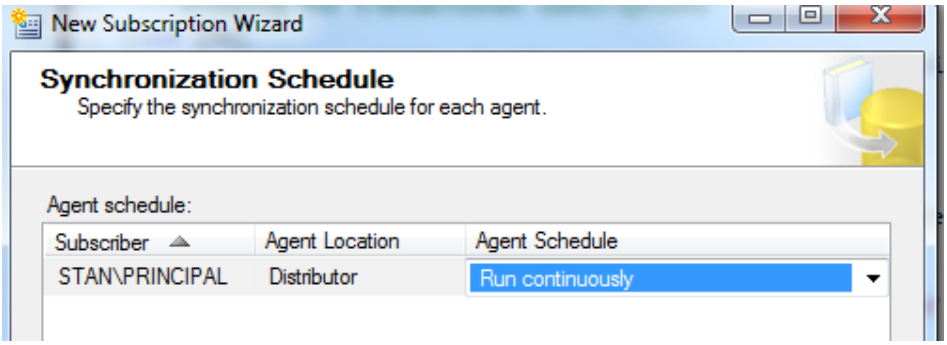

9. A kiadvány helyi menüjéből válaszd az Új feliratkozás lehetőséget.
10. A következő panelen válaszd az összes ügynök futtatását a Terjesztőn (push feliratkozás).




11. Válaszd ki az `nw_repl` adatbázist feliratkozási adatbázisként.
12. A terjesztési ügynök biztonságát állítsd be az előző demóhoz hasonlóan.
13. A naplóolvasó és terjesztési ügynök ütemezéseként add meg a Folyamatos futtatást.
14. Az Inicializálás feliratkozások panelen válaszd az Azonnal lehetőséget.




15. Teszteld a tranzakciós replikáció helyes működését. Frissítsd az employeeID mezőt az Orders tábla első rekordjában a northwind adatbázisban, majd válaszd ki ugyanazt a rekordot az `nw_repl` adatbázisban. A módosult értéknek 10 másodpercen belül meg kell jelennie.

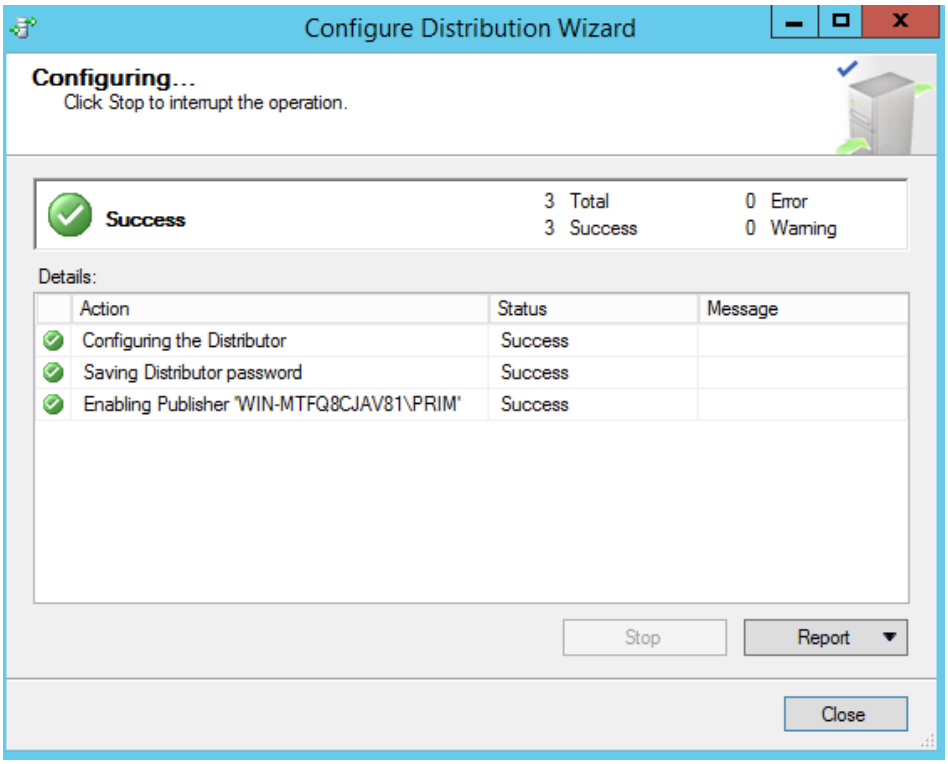


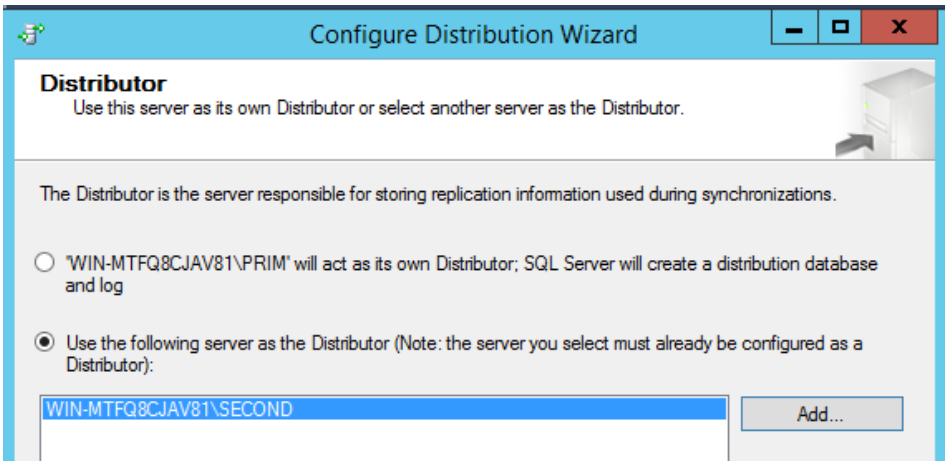

16. Ellenőrizd a replikációs ügynökök működését.
17. Takarítsd el a replikációt az összes replikációs objektum törlésével.

    **GYAKORLAT:** valósítsd meg a laza csatolásos rendelésfeldolgozási forgatókönyvet tranzakciós replikáció segítségével a Products, Orders és Order Details táblákon. Használd a fenti háromszerveres konfigurációt, ahol SECOND a terjesztő. Tételezzük fel, hogy a logisztikai részlegnek saját adatbázisa van, amely a THIRD szerveren fut.

    1. Adj hozzá egy 'status' nevű mezőt az Orders táblához a PRIM példány northwind adatbázisában, alapértelmezett értéke legyen 0.
    2. Módosítsd a meglévő rendelések állapotát 0-ról 2-re. Nem akarjuk az összes meglévő rendelést feldolgozni.
    3. Replikáld a három táblát az `nw_repl` adatbázisba tranzakciós replikáció segítségével.
    4. Mivel az előfizető módosíthatja a replikált rekordokat, és a kereskedési alkalmazás az Orders táblán *soha* nem frissíti a status mezőt, ezt a mezőt az előfizetőn arra fogjuk használni, hogy a rendelési rekordok feldolgozási állapotát naplózzuk — hasonlóan az esettanulmány megoldásához. Így elkerülünk egy extra naplótáblát. Részletek:
       - a. Tudjuk, hogy az új rendelések alapértelmezetten 0 állapotúak lesznek. A módosított rendelések megjelöléséhez egy frissítési triggert alkalmazhatunk a közzétevőn, amely a már meglévő rekord állapotát 1-re változtatja.
       - b. Az előfizetőn futó feladat 0 vagy 1 állapotú rendelési rekordokat dolgoz fel, és sikeres feldolgozás esetén az állapotot 2-re állítja.[2]
    5. Valósítsd meg és teszteld a megoldást.
    6. Töröld a kiadványt és a feliratkozást.

[2] Ez jól működik, ha a rendelést csak egyszer frissítik a közzétevő oldalon.




## Replikáció különálló szerverek között[3]

A következő demóban ugyanezt a tranzakciós replikációt valósítjuk meg egy reálisabb forgatókönyvben: a PRIM a közzétevő, a SECOND a terjesztő, a THIRD az előfizető. Még reálisabb esetben nemcsak különálló szerverpéldányok lennének, hanem különálló szervergépeken is futna mindegyik — ilyen konfigurációt a laborban nem tudunk megvalósítani.

[3] Az SQL Server 2019 és Windows 10 rendszerek egy hibája miatt a varázsló az `sp_adddistributor` segítségével — távoli terjesztő esetén — üres jelszót használ a megadott helyett, és a 21768-as kivételt adja vissza („The password specified for the @password parameter must be the same when the procedure is executed at the Publisher and at the Distributor"). A demo futtatásához add meg manuálisan a jelszót az `sp_adddistributor` hívásában: `use master / exec sp_adddistributor @distributor = 'DESKTOP-....\SECOND', @password = 'type password here'`

### A terjesztő konfigurálása

1. Indítsd el a SECOND és THIRD példányokat, és az SQL Server Agent-et a SECOND példányon.
2. Állítsd vissza a replikációs konfigurációt a PRIM-en a Terjesztés és közzététel letiltása lehetőség kiválasztásával a Replikáció menüből.
3. A SECOND példányon a Replikáció helyi menüjéből válaszd a Terjesztés konfigurálása lehetőséget, és fogadd el az első választást. Ez létrehozza a terjesztési adatbázist a SECOND-on.
4. A következő panelen add meg, hogy a Server Agent-et manuálisan indítjuk.
5. A következő panelen fogadd el a pillanatfelvétel-mappa elérési útját:

   `C:\Program Files\Microsoft SQL Server\MSSQL14.SECOND\MSSQL\ReplData`
6. A következő panelen fogadd el az alapértelmezéseket a terjesztési adatbázis helyéhez és nevéhez.
7. Ezután meg kell adni, hogy mely közzétevő szerverek jogosultak ezt a terjesztési adatbázist használni. A következő panelen szüntesd meg a SECOND kijelölését (mivel a SECOND nem fog közzétenni), majd az Hozzáadás gombbal add hozzá a PRIM példányt.
8. A következő panelen meg kell adni egy jelszót, amelyet a PRIM közzétevők fognak használni ehhez a terjesztési adatbázishoz. Add meg ugyanazt a jelszót, amelyet a bejelentkezéshez használsz. Erre a jelszóra később szükség lesz.
9. A folyamat végén sikeresen konfigurálta a terjesztőt.

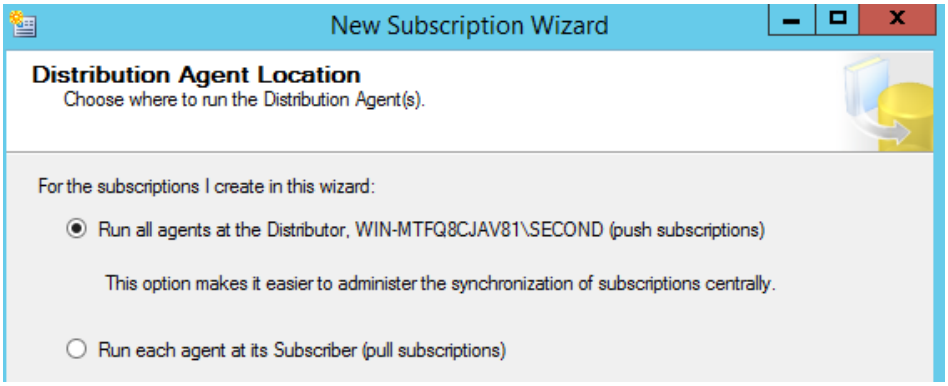
### A közzétevő konfigurálása

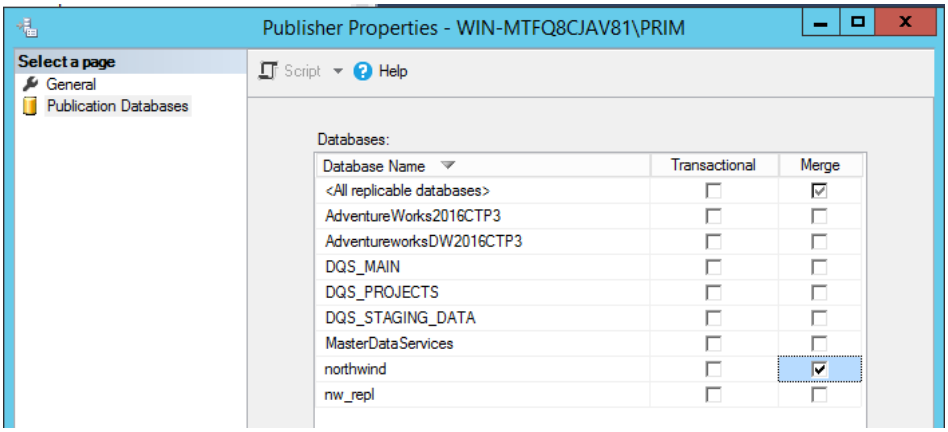
1. A PRIM konfigurálásához először letiltjuk azt terjesztőként. Ne feledd, hogy eddig a PRIM saját terjesztőjeként működött. A Replikáció helyi menüjéből válaszd a Terjesztés és közzététel letiltása lehetőséget. Miután a PRIM-et letiltottuk terjesztőként, a helyi menü megváltozik. Válaszd most a Terjesztés konfigurálása lehetőséget, és add meg a SECOND példányt a PRIM terjesztőjeként.
2. A következő panelen add meg ugyanazt a jelszót, mint korábban (a 8. lépésben).
3. A terjesztés most be van állítva.

### A kiadvány és a feliratkozás hozzáadása

1. Hozd létre a tranzakciós kiadványt a PRIM-en a szokásos módon (Orders tábla szűrő nélkül).
2. A THIRD-en ne konfiguráljuk a Terjesztési tulajdonságokat, mert a THIRD nem fog közzétenni.
3. Hozz létre új feliratkozást a THIRD példányon a Helyi feliratkozások → Hozzáadás menüponttal. Válaszd ki a PRIM-et közzétevőként, és válaszd ki az előző lépésben létrehozott kiadványt.
4. Válaszd az összes ügynök futtatása a Terjesztőn lehetőséget.
5. A varázsló a THIRD-en új adatbázisként hozhatja létre a feliratkozási adatbázist `nw_repl` névvel.

6. Teszteld a feliratkozást. Nyiss egy lekérdezésszerkesztőt a PRIM-en, és frissíts egy rekordot az Orders táblában. Nyiss egy lekérdezésszerkesztőt a THIRD-en, és ellenőrizd, hogy a módosítás 10 másodpercen belül megjelenik-e a replikált táblában.



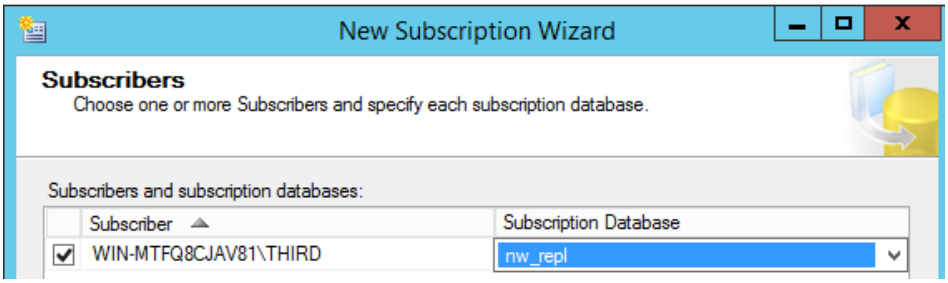

7. Töröld a kiadványt a PRIM-en és a feliratkozást a THIRD-en. Töröld a replikált táblát a THIRD-en.

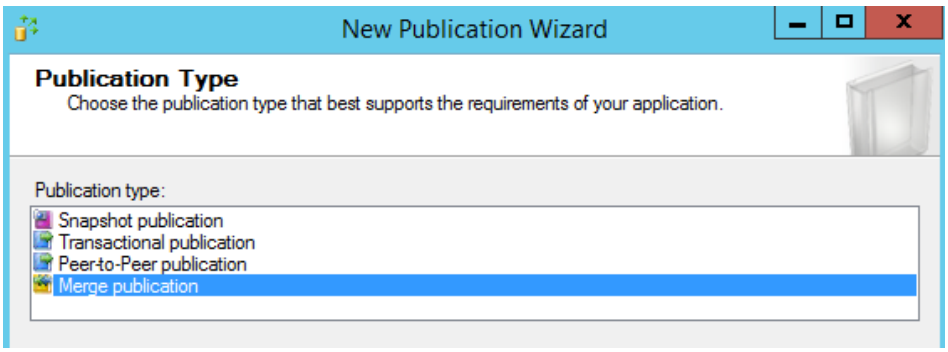
   **GYAKORLAT:** valósítsd meg a laza csatolásos rendelésfeldolgozási forgatókönyvet tranzakciós replikáció segítségével a Products, Orders és Order Details táblákon. Használd a fenti háromszerveres konfigurációt, ahol SECOND a terjesztő. Tételezzük fel, hogy a logisztikai részlegnek saját adatbázisa van, amely a THIRD szerveren fut.

   1. Adj hozzá egy ‘status’ nevű mezőt az Orders táblához a PRIM példány northwind adatbázisában, alapértelmezett értéke legyen 0.
   2. Módosítsd a meglévő rendelések állapotát 0-ról 2-re. Nem akarjuk az összes meglévő rendelést feldolgozni.
   3. Replikáld a három táblát az `nw_repl` adatbázisba tranzakciós replikáció segítségével.
   4. Mivel az előfizető módosíthatja a replikált rekordokat, és a kereskedési alkalmazás az Orders táblán *soha* nem frissíti a status mezőt, ezt a mezőt az előfizetőn a rendelési rekordok feldolgozási állapotának naplózására fogjuk használni — hasonlóan az esettanulmány megoldásához. Részletek:
      - a. Tudjuk, hogy az új rendelések alapértelmezetten 0 állapotúak lesznek. A módosított rendelések megjelöléséhez egy frissítési triggert alkalmazhatunk a közzétevőn, amely a már meglévő rekord állapotát 1-re változtatja.[4]
      - b. Az előfizetőn futó feladat 0 vagy 1 állapotú rendelési rekordokat dolgoz fel, és sikeres feldolgozás esetén az állapotot 2-re állítja.

[4] Ez jól működik, ha a rendelést csak egyszer frissítik a közzétevő oldalon.

5. Valósítsd meg és teszteld a megoldást.
6. Töröld a kiadványt és a feliratkozást.



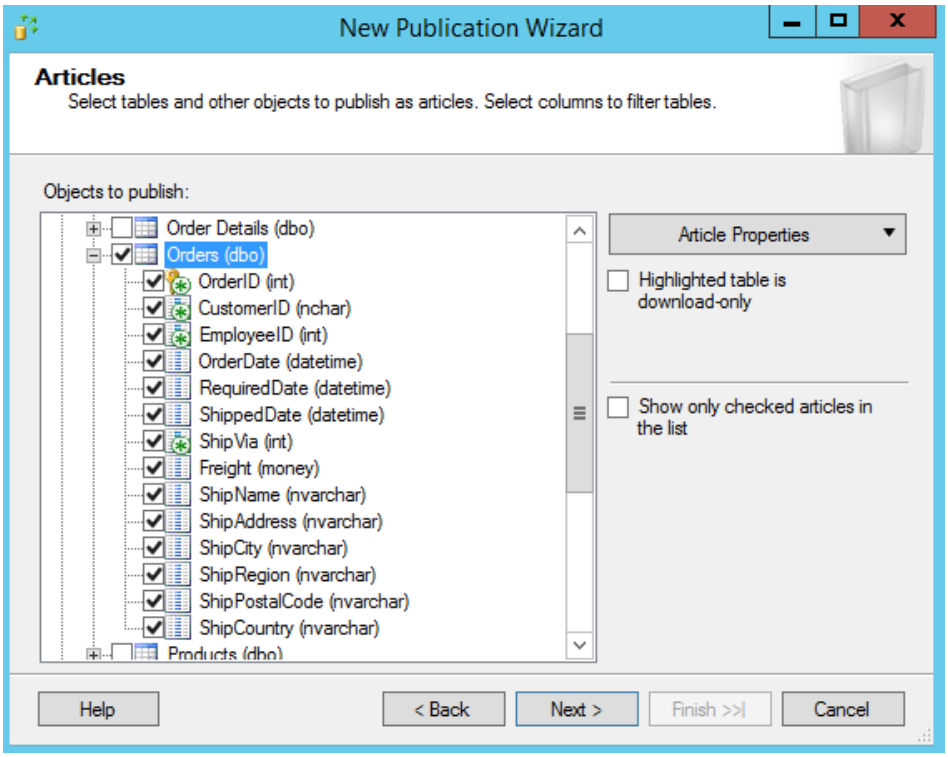




---

## Peer-to-Peer tranzakciós replikáció

Bármely csomópontba írt módosítások automatikusan és csak egyszer terjednek az összes többi csomópontra. Horizontális (scale-out) olvasási terheléselosztásra tervezték. Az azonos rekord egyidejű írásai több csomóponton — azaz írási konfliktusok — nem engedélyezettek, és kritikus hibaként kezelik őket, amelyek kézi beavatkozást igényelnek.[5]

[5] Lásd: https://learn.microsoft.com/en-us/sql/relational-databases/replication/transactional/peer-to-peer-transactional-replication




## Összefésülő replikáció

**Forgatókönyv:** az értékesítési munkatársak utaznak, és új rendeléseket vesznek fel az ügyfeleknek. Alkalmanként módosítani kell korábbi rendeléseket is, például ha megváltozik a szállítási cím. Előfordulhat az is, hogy két alkalmazott egyszerre módosítja ugyanazt a rendelést. A munkatársak nem folyamatosan kapcsolódnak az internethez. Olyan replikációs megoldást kell tervezni, amely az összes ilyen módosítást összefésüli egymással és a központi Northwind adatbázissal.

Az **összefésülő replikációban** a naplóolvasó ügynök szerepét triggerek, táblák és nézetek veszik át, amelyeket automatikusan hoznak létre minden előfizető adatbázisban és a közzétevő adatbázisban is. A triggerek az `MSmerge_*` nevű speciális rendszertáblákba naplózzák a változásokat — ezek a táblák ugyanabban az adatbázisban jönnek létre, mint a replikált tábla. Minden táblához három trigger jön létre: `MSmerge_[ins, upd, del]_*`. Vannak adatbázis szintű *sématriggerek* is, amelyek a replikált táblák sémájának változásait naplózzák.

Az összefésülő replikáció egy pillanatkép-ügynök által készített kezdeti pillanatfelvétellel indul. Alapértelmezés szerint a pillanatkép-ügynök 14 naponta fut. Ezután az összefésülési ügynök feladata hasonló a terjesztési ügynökéhez, azzal a különbséggel, hogy az összefésülő replikáció alapértelmezetten kétirányúra van konfigurálva. Ez azt jelenti, hogy az ügynök az előfizetői és a közzétevői oldalon is alkalmazza a változásokat. Minden feliratkozáshoz külön összefésülési ügynök tartozik. Az összefésülési ügynök push feliratkozás esetén a terjesztőn fut, pull feliratkozás esetén az előfizetőn.

A terjesztési adatbázis csak előzményeket és hibainformációkat tárol.

A kétirányú adatszinkronizáció támogatásához az összefésülő replikáció cikkeinek UNIQUEIDENTIFIER (GUID) típusú oszloppal kell rendelkezniük, amely hasonlóan működik az identity adattí-

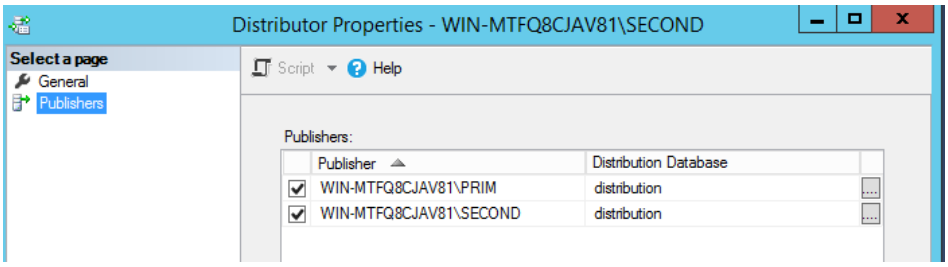
pushoz, de globálisan egyedi értékeket generál.[6] Ha ilyen oszlop nem létezik, automatikusan hozzáadódik a táblákhoz — ez azonban összeomlaszthatja a már meglévő, a táblát használó alkalmazásokat. Az új mező törlődik, amikor a kiadványt törlik.

A szinkronizálási folyamat részletei nem szerepelnek ebben a tankönyvben.[7]

### A kiadvány

**GYAKORLAT:** a megoldás kidolgozásához kövesd az alábbi lépéseket.

1. Állítsd be a SECOND példányt közzétevőként, még ha nem is fog kiadványokat közzétenni — enélkül a feloldók listája nem lesz elérhető:
2. Konfiguráld a SECOND példányt a PRIM terjesztőjeként (lásd az előző részt). Ezt követően a PRIM-en ezt kell látnod.



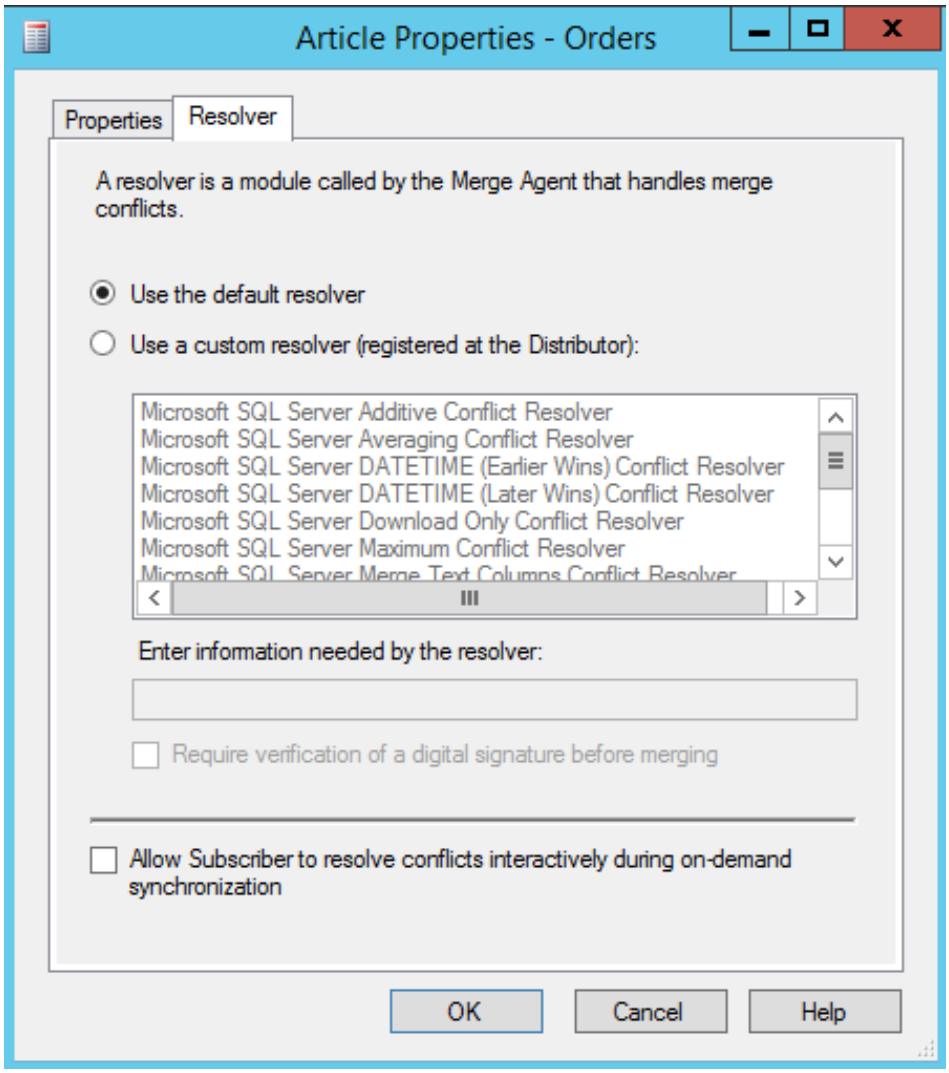

3. Engedélyezd a northwind adatbázist összefésülő replikációhoz. Válaszd ki a Kiadványi adatbázisok fület a közzétevőn (PRIM).
4. Válaszd ki a northwind adatbázist kiadványi adatbázisként, és válaszd ki a kiadvány típusát: **Összefésülő kiadvány** (Merge publication).
5. Fogadd el az alapértelmezett előfizető-típusokat, és válaszd ki az Orders táblát a kiadvány egyetlen cikkeként.
6. Minden cikkhez különféle tulajdonságokat állíthatsz be a Cikktulajdonságok kiválasztásával, beleértve a különböző konfliktustípusokhoz használandó feloldót is. Konfliktus akkor keletkezik, amikor egy ügynök olyan rekordot próbál módosítani, amelynek van egy függőben lévő módosítása. A jelenleg a szerveren regisztrált beépített feloldó modulokat az `sp_enumcustomresolvers` eljárással listázhatod. A beépített feloldók szükséges paramétereit lásd a lábjegyzetben.[8] Saját feloldót is hozzáadhatsz tárolt eljárásként vagy DLL-ként.[9] Az alapértelmezett feloldó a **„amelyik elsőként ér a közzétevőhöz, az nyer”** (first to Publisher wins) stratégiát valósítja meg:

[6] Megjegyzés: az elosztott adatbázis-rendszerek bármely megfigyelési időpontban inkonzisztensek lehetnek, de a szinkronizálási mechanizmus garantálja, hogy ha nem következik be további frissítési esemény, egy globális konzisztencia (konszenzus) elérésre kerül egy idő után. Ezt nevezzük **végleges konzisztenciának** (eventual consistency). A CAP-tétel szerint a C, A, P tulajdonságok egyszerre nem teljesíthetők teljes mértékben egyetlen elosztott adatbázis-rendszerben sem: Konzisztencia (minden olvasás a legutóbbi írást vagy hibát kapja), Rendelkezésre állás (minden olvasás tartalmaz adatokat, de lehet nem a legfrissebb), Partíciótűrés (a rendszer hálózati hibák esetén is működik) — mivel hálózati hibák előfordulhatnak, a konzisztencia és a rendelkezésre állás közötti kompromisszumot az alkalmazás követelményei alapján kell eldönteni. A bemutatott összefésülési folyamat a rendelkezésre állást részesíti előnyben a konzisztenciával szemben.

[7] Lásd: https://documentation.help/replsql/repltypes_30z7.htm

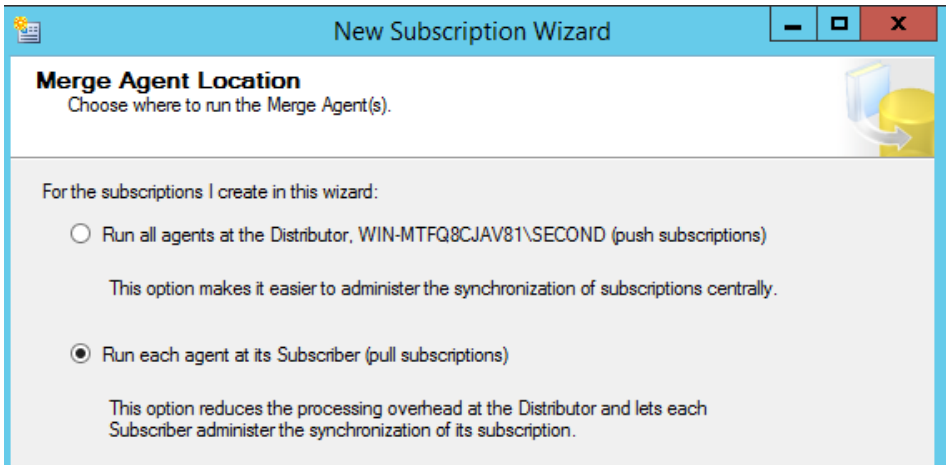
[8] Lásd: https://learn.microsoft.com/en-us/sql/relational-databases/replication/merge/advanced-merge-replication-conflict-com-based-resolvers

[9] Lásd: https://learn.microsoft.com/en-us/sql/relational-databases/replication/implement-a-custom-conflict-resolver-for-a-merge-article — Az egyéni feloldó eljárás a Közzétevőn fut, és az Összefésülési ügynök hívja meg. Lekérdezi a megváltozott rekordot az Előfizetőtől az Összefésülési ügynöktől kapott sor GUID segítségével, és egy egysoros eredményhalmazt ad vissza, amely ugyanolyan mezőkkel rendelkezik, mint az alaptábla, és a nyertes rekord értékeit tartalmazza.

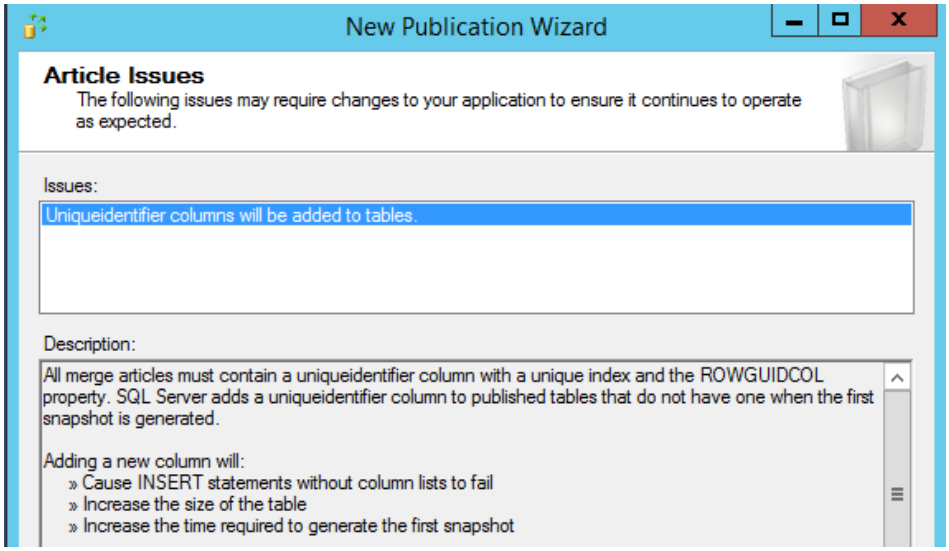



- a. Ha a konfliktus egy közzétevő és egy előfizető között lép fel, a közzétevő változása kerül elfogadásra, az előfizető értékét elutasítják.
- b. Ha a konfliktus két előfizető között lép fel pull feliratkozás esetén, az első előfizető változása kerül elfogadásra, amelyik szinkronizál a közzétevővel, a többi elutasításra kerül.

7. A következő panel figyelmeztet, hogy egy új GUID-ot adnak hozzá a táblához. Ez nem változtatja meg a tábla elsődleges kulcs és idegen kulcs megszorításait.
8. A pillanatkép-ügynök ütemezéséhez fogadd el az alapértelmezéseket, majd az ügynök biztonságát a szokásos módon állítsd be.



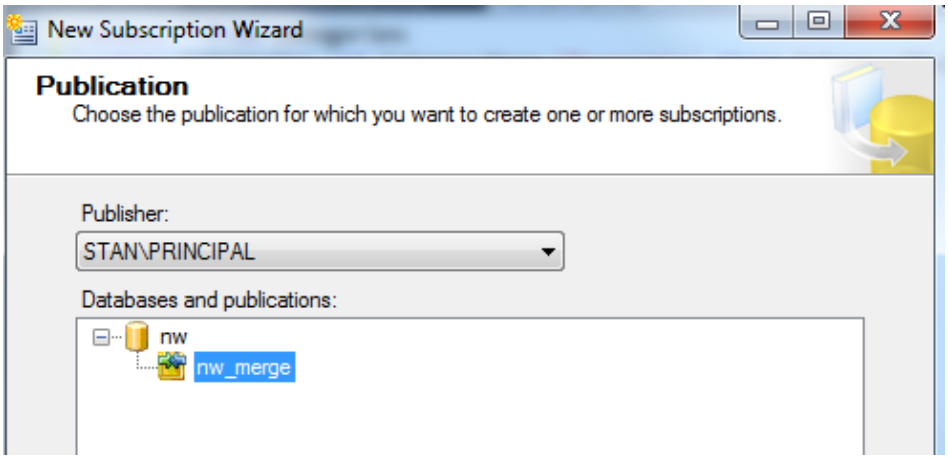

9. Nevezd el a kiadványt `nw_merge` névvel, és hozd létre. Ellenőrizd az új GUID-oszlopot és a 3 új triggert az Orders táblán.



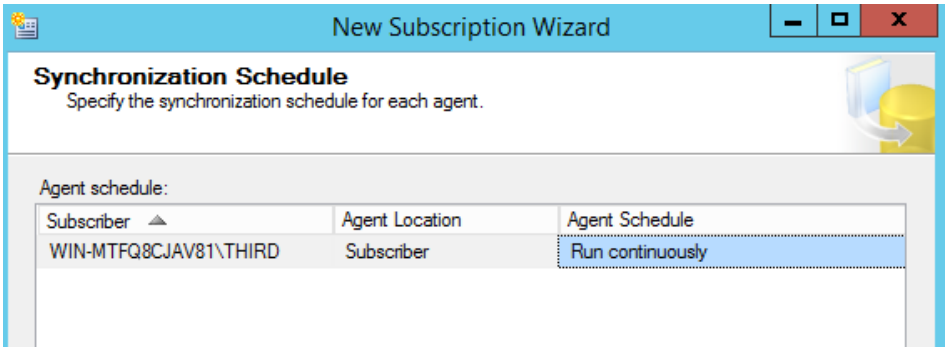

### A feliratkozás

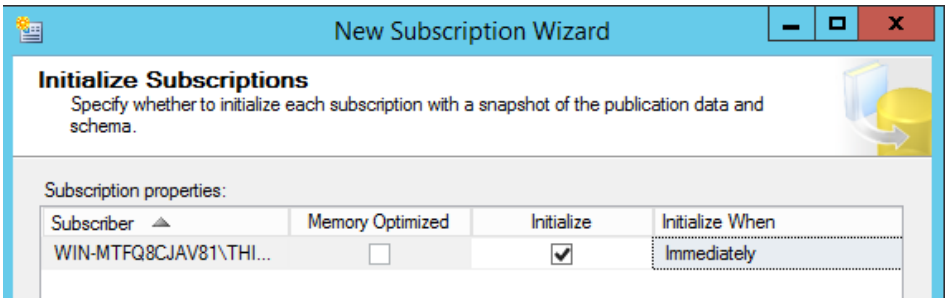
Két előfizetőt adunk hozzá az összefésülési folyamat bemutatásához.

1. Indítsd el az SQL Server Agent-et a THIRD példányon manuálisan.
2. A THIRD példányon válaszd az Új feliratkozás lehetőséget, és add meg a PRIM-et közzétevőként.
3. A következő panelen válaszd a pull feliratkozást. Ez megfelel a szokásos elvárásnak, hogy az előfizetők maguk szeretnék ütemezni a szinkronizálást.
4. Add hozzá mindkét előfizetőt: egyet a THIRD példányon, egyet a PRIM-en, újonnan létrehozott adatbázisokkal `nw_merge_1-2` névvel.




5. Az ügynökök biztonságát a szokásos módon állítsd be.
6. Mindkét összefésülési ügynök szinkronizálási ütemezését állítsd „Folyamatos futtatás”-ra. *Megjegyzés: ez csak teszt és bemutató célokra szól. Valós esetben az összefésülési folyamat meghatározott időközönként vagy igény szerint indulna el, amikor egy meghatározott esemény — például VPN-kapcsolat — létrejön az előfizetőn.*




7. Inicializáld a feliratkozásokat azonnal.
8. Összefésülő kiadványban az előfizetők újra közzétehetik azt a kiadványt, amelyre feliratkoztak (Server típusú feliratkozás), így hierarchikus feliratkozási architektúrát hozva létre. A Server típusú feliratkozás lehetővé teszi explicit numerikus prioritás hozzárendelését is minden szerverhez, amelyet konfliktus esetén használnak. Válaszd a Client típusú feliratkozást.
9. Véglegesítsd és indítsd el a replikációt.
10. Ellenőrizd, hogy a módosított értékek az összes többi előfizetőhöz és közzétevőhöz is eljutnak-e: szerkeszd meg az Orders táblát három különböző szerkesztőben. Légy türel-

mes: a szinkronizálás akár 2 percig is tarthat.[10]

11. Teszteld az alapértelmezett első-érkező-nyer (first-come-first-served) konfliktusfeloldást, amikor a két előfizető 'egyidejűleg' frissít egy rekordot (figyelembe véve, hogy az ügynökök percenként lekérdezik a naplókat). Az a módosítási kérés kerül elfogadásra, amelyik elsőként ér el a Közzétevő adatbázisba. Használd az SSMS Ablak → Vízszintes lapcsoportok hozzáadása parancsát, hogy mindhárom táblát egyszerre lásd. Az adatrácsokat a Ctrl+R megnyomásával frissítheted.

12. Az összefésülési konfliktusokat a Replikáció-konfliktus megtekintő eszközben tekintheted meg és oldhatod fel manuálisan, amely elérhető a kiadvány helyi menüjéből. Ha egy rekordot mindhárom szerver frissített — vagyis a konfliktus 3 felet érint —, két különálló konfliktusként jelenik meg, ugyanazzal a nyertessel. Elfogadhatod az alapértelmezett feloldást (Submit Winner gombra kattintva) vagy visszafordíthatod (Submit Loser).

13. A szinkronizálási folyamat aktuális állapotát a feliratkozás helyi menüjéből a Szinkronizálási állapot megtekintése lehetőséggel ellenőrizheted. A folyamatot manuálisan is elindíthatod.

14. Mindkét példányon generálj és tekints meg replikációs szkripteket a Replikáció csoport helyi menüjéből a Szkriptek generálása lehetőség kiválasztásával.

15. Töröld az összes replikációs objektumot.

**GYAKORLAT:** az új forgatókönyvben az alkalmazottak a Northwind adatbázis Products táblájának UnitPrice mezőjét frissítik, és az egyidejű módosítások összefésülnek. Az egyik alkalmazottnak alacsonyabb prioritása van, mint a másiknak. Állíts be összefésülő kiadványt a PRIM-ről pull feliratkozásokkal a SECOND-on és a THIRD-ön. A Server típusú feliratkozás segítségével állítsd be a prioritást. Használd a Lekérdezési konzolokat és a Conflict viewer eszközt annak ellenőrzésére, hogy az alacsonyabb prioritású alkalmazott mindig alulmarad, az időzítéstől függetlenül.

[10] Elosztott adatbázis-rendszerek bármely megfigyelési időpontban inkonzisztensek lehetnek, de a szinkronizálási mechanizmus garantálja, hogy ha nem következik be további frissítési esemény, egy globális konzisztencia elérésébe kerül egy idő után — ez a **végleges konzisztencia**. Lásd: https://www.bmc.com/blogs/cap-theorem/

WIN-MTFQ8CJAV81\...ind - dbo.Orders

| OrderID | CustomerID | EmployeeID | OrderDate |
|---|---|---|---|
| 10248 | HANAR | 1 | 1996-07-04 00:0... |
| 10249 | HANAR | 6 | 1996-07-05 00:0... |
| 10250 | HANAR | 4 | 1996-07-08 00:0... |

1 of 200 Cell is Read Only.

WIN-MTFQ8CJAV81\..._m2 - dbo.Orders

| OrderID | CustomerID | EmployeeID | OrderDate |
|---|---|---|---|
| 10248 | HANAR | 2 | 1996-07-04 00:0... |
| 10249 | HANAR | 6 | 1996-07-05 00:0... |
| 10250 | HANAR | 4 | 1996-07-08 00:0... |

1 of 200 Cell is Modified.

WIN-MTFQ8CJAV81\...ger2 - dbo.Orders

| OrderID | CustomerID | EmployeeID | OrderDate |
|---|---|---|---|
| 10248 | HANAR | 3 | 1996-07-04 00:0... |
| 10249 | HANAR | 6 | 1996-07-05 00:0... |
| 10250 | HANAR | 4 | 1996-07-08 00:0... |
| 10251 | VICTE | 3 | 1996-07-08 00:0 |

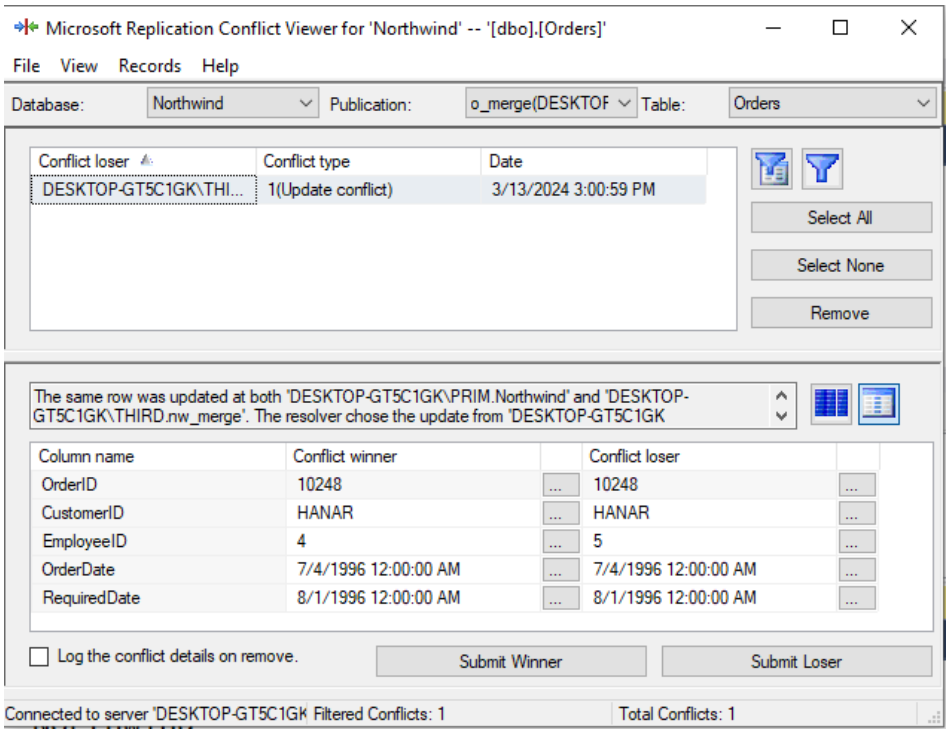
---

## Naplószállítás

Míg a replikáció elsősorban üzleti folyamatok automatizálására alkalmas, a naplószállítás egy adatbázis **katasztrófa utáni helyreállítására** (disaster recovery) való felkészülésre lett tervezve: egy pontos, általában csak olvasható másolatot hoz létre és tart szinkronban — ezt nevezik ‘meleg’ tartalék adatbázisnak. A naplót az elsődleges szerverről több másodlagos szerverre is el lehet küldeni.

A naplószállítás az adatbázis teljes biztonsági mentésével kezdődik, amelyet visszaállítanak a másodlagos szerveren. Ezután a naplószállítás három lépése a következő:

1. Az elsődleges szerveren futó **biztonsági mentési feladat** (backup job) lementi az adatbázis tranzakciós naplójának új részét a helyi szerverre.

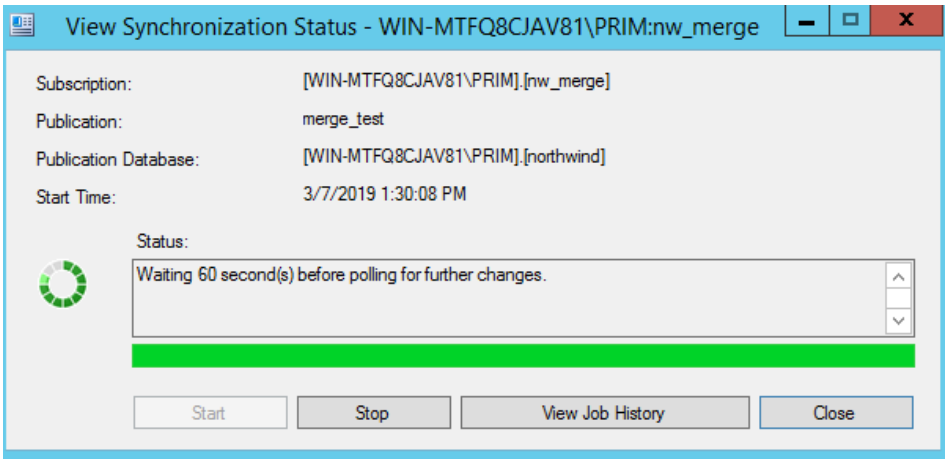



2. A másodlagos szerveren futó **másolási feladat** (copy job) átmásolja a naplót egy konfigurálható célhelyre (pl. egy hálózati fájlszerverre).
3. A másodlagos szerveren futó **visszaállítási feladat** (restore job) visszaállítja a biztonsági mentést a másodlagos adatbázis(ok)ra.

Egy figyelmeztető feladat is futhat, amely figyeli, hogy minden lépés a vártnak megfelelően és időben elvégzésre kerül-e.

A feladatok ütemezésétől függően késleltetés van a két adatbázis között. Ez a késleltetés kiaknázható, ha az elsődleges adatbázist véletlenül módosítják.

> **GYAKORLAT:** ebben a forgatókönyvben a Northwind adatbázisról hozunk létre **‘meleg biztonsági mentést’**. Ez azt jelenti, hogy szerverhiba esetén a tartalék szerver (azaz a meleg biztonsági mentés) néhány percen belül átveheti az éles szerver szerepét, anélkül hogy hosszadalmas teljes biztonsági mentés visszaállítási eljárást kellene elvégezni. Az elsődleges és másodlagos szerverek általában különböző szervergépeken futnak, de a demóban ugyanazt a gépet és két szerverpéldányt — a Primary-t és a Secondary-t — fogjuk használni. A Third szerver monitorként fog működni.

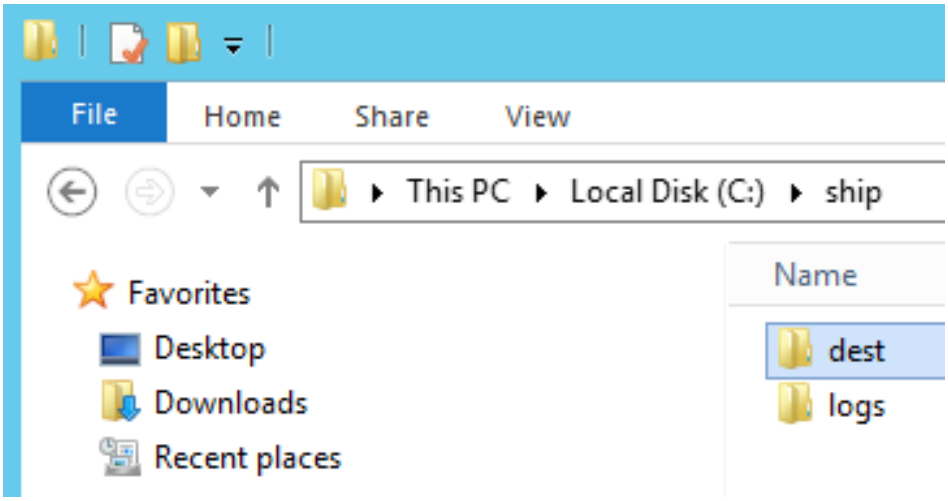



1. Hozz létre egy mappát a naplók tárolásához és egy másikat, ahová a másolatot elhelyezed:

   `C:\ship\logs` és `C:\ship\dest`

2. Mindkét mappa megosztási beállításait (Tulajdonságok/Megosztás/Megosztás) állítsd be úgy, hogy mindenki számára elérhető legyen. Írd be az „Everyone” nevet, és állítsd be az Olvasás/Írás jogosultságot.

3. Az elsődleges szerveren (PRIM) a Northwind adatbázis helyi menüjéből válaszd a Tulajdonságok lehetőséget, és ellenőrizd, hogy a Northwind adatbázis helyreállítási modellje Full értékre van-e állítva. Ez azt jelenti, hogy a napló inaktív részei nem törlődnek ellenőrzőpontnál.




4. A PRIM-en, a Northwind adatbázis Feladatok → Tranzakciós naplók szállítása paneljén engedélyezd az adatbázist szállításhoz, és válaszd a Biztonsági mentési beállítások lehetőséget. A biztonsági mentési mappa hálózati elérési útjaként add meg `\\WIN-MTFQ8CJAV81\logs`, a helyi mappa elérési útjaként pedig `C:\ship\logs` értéket.

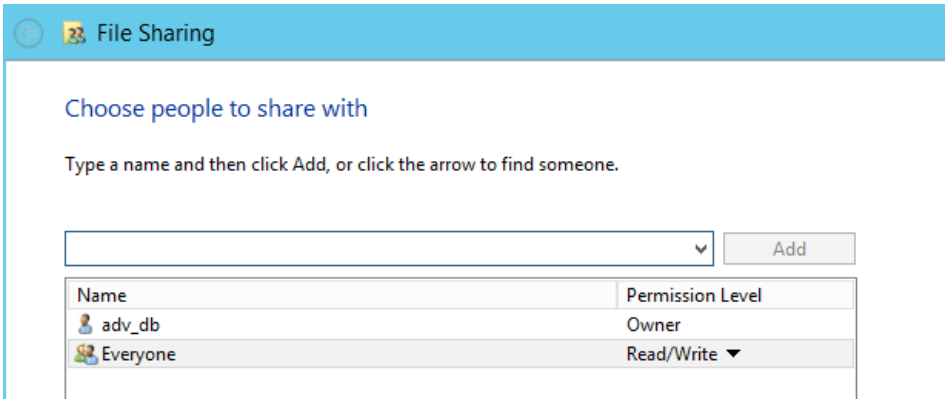
5. Válaszd a Feladat szerkesztése lehetőséget, és állíts be egy ütemezést, amely percenként futtatja a feladatot. *Megjegyzés: ez a beállítás csak bemutató célokra szól.*

6. Térj vissza az Adatbázis-tulajdonságok panelre, és kattints a Hozzáadás gombra a Másodlagos adatbázisok részben.

7. A Másodlagos adatbázis beállításai panelen add meg az `nw_ship` nevet az új adatbázisnak. Fogadd el az inicializálás alapértelmezéseit. A Fájlok másolása fülön add meg

`C:\ship\dest` értéket célmappaként. Az ütemezést állítsd percenkénti futásra. *Megjegyzés: ez csak bemutató célokra szól.*

8. A Visszaállítás fülön válaszd a Készenléti módot (Standby mode) és a Felhasználók lecsatlakoztatása lehetőséget, majd állíts be percenkénti visszaállítási ütemezést. *Megjegyzés: ez csak bemutató célokra szól.*




9. Opcionálisan konfigurálhatod a THIRD szervert Monitor szerverként. A Monitor szerver az a példány, amely figyeli az elsődleges és másodlagos szervereket, és futtatja a naplószállítás figyelő feladatát. Ez a feladat hibát generál, ha a három folyamat (biztonsági mentés, másolás, visszaállítás) bármelyike nem sikerül megfelelően az előre beállított küszöbértékidőn belül (alapértelmezés: 45 perc). Tesztelési célokra használj rövidebb időszakot, például **2 percet**. Az sa SQL Server megbízottat (principal) használd a figyelő feladat hitelesítésére az elsődleges és másodlagos példányok eléréséhez. Emellett két riasztás is automatikusan beállításra kerül a monitor példányon, az elsődleges és másodlagos példányok meghibásodásának esetére — üres válasszal (amelyet a rendszergazdának kell konfigurálnia). Tipikus válasz az operátornak küldött e-mail értesítés.

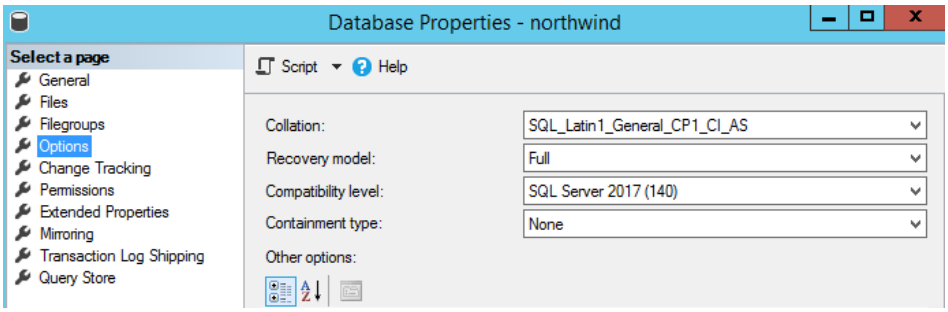
10. Véglegesítsd és futtasd a konfigurációt.

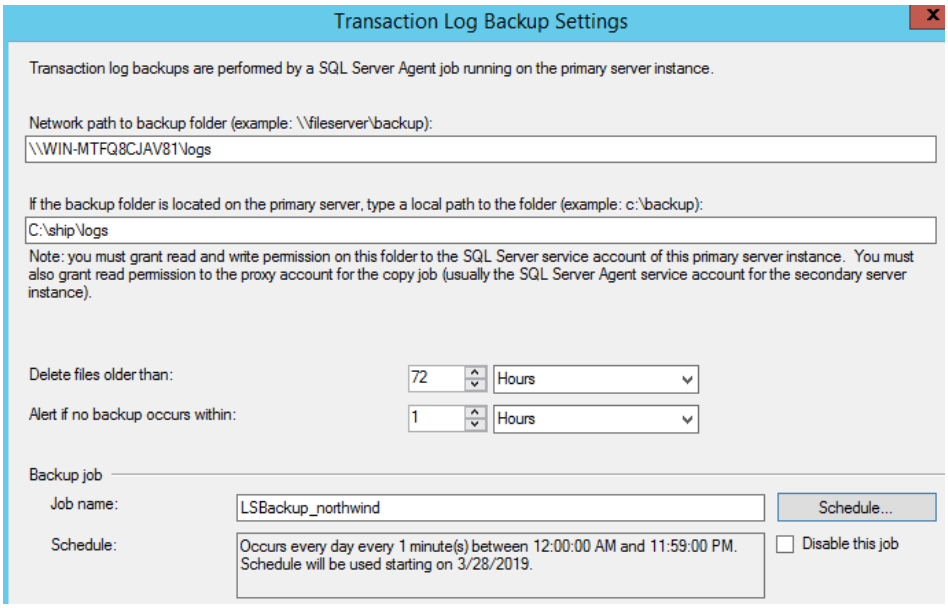
11. Ellenőrizd a helyes működést mindkét adatbázishoz csatlakozva. Lehet, hogy 3 percet kell várnod, amíg a változások megjelennek a másodlagos adatbázisban. A naplószállítás aktuális állapotáról jelentést készíthetsz a Monitor szerver helyi menüjéből: Jelentések → Standard jelentések → Tranzakciós naplószállítás állapota.

12. Ellenőrizd a `logs` és `dest` mappák tartalmát. Percenként kell megjelennie a tranzakciós napló biztonsági mentéseinek.

13. Tiltsd le a naplószállítást a PRIM szerveren. Mivel a Secondary-n szállított adatbázis készenléti állapotban van, az eldobása előtt a következő parancsokat kell végrehajtanod (lásd alább). Alternatívaként az SSMS felületen is beállíthatod az adatbázist egyfelhasználós módba, visszaállíthatod recovery-vel, majd visszaállíthatod multi-user módba.

```
use master
alter database nw_ship set single_user with rollback immediate
restore database nw_ship with recovery
alter database nw_ship set multi_user
```



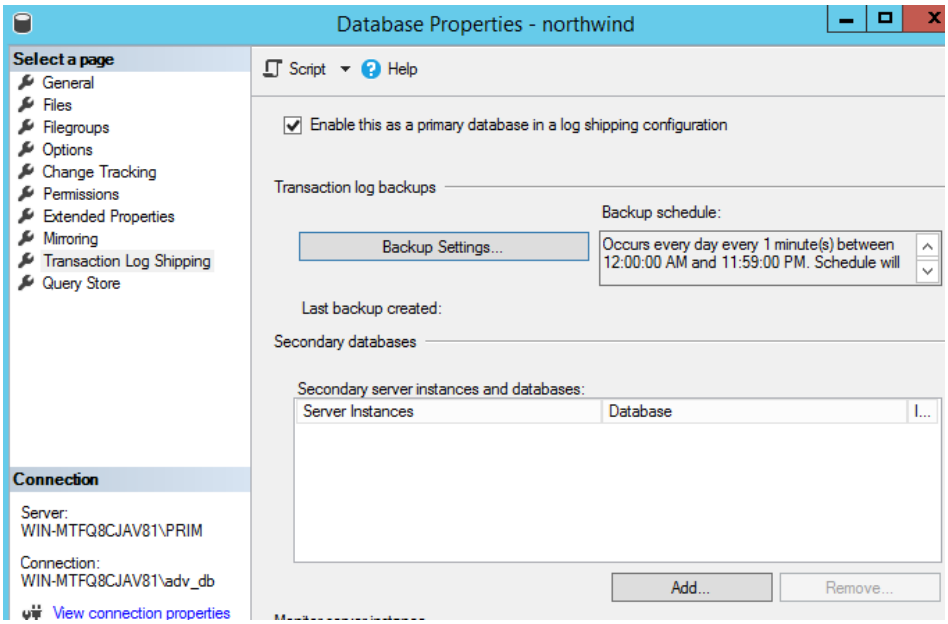



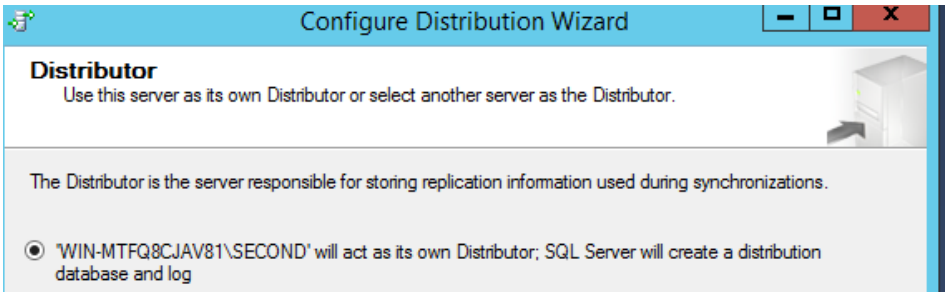

## Feladatátvételi fürtök

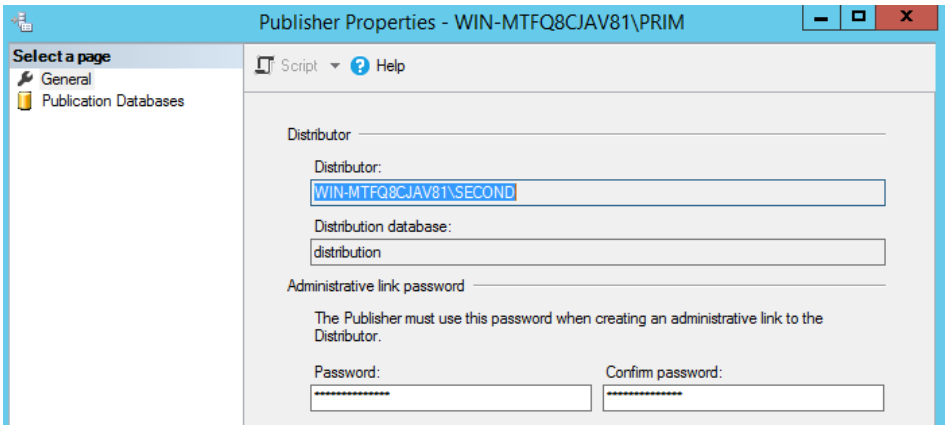
Definíció szerint: „A feladatátvételi fürtözés (FC) egy Windows Server funkció, amely lehetővé teszi, hogy több kiszolgálót egyetlen hibatűrő fürtté csoportosíts az alkalmazások és szolgáltatások — például a Microsoft SQL Server — rendelkezésre állásának és méretezhetőségének növelése érdekében.”[11] Egy FC-hez szerver típusú operációs rendszert igénylő, egynél több szervergépen futó rendszerre van szükség. Ez az egyik oka annak, hogy ezt a technológiát nem tudjuk bemutatni ebben a kurzusban.

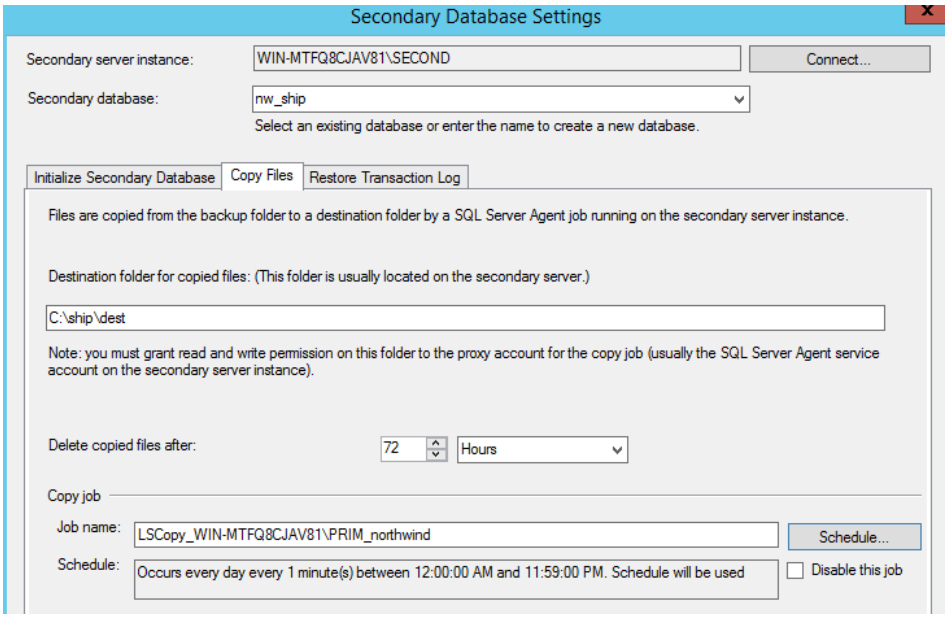
Miután az FC-t beállítottuk, a fürttagokon futó SQL Server-példányok konfigurálhatók egy Always On rendelkezésre állási csoportba. Egy ilyen csoportban meghatározhatunk egy ‘rendelkezésre állási adatbázisok’ nevű adatbáziscsoportot, amelyek másolódnak a többi példányra, és együtt hajtják végre a feladatátvételt. A feladatátvétel-támogatáson kívül ilyen architektúra konfigurálható *automatikus olvasási terheléselosztásra* is a nagy terhelésű adatbázisoknál.

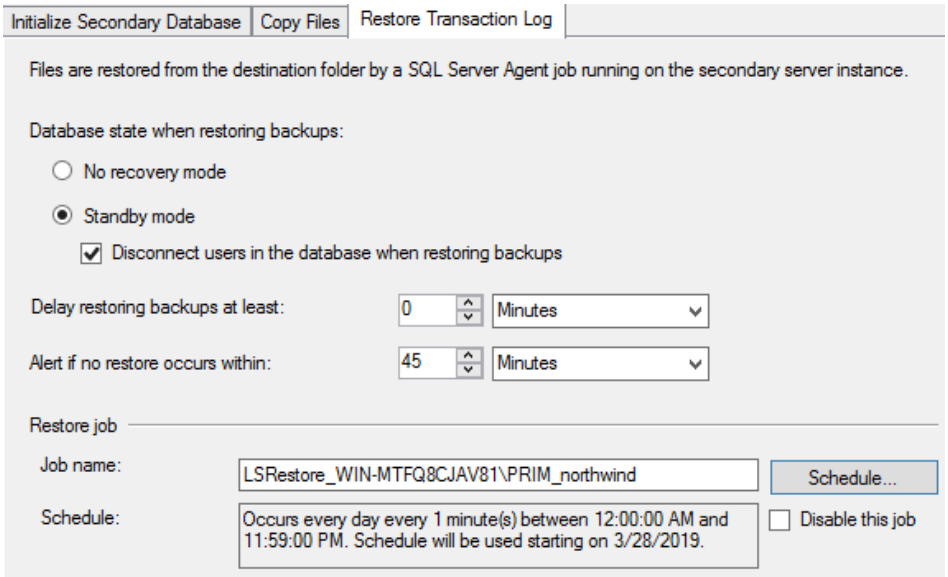
[11] Lásd: https://docs.microsoft.com/hu-hu/windows-server/manage/windows-admin-center/use/manage-failover-clusters és https://docs.microsoft.com/hu-hu/windows-server/failover-clustering/failover-clustering-overview


Database Properties - northwind
Select a page
General
Files
Filegroups
Options
Change Tracking
Permissions
Extended Properties
Mirroring
Transaction Log Shipping
Query Store
Script
Help
Enable this as a primary database in a log shipping configuration
Transaction log backups
Backup schedule:
Backup Settings...
Occurs every day every 1 minute(s) between 12:00:00 AM and 11:59:00 PM. Schedule will
Last backup created:
Secondary databases
Secondary server instances and databases:
Server Instances
Database
Add...
Remove...
Connection
Server:
WIN-MTFQ8CJAV81\PRIM
Connection:
WIN-MTFQ8CJAV81\adv_db
View connection properties


Configure Distribution Wizard
Distributor
Use this server as its own Distributor or select another server as the Distributor.
The Distributor is the server responsible for storing replication information used during synchronizations.
'WIN-MTFQ8CJAV81\SECOND' will act as its own Distributor; SQL Server will create a distribution database and log


Publisher Properties - WIN-MTFQ8CJAV81\PRIM
Select a page
General
Publication Databases
Script
Help
Distributor
Distributor:
WIN-MTFQ8CJAV81\SECOND
Distribution database:
distribution
Administrative link password
The Publisher must use this password when creating an administrative link to the Distributor.
Password:
Confirm password:




# 4. Adatminőség és törzsadat-menedzsment

Az adatok típusai egy vállalatnál:

- Tranzakciós (OLTP adatbázisokban)
- Hierarchikus adatok (pl. taxonómiák)
- Adattárházi adatok (data warehouse)
- Félig strukturált (XML, JSON)
- Strukturálatlan (e-mail, PDF, blogok)
- Törzsadat (master data)
- Metaadat (adat az adatról)

A magas adatminőség (DQ – Data Quality) követelménye szigorúbb az adatintegritásénál (pl. amikor elgépelt értékeket szeretnénk javítani).

Az adatminőség dimenziói:

- **Kemény dimenziók**
  - Teljesség (completeness) — megvan-e minden adat?
  - Pontosság (accuracy) — helyesek-e az értékek?
  - Konzisztencia (consistency) — ellentmondanak-e az adatok egymásnak különböző rendszerekben?
- **Puha dimenziók:** a felhasználók szubjektív megítélése alapján, pl. bizalom.

Ha mind a puha, mind a kemény dimenziók értékelése gyenge, Törzsadat-menedzsment (MDM – Master Data Management) megoldást kell alkalmazni. Az MDM-hez központi adattárolóra és *adatkormányzásra* (Data Governance) van szükség: ez az adatminőség megvalósításának folyamata. Ehhez *adatgondozók* (Data Stewards) kellenek: olyan személyek, akik felelősek bizonyos adattípusok minőségéért — például egy gondozó felelős lehet az ügyféladatokért. **Az MDM-technológiákat ez a tankönyv nem tárgyalja részletesen.**

Az adatminőség (DQ) tervezett vagy reaktív módon javítható, az utóbbi kisebb vállalatoknál is alkalmazható. Egy DQ-megoldás sikerességét nyomon követhetjük például azzal, hogy egy táblában a hibás vagy ismeretlen rekordok száma hogyan csökken idővel.

> **DEMO:** elemezd a táblák oszlopait teljesség, konzisztencia, funkcionális függőség stb. szempontjából:

```
use AdventureworksDW2019

--vizsgáljuk, van-e funkcionális függés két oszlop között
select SalesReasonKey, SalesReasonName, SalesReasonReasonType from DimSalesReason

--az adatok azt sugallják, hogy a SalesReasonReasonType oszlop függ
--a SalesReasonName oszloptól, azaz az előbbi mindig megállapítható az utóbbiból
--ellenőrizzük, hogy az adatok alátámasztják-e ezt a feltevést

select SalesReasonReasonType, count(*) from DimSalesReason group by
SalesReasonReasonType  --3 csoport 4, 5 és 1 taggal
select SalesReasonName, count(*) from DimSalesReason group by SalesReasonName  --minden
--csoportnak 1 tagja van -> jelölt kulcs
--következtetés: a SalesReasonName valószínűleg a SalesReasonReasonType alkategóriája

--funkcionális függés ellenőrzése
--hogyan ellenőrizzük, hogy a CountryRegionCode funkcionálisan függ-e a StateProvinceCode-tól
select * from DimGeography

select StateProvinceCode, count(*) from DimGeography group by StateProvinceCode --71
select StateProvinceCode, count(*) from DimGeography group by
StateProvinceCode,CountryRegionCode  --71
--71=71, tehát egy StateProvinceCode mindig ugyanazt a CountryRegionCode-ot kapja
--nagy valószínűséggel funkcionális függés van a két mező között
select color, count(*) from dimproduct group by color --10
select ProductLine, count(*) from dimproduct group by productline --5
select ProductLine, count(*) from dimproduct group by productline, color --27 > 5, 27
> 10
--nincs funkcionális függés a két mező között
```

> **GYAKORLAT:** elemezd a DimCustomer és DimEmployee táblák oszlopait teljesség és funkcionális függőség szempontjából. DimCustomer: Education↔Occupation, DimEmployee: SalesTerritory↔Gender.

Egy de facto funkcionális függőség az üzleti terület modellezetlen, de létező üzleti szabályából is eredhet. Az ilyen rejtett kapcsolat megsértheti a 3NF-struktúrát egy táblán belüli tranzitív függőség révén, és **redundanciához és inkonzisztenciához** vezethet. Ezért érdeke a vállalatnak, hogy adatelemzéssel felderítse az ilyen összefüggéseket.

**Példa:** az alkalmazottainknak van munkaköri kódjuk (pl. kutató, menedzser, műszaki, titkár) és iskolai végzettség-kódjuk (pl. általános iskola, középiskola, PhD). Elképzelhető, hogy a vállalatnál olyan szabályzat él, amely csak egy bizonyos végzettséget engedélyez egy adott munkakörhöz — ezzel de facto funkcionális függőséget hozva létre a két mező között. A végzettség

a munkakörtől függhet.

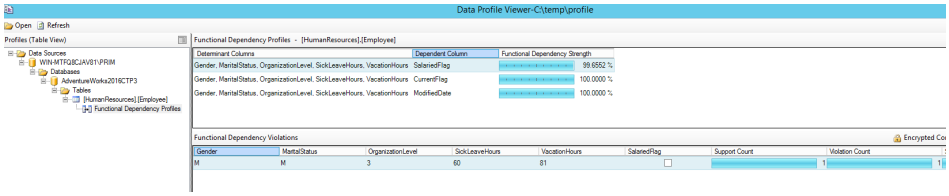
---

## Adatprofil-elemzés

Az adatprofil-elemzés (Data Profiling) célja, hogy automatikusan megtalálja egy oszlop statisztikai és adatminőséggel kapcsolatos jellemzőit:

- Funkcionális függőségek, jelölt kulcsok és potenciális idegen kulcsok azonosítása (lásd fentebb)
- Oszlopérték-eloszlások és egyéb statisztikák kiszámítása
- Null-arány és hosszeloszlás mérése szöveges típusoknál
- *De facto* oszlopminták levezetése reguláris kifejezésekként az értékekből — ezek *tartományszabályokként* (domain rules) használhatók (lásd később)

MS SQL Serverben az adatprofil-elemzés az Integration Services Data Profiling Task segítségével végezhető el, amely az eredményeket XML-fájlba menti (ebben a tankönyvben nem részletezzük).[12]




---

## SQL Server Data Quality Services

A DQS (Data Quality Services) szolgáltatás felelős a referenciaadatok tisztításáért, profilozásáért és egyeztetéséért. A DQS szerver a DQS motort és a DQS adatbázisokat foglalja magában:

- **DQS_MAIN:** a DQS motort megvalósító tárolt eljárások, valamint tudásbázisok (pl. US – Last Name tudásbázis, referenciaadatok: pl. magyarországi városok). Szerepkörök a DQS_MAIN adatbázisban: `dqs_administrator`, `dqs_kb_editor`, `dqs_kb_operator` (különböző felhasználókhoz rendelhetők)
- **DQS_PROJECTS:** a tisztítási és egyeztetési projektekhez szükséges adatok
- **DQS_STAGING_DATA:** ideiglenes tároló a tisztítandó adatokhoz és a tisztítás eredményéhez

[12] Lásd: https://docs.microsoft.com/en-us/sql/integration-services/control-flow/data-profiling-task-and-viewer

Egy Data Quality kliens által elvégzendő feladatok: tudásbázisok kezelése, tisztítási/egyeztetési projektek végrehajtása, DQS adminisztrálása. Kliensimplementációk:

- Egy SQL Server Integration Services 'DQS Cleansing transformation' komponens elvégezheti a tisztítást egy SSIS-csomag adatfolyamában
- SQL Server 2017 Data Quality Client (DQC), egy asztali alkalmazás

**Adattisztítási projektek**

A DQS-projektek két fajtája:

- **Alapvető adattisztítás** (Basic data cleansing): az egyedi mezőértékek javítása a cél. Ezt a projekttípust mutatjuk be ezen a kurzuson.
- **Azonosság-leképezés és deduplikáció** (Identity mapping and de-duplicating) egyeztetési szabályok alapján: a cél az esetlegesen duplikált rekordok — vagyis entitások — azonosítása és eltávolítása. Az egyeztetési szabályt tudásbázis szinten kell definiálni, az adattudományban elterjedt távolságmértékek alkalmazásával. Ezt a projekttípust nem mutatjuk be ezen a kurzuson.

Mindkét típus a DQS_MAIN adatbázisban már telepített tudásbázison (KB) alapul, és ezeket a paramétereket használja:

- **Tisztítás:** a Javaslat minimális pontszáma (Min Score for Suggestions) a javaslat generálásához szükséges minimális hasonlóságot adja meg. Ennek *kisebbnek* kell lennie, mint az Automatikus javítás minimális pontszáma (Min Score for Auto Correction).
- **Egyeztetés (deduplikáció):** küszöbérték az egyeztetési szabályhoz. Ez a rekordok hasonlóságának mértéke.

Egy DQS **tudásbázis** (Knowledge Base, KB) több **tartományt** (domain) tartalmazhat, vagyis a tisztításhoz használt referencia-értékkészleteket. Egy tartományt a következő összetevők határoznak meg:

- **Név és adattípus**, pl. string
- Normalizálási beállítások, mint nagybetűssé alakítás, üres karakterek eltávolítása, formázás és helyesírás-ellenőrzés

- Referenciaadatok, amelyek tárolhatók a DQS_MAIN adatbázisban, vagy alternatívaként külső szolgáltatás is nyújthatja őket
  - A referencia-értékeket **tartományértékeknek** (domain values – referenciaértékek) hívják. Egy tartományérték egy olyan sor, amely tartalmaz egy **főértéket** (leading value – alapértelmezett érték) és annak szinonimáinak listáját
- **Tartományszabályok** (Domain rules), hasonlóak a CHECK-megszorításokhoz
- **Szövegrész-alapú kapcsolatok** (Term based relations), amelyek az érték egy részére vonatkoznak, pl. `'%Ltd.%'` → `'%Limited Company%'`

**Összetett tartományok** (Composite domains) szemantikailag összetartozó mezőkből hozhatók létre, pl. egy több oszlopban tárolt cím város, utca stb. részeiből.

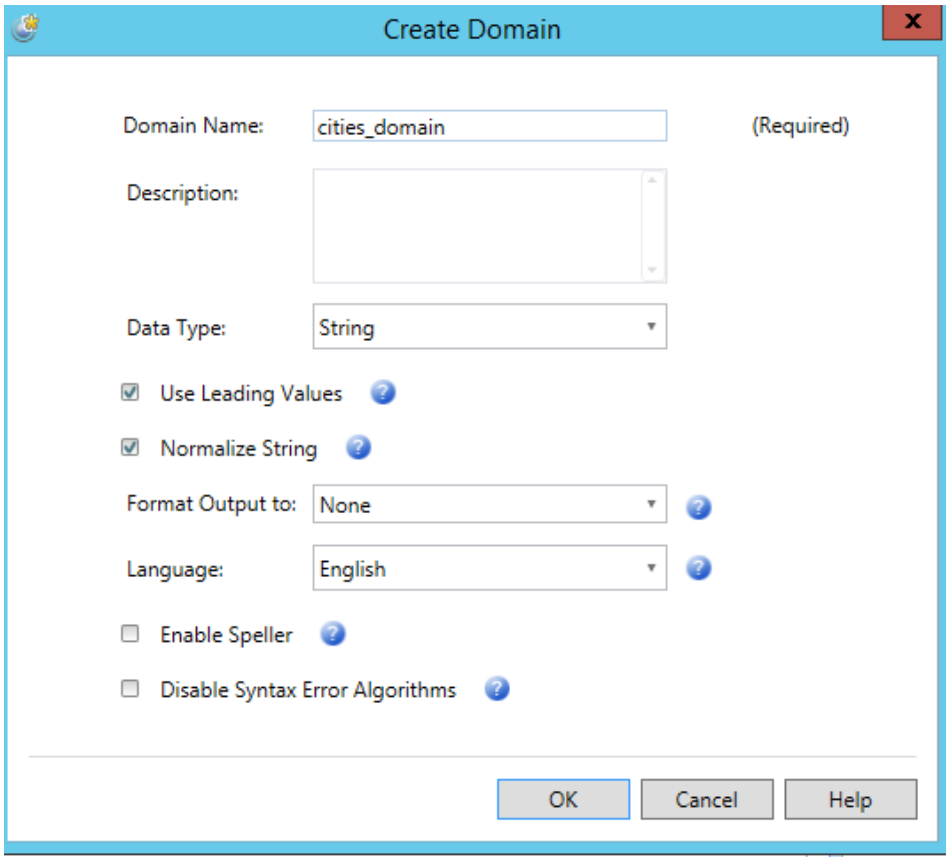
A PRIM szerveren a DQS_MAIN-ben már telepítve vannak alapértelmezett tudásbázisok, pl. Country/Region, US-Counties, US-Last names. Saját tudásbázist is létrehozhatunk.

**DEMO:** megtisztítjuk egy tábla vezetékneveit az alapértelmezett KB segítségével.

Indítsd el a DQC-t, és add meg a szervernevet: `WIN-MTFQ8CJAV81\PRIM`.

- Az Administration → Configuration → General settings panelen tekintsd át a Min Score for Suggestions és Min Score for Auto Correction beállításokat. Ezeket kísérletezéssel lehet finomhangolni.
- A Knowledge base management panelen tekintsd át az előre telepített tudásbázisokat.
- Hozz létre egy új adattisztítási projektet `Lastnames` névvel.
- Válaszd ki a KB-t: DQS Data, és a US_Last Name tartományt. A tevékenység legyen Cleansing (Tisztítás).
- A Map oldalon válaszd ki az AdventureworksDW2019 adatbázist, a DimCustomer táblát és a Lastname mezőt, és rendeld hozzá a US_Last Name tartományhoz.
- Indítsd el a tisztítási folyamatot. Mivel a DimCustomer táblában több mint 18 000 vezetéknevet kell megtisztítani, az előfeldolgozás kb. 5 percig, a tisztítás kb. 30 másodpercig tart. **FIGYELEM:**[13]
- Ellenőrizd a kemény dimenziókat: Teljesség (100%) és Pontosság (98%). A tisztítás után egy érték a következő állapotok egyikébe kerülhet:
  - **Helyes** (Correct): megtalálható a tartományértékek között
  - **Javított** (Corrected): automatikusan módosítva egy tartományértékre, mert a hasonlóság meghaladta az Automatikus javítás minimális pontszámát
  - **Javasolt** (Suggested): új értéket csak javasol a rendszer, mert a hasonlóság meghaladta a Javaslat minimális pontszámát
  - **Új** (New): a tartományértékek között nem található hasonló érték, de az érték megfelel a tartományszabályoknak
  - **Érvénytelen** (Invalid): egy új érték, amely ellentmond a tartományszabályoknak
- A fenti Min Score beállításokkal a folyamat néhány esetet talál, ahol javítást javasol, és 3 esetet, ahol automatikus javítást végzett. 328 nevet helyesen talált meg a KB-ban.
- Próbálj meg jóváhagyni egy javasolt módosítást → a sor átkerül a Javított panelre.
- Végül mentsd el a megtisztított táblát a LastNameSource és LastNameOutput oszlopokkal az eredeti adatbázisba, és dolgozd fel T-SQL parancsokkal.

[13] Ha a 'Parallel processing task failed' hibát kapod az előfeldolgozás során, ellenőrizd a DQServerLog.DQS_MAIN.log fájlt a példány Log könyvtárában. A hiba: *'The DELETE statement conflicted with the REFERENCE constraint "B_INDEX_LEXICON_EXTENSION_B_INDEX_LEXICON_FK". The conflict occurred in database "DQS_PROJECTS", table "DQProject1000001.B_INDEX_LEXICON_EXTENSION", column 'TERM_ID' '*. Az SSMS Modify keys paneljén állítsd a B_INDEX_LEXICON_EXTENSION_B_INDEX_LEXICON_FK idegen kulcs-megszorítás On Delete műveletét CASCADE értékre.




**GYAKORLAT:** tisztítsd meg az AdventureworksDW2019.DimGeography.EnglishCountryRegionName mezőt a Country/Region tartomány segítségével. Próbáld meg újra, miután hozzáadtál egy elírt országnevet a táblához (Austrila).

### Saját tudásbázis létrehozása

Egy táblamező meglévő értékkészletét felhasználhatjuk saját KB összeállításához. A lépések:

1. Automatikus tudásfelderítés az adatminta alapján
2. Manuális tartománykezelés és -konfiguráció

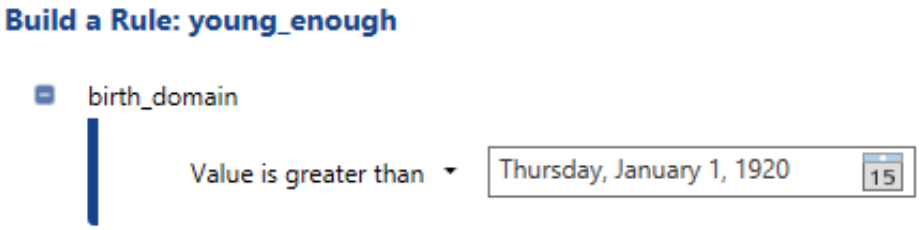
**DEMO:** létrehozunk egy KB-t.

- Hozz létre egy új nézetet a DQS_STAGING_DATA adatbázisban:

```
use DQS_STAGING_DATA
go
create view vi_places as
select distinct City, StateProvinceName StateProvince, EnglishCountryRegionName
CountryRegion
from AdventureworksDW2019.dbo.DimGeography
go
```

- Indítsd el a DQS klienst, és hozz létre egy új, `places` nevű KB-t.
- Válaszd a Knowledge discovery lehetőséget, állítsd be forrásként a `vi_places` nézetet, és hozz létre két tartományt: `cities_domain` és `countryregion_domain`, a `vi_places` nézet azonos nevű mezőit használva.
- Indítsd el a felderítést.

- A Muehlheim-bejegyzésnél állítsd a típust **Error** értékre, és a ‘Correct to’ mezőbe írd be manuálisan: `Mühlheim`, *majd nyomj Entert*.
- Válaszd ki az új KB-t a főmenüből, és a helyi menüből válaszd a Domain management lehetőséget.
- Adj hozzá manuálisan egy új tartományt `birth_domain` névvel, egy egyszerű tartomány-szabállyal:




**Build a Rule: young_enough**

- `birth_domain` → Érték nagyobb mint: 1920. január 1., csütörtök
- Tedd közzé a KB-t.
- Válaszd ki az új KB-t kezelésre, és exportáld egy `places.dqs` nevű fájlba.
- Hozz létre egy `test_customer` nevű táblát a DQS_STAGING_DATA adatbázisban a DimCustomer és DimGeography táblák GeographyKey oszlopon összekapcsolt adataiból.
- Módosítsd az első ügyfél születési dátumát 1925-01-02-re, városát Muehlheimre. A második ügyfél városát változtasd Piripócsra.
- Az új ‘places’ KB segítségével tisztítsd meg a `test_customer` táblát, és tekintsd meg az eredményeket.

  **GYAKORLAT:** építs KB-t a DimProduct tábla ProductName mezőjét felhasználva, és mutasd be alkalmazását a fentiek szerint.

Ingyenes, független eszközök is elérhetők adattisztításhoz és törzsadat-menedzsmenthez, pl. https://openrefine.org/

---

# 5. Oszloptár és particionálás

## Oszloptár (Columnstore)

Az oszloptár (columnstore) egy alternatív módszer a relációs adatok tárolására. **Sok mezőt tartalmazó, nagy adattárházi táblákhoz és ritkán változó adatokhoz** ajánlott. Az adatlapokon (page-eken) egy rekord mezői nem együtt tárolódnak; ehelyett minden mezőt önállóan tárolnak, és a rekord értékei logikai rekordazonosítók alapján kapcsolódnak össze.[14] Gyorsaságát az alacsony I/O-műveletszámnak (blokkos olvasásnak) köszönheti, amely az alábbi tényezőkből ered:

- A lapok tömörítése[15] (kisebb méret → kevesebb lap), valamint az a tény, hogy a lekérdezések általában nem igénylik az egész rekord olvasását, csak néhány mezőét

[14] Lásd: https://learn.microsoft.com/en-us/sql/relational-databases/indexes/columnstore-indexes-overview?view=sql-server-ver16

[15] Lásd: https://learn.microsoft.com/en-us/sql/relational-databases/data-compression/data-compression?view=sql-server-ver16

- Ez különösen szembetűnő, ha a tábla nagyon sok mezőt tartalmaz — ami az adattárházakra jellemző.

Tömörítési típusok:

- **Sor-alapú tömörítés** (Row compression): a rögzített hosszúságú adattípusokat változó hosszúságú adattal helyettesíti, pl. egy 4 bájtos integer mező az aktuális értéktől függően 1 bájtban is tárolható. OLTP alkalmazásokban alkalmazható.
- **Lap-alapú tömörítés** (Page compression): teljes lapokat tömörít; adattárházi alkalmazásokra vagy ritkán frissített OLTP alkalmazásokra inkább alkalmas. Különösen hatékony, ha egy mező értékei nagyon hasonlóak — ezeket *alacsony kardinalitású* (low cardinality, azaz kevés különböző értékkel rendelkező) mezőknek nevezzük.

Az alábbiakban oszloptárat és lap-alapú tömörítést mutatunk be SQL Serveren — a Postgres ingyenes kiadása jelenleg nem támogatja a tömörítést.

```
use AdventureWorksDW2016_ext

/*
--BTREE tábla létrehozása
select * into FactResellerSalesXL_BTREE from FactResellerSalesXL_CCI --2min 30s
alter table FactResellerSalesXL_BTREE alter column SalesOrderLineNumber tinyint not null
alter table FactResellerSalesXL_BTREE alter column SalesOrderNumber nvarchar(20) not null
alter table FactResellerSalesXL_BTREE add constraint c1 primary key
(SalesOrderLineNumber, SalesOrderNumber) --2min
*/

--SZERVER ÚJRAINDÍTÁSA
set statistics time on
set statistics io on
go
select count(*) from FactResellerSalesXL_BTREE  --11669638
exec sp_spaceused 'dbo.FactResellerSalesXL_BTREE', @updateusage = 'TRUE'
--adat: 2523168 KB, index: 9776 KB

select count(*) from FactResellerSalesXL_PageCompressed  --11669638
exec sp_spaceused 'dbo.FactResellerSalesXL_PageCompressed', @updateusage = 'TRUE'
--adat: 695624 KB, index: 2344 KB

select count(*) from FactResellerSalesXL_CCI  --11669638
exec sp_spaceused 'dbo.FactResellerSalesXL_CCI', @updateusage = 'TRUE'
--adat: 525344 KB, index: 157624 KB

go
dbcc freeproccache --

dbcc dropcleanbuffers -- adatpuffer ürítése

--B-TREE
--========
select b.SalesTerritoryRegion
  ,FirstName + ' ' + LastName as FullName
  ,count(SalesOrderNumber) as NumSales
  ,sum(SalesAmount) as TotalSalesAmt
  ,Avg(SalesAmount) as AvgSalesAmt
  ,count(distinct SalesOrderNumber) as NumOrders
  ,count(distinct ResellerKey) as NumResellers
from FactResellerSalesXL_BTREE a
inner join DimSalesTerritory b on b.SalesTerritoryKey = a.SalesTerritoryKey
inner join DimEmployee d on d.Employeekey = a.EmployeeKey
inner join DimDate c on c.DateKey = a.OrderDateKey
where b.SalesTerritoryKey = 3 and c.FullDateAlternateKey between '1/1/2006' and
'1/1/2010'
group by b.SalesTerritoryRegion,d.EmployeeKey,d.FirstName,d.LastName,c.CalendarYear
--CPU idő = 3949 ms, eltelt idő = 9568 ms

--DATA_COMPRESSION = PAGE
--==================================
```

```sql
go
select b.SalesTerritoryRegion
  ,FirstName + ' ' + LastName as FullName
  ,count(SalesOrderNumber) as NumSales
  ,sum(SalesAmount) as TotalSalesAmt
  ,Avg(SalesAmount) as AvgSalesAmt
  ,count(distinct SalesOrderNumber) as NumOrders
  ,count(distinct ResellerKey) as NumResellers
from FactResellerSalesXL_PageCompressed a
inner join DimSalesTerritory b on b.SalesTerritoryKey = a.SalesTerritoryKey
inner join DimEmployee d on d.Employeekey = a.EmployeeKey
inner join DimDate c on c.DateKey = a.OrderDateKey
where b.SalesTerritoryKey = 3
  and c.FullDateAlternateKey between '1/1/2006' and '1/1/2010'
group by b.SalesTerritoryRegion,d.EmployeeKey,d.FirstName,d.LastName,c.CalendarYear
GO  -- CPU idő = 3264 ms, eltelt idő = 3776 ms.

--oszloptár lap-alapú tömörítéssel
--create clustered columnstore index [IndFactResellerSalesXL_CCI] on
--[dbo].[FactResellerSalesXL_CCI]
--with (drop_existing = off, compression_delay = 0, data_compression = columnstore) on
--[primary]
--================================
select b.SalesTerritoryRegion
  ,FirstName + ' ' + LastName as FullName
  ,count(SalesOrderNumber) as NumSales
  ,sum(SalesAmount) as TotalSalesAmt
  ,Avg(SalesAmount) as AvgSalesAmt
  ,count(distinct SalesOrderNumber) as NumOrders
  ,count(distinct ResellerKey) as NumResellers
from FactResellerSalesXL_CCI a
inner join DimSalesTerritory b on b.SalesTerritoryKey = a.SalesTerritoryKey
inner join DimEmployee d on d.Employeekey = a.EmployeeKey
inner join DimDate c on c.DateKey = a.OrderDateKey
where b.SalesTerritoryKey = 3
  and c.FullDateAlternateKey between '1/1/2006' and '1/1/2010'
group by b.SalesTerritoryRegion,d.EmployeeKey,d.FirstName,d.LastName,c.CalendarYear
GO  -- CPU idő = 360 ms, eltelt idő = 492 ms
```

Megjegyzés az olvasási statisztikákhoz:

- **Logikai olvasások** (Logical reads): 8 KB-os lapok olvasása az adatpufferből
- **Fizikai olvasások** (Physical reads): a tárhelyről (OS-től) beolvasandó lapok — ezek blokkolják a lekérdezés végrehajtását
- **Előolvasások** (read-ahead reads): aszinkron olvasási kérések az OS felé a lekérdezés által valószínűleg hamarosan szükséges lapokhoz — ezek nem blokkolják a lekérdezést.

A kimenet a Workfile és Worktable táblázatokat is megemlíti a többi tábla mellett. A worktable egy belső tábla, amelyre mindig szükség van egy soros módú hash join művelethez, hogy a bemenetet hash-partíciókra ossza szét. A workfile-okat hash join és hash aggregáció ideiglenes eredményeinek tárolására használják.

---

## Particionálás

A particionálás célja: oszd meg és uralkodj. Ha egy táblát egy mező mentén particionálunk:

- Mivel a fizikai olvasások a lekérdezés-végrehajtás szűk keresztmetszetét jelentik, a lekérdezések gyorsabban futnak, ha minden partíció külön fizikai tárolón van, mert az olvasások párhuzamosíthatók. A legtöbb lekérdezésnél elegendő néhány partíció olvasása (nem kell mindet).
- A nagy tábláknak gyakorta korlátozott betöltési ideje van (pl. éjjelente egy banki rendszerben), olyan időszakokra, amikor az adatok nem változnak. Az INSERT és DELETE naplózott

utasítások (implicit naplózott tranzakciók), ezért egyszerre sok rekord beszúrása vagy törlése lassú (pl. az indexeket ezután újra kell építeni). A particionálás gyorsabbá teszi a betöltést, mert egy nagy tábla-insert helyett egy üres táblapartícióba írunk — ez egy gyors, 'minimálisan naplózott' művelet (egyszerű helyreállítási módban).

Nehéz meghatározni azt a pontos táblméretet, ahol a particionálás már megéri, de ökölszabályként elmondható, hogy a tábla méretének meg kell haladnia az adatbázis-szerver fizikai memóriájának méretét.

Tipikus particionálási kulcs a dátum, pl. hónap vagy év szerint. A tábla oszloptár-indexét, ha van, szintén particionálni kell (igazított index). Ekkor a partícióváltás nem igényli az index újraépítését, csak a metaadat változik. Az SQL Server particionálás alapfogalmai:

- **Particionálási séma** (Partitioning scheme): partíciókat rendel fájlcsoportokhoz
- **Particionálási függvény** (Partitioning function): minden rekordot egy partícióhoz rendel a particionálás alapjául szolgáló mező(k) alapján

*Megjegyzés: a partíciók mérete (rekordjainak száma) eltérő lehet.*

Nagy tábla particionálásának és minimálisan naplózott betöltésének ajánlott eljárása (pl. amikor új adatok érkeznek az adattárházba):

1. Hozd létre a particionálási sémát és függvényt
2. Töltsd be a meglévő adatokat (kezdeti betöltés)
3. Az új adatok betöltéséhez hozz létre egy üres segédtáblát (staging) ugyanolyan sémával (és tömörítéssel), mint a tábla
4. Ebben a táblában egy check-megszorítás védi a particionálási kulcsot, hogy helytelen adat ne kerülhessen be
5. Az új adatokat a segédtáblába töltsd be, és hozz létre oszloptár-indexet
6. A segédtáblát cseréld át (partition switch) a ténytábla következő üres partíciójára
7. A következő partíció betöltése előtt töröld az oszloptár-indexet, és frissítsd a check-megszorítást ennek megfelelően

```
set statistics time off
set statistics io off
--év szerint:
select min(OrderDate), max(OrderDate)  from FactInternetSales s   --2010..2014
--5 évhez 5 partíciót hozunk létre
--drop table InternetSales
go
--particionáló függvény:
--drop partition function PfInternetSalesYear
create partition function PfInternetSalesYear (tinyint) as range left for values (10,
11, 12, 13)
--pl. a '10' azt jelenti, hogy év <= 2010 (a LEFT miatt)
--TINYINT: 1 bájt, 0..255
--particionálási séma: minden ugyanabba a fájlcsoportba
--drop partition scheme PsInternetSalesYear
create partition scheme PsInternetSalesYear as partition PfInternetSalesYear all to
([PRIMARY])
go
--Megjegyzés: az identity kulcsot kombinálnunk kell a partíciószámmal, mert az identity
--csak a partíción belül lehet egyedi.
--drop table InternetSales
create table InternetSales(
   InternetSalesKey int not null identity(1,1),
   PcInternetSalesYear tinyint not null,  --partíciószám
   ProductKey int not null,
   DateKey int not null,
   OrderQuantity smallint not null default 0,
   SalesAmount money not null default 0,
   UnitPrice money not null default 0,
   DiscountAmount float not null default 0,
   constraint PK_InternetSales primary key (InternetSalesKey, PcInternetSalesYear)
)
```

```
ON PsInternetSalesYear(PcInternetSalesYear) --ez hozza létre a partíciókat
GO
--külső kulcsok hozzáadása és lap-alapú tömörítés
--ALTER TABLE dbo.InternetSales ADD CONSTRAINT FK_InternetSales_Customers FOREIGN
--KEY(CustomerDwKey)
--REFERENCES dbo.Customers (CustomerDwKey);
ALTER TABLE dbo.InternetSales ADD CONSTRAINT FK_InternetSales_Products FOREIGN
KEY(ProductKey)
REFERENCES dbo.DimProduct (ProductKey);
ALTER TABLE dbo.InternetSales ADD CONSTRAINT FK_InternetSales_Dates FOREIGN
KEY(DateKey)
REFERENCES dbo.DimDate (DateKey);
ALTER TABLE dbo.InternetSales REBUILD WITH (DATA_COMPRESSION = PAGE);
GO
--adatbetöltés 2013-ig
INSERT INTO dbo.InternetSales (PcInternetSalesYear, ProductKey, DateKey,
OrderQuantity, SalesAmount, UnitPrice, DiscountAmount)
SELECT CAST(SUBSTRING(CAST(OrderDateKey AS CHAR(8)), 3, 2) AS TINYINT) AS
PcInternetSalesYear,
--figyelj a trükkre: a dátumok integerként tárolódnak, pl. 20110223, helytakarékosság miatt
   ProductKey, OrderDateKey, OrderQuantity, SalesAmount, UnitPrice, DiscountAmount
FROM FactInternetSales AS FIS
WHERE CAST(SUBSTRING(CAST(FIS.OrderDateKey AS CHAR(8)), 3, 2) AS TINYINT) < 13
GO
--hány rekord van a partíciókban?
SELECT $PARTITION.PfInternetSalesYear(PcInternetSalesYear) AS PartitionNumber,
COUNT(*) AS NumberOfRows
FROM InternetSales GROUP BY $PARTITION.PfInternetSalesYear(PcInternetSalesYear)
PartitionNumber     NumberOfRows
1                   14          --2010
2                   2216        --2011
3                   3397        --2012
--az utolsó partíció még üres

--oszloptár:
CREATE COLUMNSTORE INDEX CSI_InternetSales ON dbo.InternetSales
   (InternetSalesKey, PcInternetSalesYear, ProductKey, DateKey,
   OrderQuantity, SalesAmount, UnitPrice, DiscountAmount)
ON PsInternetSalesYear(PcInternetSalesYear) --Megjegyzés: az index igazított a partícióhoz
GO

--új segédtábla (staging) a 2013-as adatokhoz
--a hibák elkerülésére check-megszorítást használunk
create TABLE dbo.InternetSalesNew (
   InternetSalesKey INT NOT NULL IDENTITY(1,1),
   PcInternetSalesYear TINYINT NOT NULL CHECK (PcInternetSalesYear = 13), --check
   ProductKey INT NOT NULL,
   DateKey INT NOT NULL,
   OrderQuantity SMALLINT NOT NULL DEFAULT 0,
   SalesAmount MONEY NOT NULL DEFAULT 0,
   UnitPrice MONEY NOT NULL DEFAULT 0,
   DiscountAmount FLOAT NOT NULL DEFAULT 0,
   CONSTRAINT PK_InternetSalesNew PRIMARY KEY (InternetSalesKey,
PcInternetSalesYear)
)
GO
ALTER TABLE dbo.InternetSalesNew ADD CONSTRAINT FK_InternetSalesNew_Products FOREIGN
KEY(ProductKey) REFERENCES dbo.dimProduct (ProductKey);
ALTER TABLE dbo.InternetSalesNew ADD CONSTRAINT FK_InternetSalesNew_Dates FOREIGN
KEY(DateKey) REFERENCES dbo.dimDate (DateKey);
go
ALTER TABLE dbo.InternetSalesNew REBUILD WITH (DATA_COMPRESSION = PAGE)
GO
--2013-as adatok betöltése
--mivel a tábla üres volt, a beszúrás nem kerül naplózásra
INSERT INTO dbo.InternetSalesNew (PcInternetSalesYear,ProductKey, DateKey,
   OrderQuantity, SalesAmount,UnitPrice, DiscountAmount)
SELECT CAST(SUBSTRING(CAST(OrderDateKey AS CHAR(8)), 3, 2) AS TINYINT) AS
PcInternetSalesYear,
   ProductKey, OrderDateKey,OrderQuantity, SalesAmount, UnitPrice, DiscountAmount
```

```
FROM FactInternetSales
WHERE CAST(SUBSTRING(CAST(OrderDateKey AS CHAR(8)), 3, 2) AS TINYINT) = 13
GO
--oszloptár a segédtáblára
CREATE COLUMNSTORE INDEX CSI_InternetSalesNew
ON dbo.InternetSalesNew (InternetSalesKey, PcInternetSalesYear, ProductKey, DateKey,
OrderQuantity, SalesAmount, UnitPrice, DiscountAmount);
GO
--a kulcslépés a 4. partíció betöltése
ALTER TABLE dbo.InternetSalesNew SWITCH TO dbo.InternetSales PARTITION 4
--ez nem igényelt adatátvitelt
--partíciók:
SELECT $PARTITION.PfInternetSalesYear(PcInternetSalesYear) AS PartitionNumber,
COUNT(*) AS NumberOfRows
FROM dbo.InternetSales GROUP BY $PARTITION.PfInternetSalesYear(PcInternetSalesYear)
order by PartitionNumber
PartitionNumber     NumberOfRows
1                   14
2                   2216
3                   3397
4                   52801

select count(*) from InternetSalesNew --0!
```

Mindez elvégezhető az SSMS felületen is, a Storage menün keresztül.

> **GYAKORLAT:** hozz létre particionálási sémát a FactResellerSales táblához a DueDateKey mező szerint, és töltsd be partíciónként.

### Postgres-alternatív

A Postgres-ben a partíciók közvetlen, táblaként való manipulálása szükséges.[16]

```
select * from products where productname <'N'
select * from products where productname >='N'

--a szülőtábla törlése
create table products_p --the parent table is virtual, does not store data
   (productid int, productname varchar(40), part_key char(1), unitprice money)
partition by range(part_key)

create table products_p_a_m partition of products_p for values from ('a') to ('m');  -
-mind the overlap, 'm' will go to the second partition
create table products_p_m_z partition of products_p for values from ('m') to ('z');
--create table products_p_n_z partition of products_p for values from ('n') to ('z') gyors
↪  meghajtón
tablespace very_fast_drive;

insert into products_p (productid, productname, part_key, unitprice)
   select productid, productname, substring(productname, 1,1), unitprice from
products
--a lekérdezések a megfelelő partícióba irányítódnak
select * from products_p where productname <'d' --5
--a partíciók közvetlenül is lekérdezhetők
select * from products_p_a_m where productname <'d' --5
select * from products_p_m_z where productname <'d' --0

--a partíciók egymástól függetlenül kezelhetők
--FIGYELEM: a DROP törli az adatokat is
drop table products_p_a_m;
--a DETACH lecsatolja a partíciót a szülőtábláról, de az adatokat megtartja
alter table products_p detach partition products_p_a_m;
--betöltéshez az új partíció függetlenül létrehozható, majd csatolható a szülőtáblához
alter table products_p attach partition products_p_a_m for values from ('a') to ('m');
--érdemes check-megszorításokat létrehozni a partíciókon -> segíti a lekérdezésoptimalizálót
```

[16] Lásd: https://www.postgresql.org/docs/current/ddl-partitioning.html#DDL-PARTITIONING-DECLARATIVE-BEST-PRACTICES

A Postgres-ben a táblaörökítés mechanizmusa is felhasználható a particionálás támogatásához, bár ez a folyamat bonyolultabb:[17]

1. Hozd létre az „ős” táblát, amelyből az összes partíció örökölni fog.
2. Hozz létre több „leszármazott” táblát (mindegyik az adatok egy partícióját képviseli), amelyek az ős táblából örökölnek.
3. Adj megszorításokat a partíciótáblákhoz, hogy meghatározd az egyes partíciókban lévő sorértékeket.
4. Hozz létre indexeket az ős és a leszármazott táblákon egyenként. (Az indexek nem öröklődnek az ős tábláktól a leszármazott táblákra.)
5. Írj egy triggerfüggvényt az ős táblához, amely az ős táblába irányuló INSERT-eket a megfelelő partíciótáblába irányítja át.
6. Hozz létre egy triggert, amely meghívja a triggerfüggvényt.
7. Ne feledd, újra kell definiálni a triggerfüggvényt, ha a leszármazott táblák halmaza megváltozik.

Az 5–6. lépések kiválthatók egy szabállyal, ahogy az alábbi példában látható:

```
create table products_i --the parent table is virtual, does not store data
   (productid int, productname varchar(40), unitprice money);
--drop table products_i_a_m cascade;
--drop table products_i_n_z cascade;
create table products_i_a_m () inherits (products_i);
alter table products_i_a_m add constraint df_a_m check (lower(left(productname, 1)) >=
'a' and lower(left(productname, 1)) <= 'm');
create table products_i_n_z () inherits (products_i);  --no overlap!
alter table products_i_n_z add constraint df_n_z check (lower(left(productname, 1)) >
'm' and lower(left(productname, 1)) <= 'z');

create rule products_insert_a_m as
   on insert to products_i where lower(left(productname, 1)) >= 'a' and
lower(left(productname, 1)) <= 'm'
   do instead insert into products_i_a_m values (new.*);
create rule products_insert_n_z as
   on insert to products_i where lower(left(productname, 1)) > 'm' and
lower(left(productname, 1))<= 'z'
   do instead insert into products_i_n_z values (new.*);

delete from products_i;
--a betöltött adatok a megfelelő partícióba kerülnek
insert into products_i (productid, productname, unitprice)
   select productid, productname, unitprice from products ;

--egy select az összes partícióban keres
select * from products_i --77
select * from only products_i --0
select * from products_i_a_m --41
select * from products_i_n_z --36
```

---

# 6. Adatbázis-migráció

Az adatbázis-migráció az adatbázis-technológia lecserélését vagy átalakítását jelenti egy információs rendszerben. A probléma összetettsége a változás mértékétől függ:

- Ugyanaz a fő technológia, ugyanaz a gyártó, más verzió. Példa: Postgres 8 → 17
- Ugyanaz a fő technológia, más gyártó. Példa: Postgres 17 → MS SQL Server 2019
- Más fő technológia, pl. relációs, gráf, dokumentumtároló, kulcs-érték tároló. Példa: Postgres → Firestore

[17] Lásd: https://medium.com/timescale/scaling-partitioning-data-postgresql-10-explained-cd48a712a9a1

## Migráció relációs technológiák között

A **relációs → relációs migráció** elvégezhető importálási/exportálási funkciókkal vagy a forrásadatbázisból generált dump kézi szerkesztésével — de sok esetben csak specializált eszköz vagy egyedi fejlesztésű szoftver tudja megoldani a problémát. Tipikus problémák:

- Adattípusok
- SQL DDL/DML szintaxis-különbségek
- Hitelesítési modellek közötti különbségek
- A szerveren futó logikát (pl. tárolt eljárások) újra kell implementálni

Ebben a demóban az SQL Server Northwind adatbázisát migráljuk Postgres-re. Telepítsd a Postgres-t és a pgAdmin-t, majd futtasd a `pg_script.sql` szkriptet a `postgres` adatbázisban. A szkript tartalma a következő:

1. Létrehozza a Products, Orders, Order details, Customers táblákat, és betölti az adatokat
2. Létrehoz egy új tárolt függvényt, amely támogatja egy új rendelés létrehozását
3. Létrehoz egy új nézetet, amely listázza az utolsó 5 rendelés adatait

Megjegyzések:

A szkriptet az SSMS által generált dump szkript alapján manuálisan szerkesztettük.[18]

Példa az alapértelmezett megszorítás két dialektusára:

- Postgres: `ALTER TABLE products ALTER COLUMN Unitprice SET DEFAULT 0`
- SQL Server: `ALTER TABLE products ADD CONSTRAINT DF_Products_UnitPrice DEFAULT ((0)) FOR UnitPrice`

Másik példa a Boolean típusra: SQL Server elfogad 0/1 értéket, a Postgres nem.

A szkript tartalmaz egy tárolt függvényt, amely tranzakciót valósít meg. Ennek a tranzakciónak az **atomi tulajdonsága (atomicitása)** fontos üzleti szabály, amelyet be kell tartani.

---

## Migráció relációsból dokumentumtárolóba

### Adatmodellek: relációs vs. dokumentumtároló

A relációs adatmodellt a tartománymodell határozza meg, és 3NF-ben normalizált. A relációs modell univerzális: bármely lekérdezés hatékonyan futtatható rajta. A modell indexek segítségével finomhangolható egy adott alkalmazáshoz.

A noSQL modellekkel — például dokumentumtárolókkal és kulcs-érték tárolókkal — a megfelelő és hatékony adatmodell csak a tervezett alkalmazás és a várható lekérdezések figyelembevételével határozható meg. Emiatt míg egy relációs modell viszonylag simán migrálható egy másik relációs technológiába, a relációs modell noSQL modellbe való átültetésére nincs egyszerű vagy automatizált megoldás.

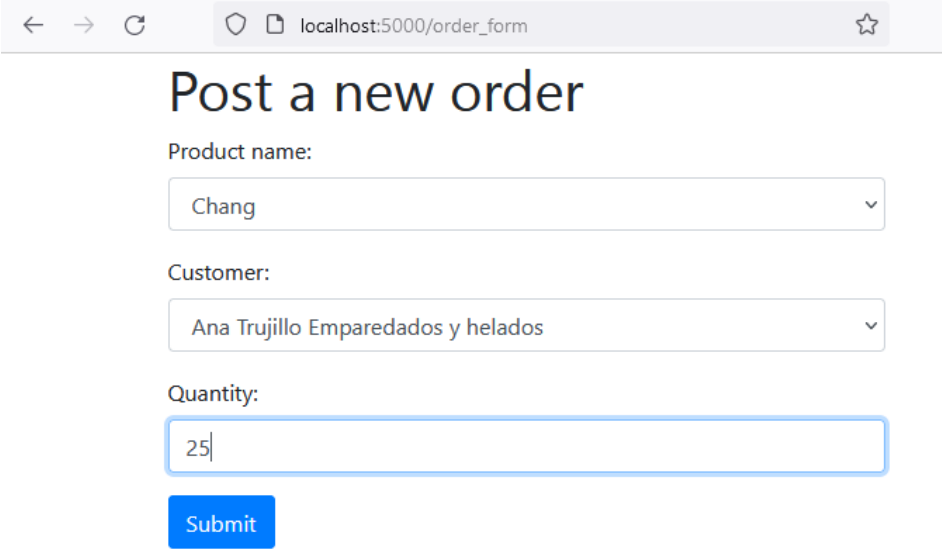
A migráció különösen nehézkes, ha az adatmodell összetett relációs struktúrákat tartalmaz. Míg a hierarchikus egy-a-többhöz kapcsolatok könnyen leképezhetők noSQL struktúrákra, a **több-a-többhöz** kapcsolatok külön tervezést igényelnek. A struktúrát kézzel kell megtervezni, és egyedi alkalmazást kell megvalósítani az adatbetöltési folyamat kezelésére.

### A demóalkalmazás

A migrációs folyamat bemutatásához egy egyszerű Python Flask alkalmazást használunk. Az alkalmazás a Northwind adatbázis Orders, Order details, Products és Customers tábláit használja.

[18] Erről bővebben: https://severalnines.com/database-blog/migrating-mssql-postgresql-what-you-should-know

Funkcionalitása:




- A felhasználó egy legördülő listából kiválaszthat egy terméket, megadhat egy mennyiséget, és feladhat egy rendelést a Northwind adatbázisban. A visszaadott oldal megerősíti a rendelést, és listázza az utolsó 5 rendelést.
- Az alkalmazás hibaüzenetet ad, ha a termék készlete vagy az ügyfél egyenlege túl alacsony.
- A rendelés feladása tranzakcióban fut.

### A demóalkalmazás futtatása localhostról Postgres háttérrel

Végezd el a következő lépéseket:

1. (*már telepítve a VM-en*) Telepítsd az Anaconda Pythont.
2. (*már telepítve a VM-en*) Hozz létre virtuális környezetet a projekthez:[19]
   - Anaconda admin prompt: `set https_proxy=http://proxy.uni-pannon.hu:3128`
   - `http://proxy.mik.uni-pannon.hu`
   - (ellenkező esetben 'SSL: WRONG_VERSION_NUMBER' hibát kapsz)
   - `conda create -n flask_demo pip` //python=3.8.5
   - python verzió: `python -version` //3.9.11
   - elérhető virtuális környezetek: `conda env list`
   - `conda activate flask_demo` (később telepítjük a szükséges csomagokat)
   - `pip install flask`
   - `pip install psycopg2`
3. Töltsd le a `northwind.py` alkalmazást, és lépj be a Flask projekt mappájába (vagyis a `flask` nevű könyvtárba).
4. Indítsd el az Anaconda Spyder Python szerkesztőt, és állítsd be a jelszót és az 5432-es portot a 59. sorban. Nézd át az alkalmazást. Megjegyzendő, hogy a `with connection:` blokk a `new_order()` függvény hívását **tranzakcióban** futtatja.
5. A conda parancssorban: `set FLASK_APP=northwind.py` (a `DEBUG=1` beállítással nem kell manuálisan újraindítani a webszervert minden módosítás után). Megjegyzés: a GCP-n `main.py` névként fogjuk használni, így ott nincs szükség különleges beállításra.

[19] Lásd: https://blog.usejournal.com/why-and-how-to-make-a-requirements-txt-f329c685181e

6. `flask run`
7. Teszteld a lapot a `http://localhost:5000/order_form` címen. Felhasznált technológiák:
   - a. A `templates` mappa tartalmazza a jinja template engine által feldolgozott HTML lapokat. A Python változók elérhetők a HTML sablonokban (`{% for ... in / if ...else / with / block, extends %}`), és a sablonok egymásba ágyazhatók a 'block' segítségével.[20]
   - b. Bootstrap (formázáshoz)
8. Az alkalmazás tovább javítható vagy biztonságosabbá tehető, de ez most nem célunk.[21]

---

## Dokumentumtárolók: A Cloud Firestore áttekintése

Ezt az alkalmazást a Google Cloud Firestore-ba migráljuk. A Google Cloud Firestore egy dokumentumtároló. Több dokumentumgyűjtemény tárolását támogatja. A korábbi Realtime Database új kiadása, amely egyetlen monolitikus JSON-t használt.[22] A Cloud Firestore jellemzői:

- „Firestore adatbázis = dokumentumok gyűjteményei".
- Minden dokumentum lényegében egy JSON dokumentum, pl.:

```
name :
      first : "Joe"
      last : "Long"
born : 1995
```

- A gyűjtemény **nem kényszerít ki sémát**, vagyis tetszőleges szerkezetű dokumentumokat tárolhat ugyanabban a gyűjteményben. Mindazonáltal még ha nincs is sémaellenőrzés beszúráskor, minden valósághű alkalmazás, amely a gyűjteményt olvassa, egy bizonyos sémát vár — ez a **schema-on-read (olvasáskori séma)** rendszer. Ez ellentétben áll a relációs adatbázisok **schema-on-write (íráskori séma)** megközelítésével, ahol a sémának nem megfelelő írások nem engedélyeztek.
- Minden dokumentumnak egyedi kulcsa kell legyen, amely automatikusan generálható, de **nem lehet kizárólag szám**.
- A gyűjtemények nem ágyazhatók egymásba. Egy dokumentum azonban tartalmazhat egy hozzá csatolt algyűjteményt, egy coll-doc-coll-doc-coll-... stb. hierarchiában, legfeljebb 100 mélységig.
- Speciális dokumentum adattípusok: 1-D tömb, map (asszociatív tömb), reference típus. A referencia egy dokumentumhoz vezető elérési út string, pl. egy hierarchiában lévő dokumentumra mutató referencia. Formája lehet: `coll_id/doc_id/coll_id/...` stb. Algyűjteményekkel kapcsolatos lehetséges probléma, hogy 'árva' állapotban is fennmaradhatnak, ha a szülő gyűjtemény törlésre kerül.
- Az egy-a-többhöz kapcsolatok beágyazással valósíthatók meg; **a több-a-többhöz gyűjtemények beágyazás és referenciák kombinációjával** valósíthatók meg.
- A Firestore valós idejű egyidejű frissítéseket és atomi tranzakciókat támogat. A lekérdezések szűrőkkel valósíthatók meg.

[20] Flask app: https://www.youtube.com/watch?v=MwZwr5Tvyxo / HTML sablonok: https://www.youtube.com/watch?v=QnDWIZuWYW0
[21] WTF Forms: https://www.youtube.com/watch?v=UIJKdCIEXUQ / Database ORM: https://www.youtube.com/watch?v=cYWiDiIlUxQc
[22] Összehasonlításért lásd: https://firebase.google.com/docs/firestore/rtdb-vs-firestore#key_considerations

- Egy Realtime/snapshot listener eseményt generál, amikor egy dokumentum megváltozik, és egy alkalmazás egy visszahívó függvénnyel (callback) feliratkozhat erre az eseményre.[23]

### A Firestore dokumentumtárunk tervezése, létrehozása és feltöltése

Az adatokat újra kell modellezni — ez alkalmazásfüggő folyamat. Tervezett lekérdezéseink:

- Termékek és ügyfelek listázása
- Új rendelés feladása egyetlen tétellel
- Az utolsó 5 rendelési tétel listázása a rendelés- és ügyfélAdatokkal

E lekérdezések figyelembevételével 3 gyűjteményt tervezünk: egy lapos Products, egy lapos Customer és egy hierarchikus Orders→Orderitems gyűjteményt. Megjegyzendő, hogy ez nem illik jól egy olyan lekérdezéshez, amely azt kérdezi, mi lett megrendelve és mikor — de ilyen lekérdezés (még) nem része az alkalmazásunknak.

Alternatívaként az Orders→Orderitems gyűjteményt a Customers gyűjteménybe ágyazhatnánk, de ez megnehezítené az időrend szerinti rendeléslista lekérdezését, mivel az algyűjtemények lekérdezése általában **nem engedélyezett** a szülő dokumentumgyűjtemény iterálása nélkül.

Egyéb megfontolások:

- Automatikus dokumentum ID-kat használunk
- A rendeléseket dátum szerint rendezzük az utolsó 5 tétel listázásának támogatásához
- A CustomerID automatikusan indexelődik

A migrációs folyamatnak 2 fő lépése van:

1. Az új adatbázis megvalósítása és az adatok átalakítása/betöltése
2. A kliens alkalmazás újraimplementálása az új adatbázishoz

### Az új adatbázis megvalósítása és az adatok átalakítása/betöltése

A dokumentumtárat programozottan hozzuk létre és töltjük fel: a localhostlon futó Postgres adatbázisból olvasunk, és a felhőbeli dokumentumtárba írunk.

GCP szolgáltatásfiókot kell létrehozni a Firebase tároló eléréséhez az asztali gépünkről.[24]

1. Nyisd meg a Firebase konzolt: https://console.firebase.google.com/
    - a. Google-hitelesítés után hozz létre egy új projektet
    - b. Nincs szükség sem Gemini-re, sem Google Analytics-re ebben a projektben
    - c. Lépj a https://console.firebase.google.com/u/0/project/[Your_project_ID]/firestore oldalra, és hozz létre egy adatbázist. Kísérletezz manuálisan gyűjtemények és dokumentumok hozzáadásával/törlésével/szerkesztésével itt vagy a GCP felületen.
    - d. Hozz létre egy books nevű gyűjteményt két könyv-dokumentummal
    - e. A Firebase konzolban: fogaskerék ikon → Service accounts → Create service account → Generate new private key. Itt generálhatunk egy JSON-fájlt, amely az automatikusan létrehozott szolgáltatásfiók kulcsát tartalmazza. Mentsd el ezt a fájlt biztonságos helyre, mert hozzáférést biztosít az adatbázishoz.
    - f. Python kód is található itt a csatlakozáshoz.
2. (*már telepítve a VM-en*) A conda parancssorban, lépj a virtuális környezetbe és futtasd: `pip install firebase_admin`

[23] Lásd: https://firebase.google.com/docs/firestore/query-data/listen

[24] Az alkalmazáson belüli GCP-projektből nem lenne szükségünk szolgáltatásfiókra a tároló eléréséhez, de sokkal egyszerűbb és gyorsabb az alkalmazást helyben fejleszteni és hibakeresni, majd szükség esetén GCP-re telepíteni. Lásd még: https://cloud.google.com/compute/docs/access/create-enable-service-accounts-for-instances

3. Mostantól csatlakozni tudsz az adatbázishoz: `db = firestore.client()`
    - a. Ellenőrizd, hogy a Postgres szolgáltatás fut, és a Northwind táblák elérhetők
    - b. Győződj meg arról, hogy a proxy KI van kapcsolva a VM-en
    - c. Tekintsd át és futtasd a `flask_gcp_firestore` mappában lévő `main.py` Python szkriptet, amely betölti a 3 gyűjteményt. Ehhez nincs szükség Flask webszerverre; a szkriptet a conda parancssorból is elindíthatod: `python main.py`[25]
    - d. Ellenőrizd a gyűjtemények tartalmát a Firebase konzolban.

**GYAKORLAT:** tekintsd át és futtasd az új `northwind.py` webalkalmazást, amely a Firestore adatbázist használó rendelés-feldolgozási tranzakciót valósítja meg.

- Manuálisan töröld a `main.py` által beszúrt demo rendelést.
- Valósítsd meg a hiányzó `list_orders()` függvényt, amely összegző sort ad vissza az utolsó 5 rendelés minden rendelési tételéhez. Az eredményt az `order_list.html` sablon jeleníti meg.

A reprodukálandó funkcionalitás a következő:

```
select o.orderdate::timestamp(0) without time zone as orderdate,
c.companyname, c.country, c.balance, p.productname, od.quantity,
od.quantity * od.unitprice as value, p.unitsinstock
from products p join orderdetails od on p.productid=od.productid
    join orders o on o.orderid=od.orderid
    join customers c on c.customerid=o.customerid
order by orderdate desc limit 5;
```

- Ugyanazokat a HTML sablonokat használhatod.
- Teszteld a webalkalmazást a `http://localhost:5000/` címen.

---

## Az adatintegritás érvényesítésének összehasonlítása

Az alábbi táblázat összehasonlítja a Postgres és Firestore adatintegritást érvényesítő eszközeit:

| **Postgres** | **Firestore** |
|---|---|
| A séma minden DML-műveletben érvényesített: ‘schema on write’ | Egy gyűjtemény minden dokumentumának eltérő attribútumai (mezői) lehetnek: ‘schema on read’ |
| Gazdag adattípus-készlet és felhasználó által definiált adattípusok (pl. enumerációs típusok) | Nagyon korlátozott adattípus-készlet, pl. nincs pénz vagy decimális típus |
| Elsődleges kulcsok és idegen kulcsok | A gyűjtemény azonosítójának egyedinek kell lennie és automatikusan generálható (mint a serial típus), de nem lehet kizárólag szám. Az idegen kulcsok attribútumként tárolhatók és biztonsági szabályokkal érvényesíthetők. |
| Egyedi megszorítások és check-megszorítások | Biztonsági szabályok[26] |

[25] Ha hitelesítési hibát kapsz, az valószínűleg a virtuális gép helytelen rendszeridő-beállítása miatt van. A dátum és idő beállítás rendszerpanelén módosítsd a rendszeridőt kézire, majd vissza automatikusra az idő frissítéséhez.

[26] A Firebase Admin SDK-t használó kliensalkalmazás adminisztrátori jogosultságokkal fut, és megkerüli a Firestore adatbázishoz definiált biztonsági szabályokat. Biztonsági szabályokat csak mobil kliensen lehet bemutatni. Ezért a szabályok ebben a szövegben nem kerülnek részletezésre. Bővebb információért lásd: https://firebase.google.com/docs/firestore/security/get-started

| Postgres | Firestore |
|---|---|
| ACID tranzakciós biztonság támogatott | A tranzakciók elindíthatók a kliensről a test-and-set típusú hibák megelőzésére, és az atomicitás kötegelt írásokkal megőrizhető,[27] de a tranzakciók nem kombinálhatók kötegelt írásokkal (lásd később). |

---

## Tranzakciók Firestore-ban és Postgres-ben

**DEMO:** teszteld a rendelési tranzakció atomicitását Postgres-ben. A PgAdmin konzolban injektálj hibát a `new_order()` függvénybe az alábbi utasítás felülírásával:

```
insert into orders (orderid, customerid) values (var_orderid, var_custid);
```

ezzel az utasítással:

```
insert into orders (orderid, customerid) values (var_orderid, 'ERROR');
```

Ez idegen kulcs-megszorítást sért, és futásidejű hibát okoz a következő hívásnál. A webalkalmazást használva adj fel egy új rendelést, és ellenőrizd, hogy a hiba után az ügyfél egyenlege visszaáll az eredeti értékre — annak ellenére, hogy az INSERT utasítás előtt már csökkentve lett. *A tranzakció automatikusan visszagörgetett.*

**DEMO:** logikai hibák Firestore-ban párhuzamos környezetben

- Adj hozzá 10 másodperces késleltetést a készlet és egyenleg ellenőrzése után (`import time` és `time.sleep(10)`)
- Állítsd be kézzel mindhárom ügyfél egyenlegét 10 000-re, és a Chai termék készletét 3-ra a Cloud Firestore webkonzolon
- Indítsd el az alkalmazást 3 böngészőablakban, és rendelj mindegyik ügyfélnek 2 egység Chai-t (értéke: 2×18=36). Elvárjuk, hogy a korlátozott készlet miatt *csak egy rendelés legyen sikeres*.
- Ellenőrizd: mindhárom rendelés SUCCESS eredménnyel kerül feldolgozásra, mindhárom ügyfél egyenlege 36-tal csökken, és az új készlet 1. Látható, hogy tranzakciós ellenőrzés nélkül *súlyos logikai hibák fordulhatnak elő*.

### Firestore-tranzakció használata

Egy Firestore-tranzakció olvasási műveleteket tartalmazhat, amelyeket írási műveletek követnek. *Ha egy tranzakció olvas egy dokumentumot, majd a dokumentumot egy másik kliens módosítja a commit előtt, a tranzakció automatikusan visszagörgetésre kerül és újrafuttatódik (legfeljebb 5-ször).* Az atomicitás egyetlen commit művelet szintjén garantált, de párhuzamos módosítások esetén a tranzakció újrafuttatása szükséges lehet. Példa:

```
# create a new customer called test manually and set its balance to 9
# then run this script in two parallel sessions

import firebase_admin
from firebase_admin import credentials
from firebase_admin import firestore
import time

cred = credentials.Certificate("token.json")
firebase_admin.initialize_app(cred)
db = firestore.client()
transaction = db.transaction()
```

[27] Lásd: https://firebase.google.com/docs/firestore/manage-data/transactions#python_1

```
cust_ref = db.collection('customers').document('test')

@firestore.transactional
def update_in_transaction(transaction, cust_ref):
  cust_ref_snapshot = cust_ref.get(transaction=transaction)
  new_balance = cust_ref_snapshot.get('balance') + 1
  print("New balance: ", new_balance)
  time.sleep(10)
  if (new_balance <= 10):
    transaction.update(cust_ref, {'balance': new_balance})
    print("Balance increased")
  else:
    print("Balance already maximal")

update_in_transaction(transaction, cust_ref)
```

Állítsd be a tesztvevő egyenlegét kézzel 9-re, majd futtasd a fenti szkriptet két párhuzamos munkamenetből. Bár mindkét munkamenetből megkapjuk a 'Balance increased' üzenetet, a végső egyenleg 10 lesz (és nem 11), mivel az egyik tranzakció (amelyik a 'Balance already maximal' üzenetet adta vissza) visszagörgetésre került és újrafuttatódott. Ezért *sikerült érvényesítenünk azt az üzleti szabályt, hogy az egyenleg nem haladhatja meg a 10-et*, még párhuzamos esetben sem.

> **GYAKORLAT:** a fenti kódrészletet sablonként használva add hozzá a tranzakciós vezérlést a Firestore-alapú Python Flask alkalmazás `order_proc` függvényéhez, és teszteld a helyes működését a Párhuzamossági demó részben leírt forgatókönyvben. Elvárjuk, hogy több párhuzamos tranzakció közül *csak egy* fusson sikeresen, és az adatbázis konzisztenciája megmaradjon. Ez azt jelenti, hogy az összes tranzakció végén a készlet helyes legyen, *csak egy* rendelés kerüljön elfogadásra, és *csak egy* ügyfél egyenlege csökkenjen.

### Kötegelt írások (Batched writes)

Több 'doc.set', 'update' vagy 'delete' műveletet tartalmazó köteg atomicitása a `db.batch()` objektummal érvényesíthető. Megjegyzendő, hogy ez a megoldás nem nyújt védelmet a kliens által már beolvasott dokumentumokon végrehajtott külső frissítések ellen — ezért *nem képes megakadályozni a 'test-and-set' típusú logikai hibákat.*[28]

*Nem lehetséges kötegelt írásokat tranzakcióba ágyazni. Ezért dönteni kell: az ACID-követelmények atomicitása vagy az izoláció biztosítása a cél.*

A tranzakciók csak online működnek, azaz szerverkapcsolat szükséges, de a kötegelt írások offline is működhetnek.

> **DEMO:** tekintsd át és teszteld a demóalkalmazás kötegelt verziót (`northwind_batch_write.py`) egy futásidejű hiba injektálásával, pl. `print(1/0)` két írás közé.

Néhány további Firestore-korlát:

- Egy tranzakció vagy köteg legfeljebb 500 dokumentumba írhat
- Az olvasási műveleteket (`get()`) a írási műveletek (`set()`, `update()`) előtt kell elvégezni
- „A tranzakciókra vagy kötegelt írásokra vonatkozó biztonsági szabályokban az atomi művelet egészére vonatkozóan legfeljebb 20 dokumentum-hozzáférési hívás engedélyezett, a kötegben lévő minden egyes dokumentumra vonatkozó normál 10 hívási korláton felül."
- stb. …

*Általános következtetésként megállapítható, hogy a NoSQL-technológiák esetén a kliensnek (vagy az üzleti logikai rétegnek) kell támogatnia és megvalósítania az összes alacsony szintű adatfeldolgozási műveletet, mint például gyűjtemények összekapcsolása stb. A tranzak-*

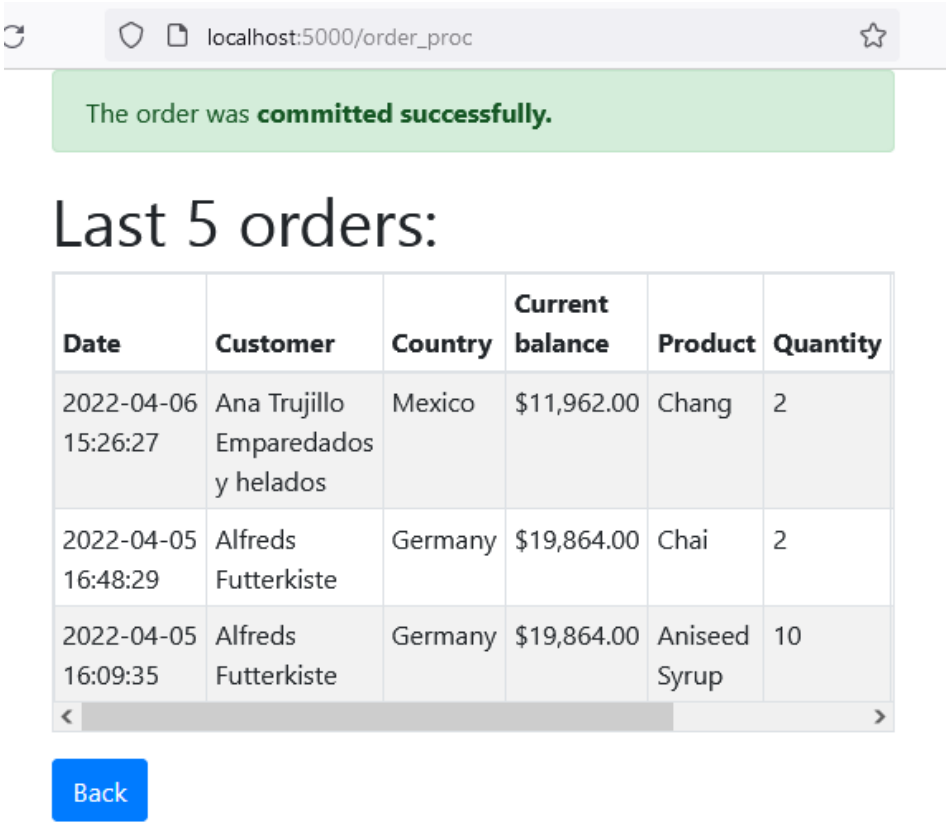
[28] Vö.: „Firestore allows you to run sophisticated ACID transactions against your document data." https://cloud.google.com/firestore

*ciós vezérlés és az adatintegritás érvényesítése is kevésbé robusztus a relációs adatbázis-technológiákhoz képest. Ezért a NoSQL-technológiák viszonylag* ***egyszerű üzleti területek és folyamatok*** *esetén ajánlhatók — például egy bloggeroldal esetén.*

**Egyéb érdekes Firestore-funkciók (nem részletezve)**

- Valós idejű frissítések
- Offline adatpersistálás
- Integráció felhőszolgáltatásokkal

---




# 7. Felhőalapú adatbázis-technológiák

## Áttekintés

Az olyan felhőszolgáltatók, mint az Amazon, a Google, az IBM és a Microsoft, adatbázis-kezelési szolgáltatásokat kínálnak SaaS vagy PaaS alapon. Egy ilyen megoldás alkalmazhatósága a saját adatbázis-szerverek futtatásával szemben gazdasági és részben műszaki szempontoktól függ. Néhány szempont:

**Előnyök:**

- A felhőszolgáltatások magas rendelkezésre állást (a vállalati szervereket meghaladó szinten) nyújtanak, földrajzilag elosztott infrastruktúrával.
- A felhőszolgáltatások automatizált, igény szerinti méretezhetőséget kínálnak rugalmas, legalább részben terhelésfüggő számlázási sémákkal.

- A felhő számos egyéb felhőn belüli szolgáltatást is kínál, mint például adatelemzés, képfeldolgozás, NLP stb., amelyek hozzáférnek a felhőalapú adatbázishoz, és integrálhatók egy megoldásba.

**Hátrányok:**

- A felhőalapú adatbázis-szolgáltatások által nyújtott funkcionalitás jellemzően korlátozottabb, mint a saját szervereké, bár egyes funkciók más eszközökkel pótolhatók.
- Bár egyes költségek arányosak a felhasználással, fix költségek is vannak, pl. az adattárolásé. Ezeket a tervezett szolgáltatás vagy megoldás üzleti tervében figyelembe kell venni.
- Az adatbiztonság veszélybe kerülhet, ha erősen érzékeny adatok (pl. személyes egészségügyi rekordok) fizikailag elhagyják a vállalati hálózatot. Biztonsági incidensek vagy súlyos adatvesztés is előfordulhat.[29]
- A kritikus szolgáltatások, pl. a gyártásvezérlő rendszerek nem függhetnek az internethozzáféréstől. Ez utóbbi két aggályt egy **privát felhő**[30] enyhítheti.

---

## Új GCP-projekt indítása

Ebben a projektben az App Engine-t használjuk a Flask alkalmazás futtatásához, egy felhőalapú SQL adatbázist és egy Firestore dokumentumtárat.

1. Indíts el egy privát böngészőt, és engedélyezd a felugró ablakokat.
2. Aktiváld a 50 dolláros kupont az egyetemi e-mail-címeddel, az Neptun-üzenetben kapott utasítások szerint.
3. Lépj be a https://console.cloud.google.com oldalra.
4. Indíts el egy új projektet, vagy töröld a meglévőt: IAM & Admin → Manage resources.
5. Meg kell adnod egy globálisan egyedi projektnevet, amelyre ebben a kézikönyvben YOUR-PROJECT-ID-ként hivatkozunk. Ellenőrizd a projekthez kapcsolt számlát — induláskor 50 dolláros kredittel kell rendelkeznie.
6. A Firestore oldalon válaszd: SELECT NATIVE MODE, majd CREATE DATABASE. Egy GCP-projektben csak egyetlen Firestore adatbázis engedélyezett.

---

## A Postgres adatbázis migrálása GCP-re

1. GCP Databases → SQL → Create instance → PostgreSQL (engedélyezni kell a Compute Engine API-t).
2. Válassz egy példányazonosítót és jelszót. A példány neve az adatbázis-szerver neve, nem az adatbázisé vagy az adatbázis-felhasználóé. Állítsd be:
   - Régió: Europe, PostgreSQL 13-as verzió, single zone (feladatátvétel nélkül)
   - Testreszabás: publikus IP engedélyezve
3. Kapcsolatok: „Ebben a projektben az összes alkalmazás alapértelmezetten engedélyezett”, azonban tervezzük az adatbázis elérését az interneten keresztül PgAdmin-ból, ezért engedélyezünk bármilyen IP-t (=pg_hba.conf bejegyzés). Hálózat hozzáadása: `0.0.0.0/0`. Mentés.
4. A Speciális beállításokban (Advanced options) engedélyezd a Query Insights funkciót.
5. Várd meg, amíg a példány elkészül, majd változtasd meg a Postgres-felhasználó jelszavát. Az SQL Overview-ra navigálva megtalálod a szerver publikus IP-címét.

[29] Lásd: https://www.theguardian.com/australia-news/article/2024/may/09/unisuper-google-cloud-issue-account-access

[30] Pl. https://www.openstack.org/

6. PgAdmin-ban csatlakozz a felhőpéldányhoz a publikus IP-cím segítségével, majd tekintsd át és futtasd a `pg_script.sql` dump-fájlt.
7. Módosítsd a Python alkalmazásban az IP-t a nyilvános GCP IP-re, és teszteld az alkalmazást. Az új rendeléseknek meg kell jelenniük a Postgres táblákban.
8. Visszatérve a GCP-re, nyisd meg a Query Insights-ot, és ellenőrizd a `SELECT * FROM last_orders` lekérdezés végrehajtási tervét. Megjegyzendő, hogy egy lekérdezés megjelenése a Query Insights statisztikákban akár 10 percet is igénybe vehet — ez egy offline elemzési eszköz.

---

## A demóalkalmazás megvalósítása GCP-n[31] → Cloud programozás MSc kurzus

Az alkalmazást a GCP App Engine-re telepítjük.[32]

1. (*már telepítve a VM-en*) Telepítsd a Cloud SDK-t és a gcloud CLI-t az asztali számítógépedre.
2. Nyiss meg egy cmd terminált **Windows rendszergazdaként**, lépj be a `flask_gcp` könyvtárba, és állítsd be: `HTTPS_PROXY=http://proxy.uni-pannon.hu:3128`
3. A parancssorban: `gcloud init` (saját Google-profillal való hitelesítés szükséges), majd válaszd ki a YOUR-PROJECT-ID-t.
4. Az SDK nem tartalmazza az App Engine bővítményt, ezért ezt külön kell telepíteni. A Google Cloud SDK shell parancssorában: `gcloud components install app-engine-python`. Ez néhány percet vesz igénybe.
5. `gcloud config set project YOUR-PROJECT-ID`
6. `//check billing: gcloud beta billing accounts list`
7. A `northwind.py` szkriptben módosítsd az adatbázis-kapcsolati paramétereket a nyilvános GCP IP-re, és mentsd el `main.py` névvel. (Frissítsd a `requirements.txt` fájlt is.)
8. A tároló (container) futtatásához a Cloud Build API szükséges, ezért ezt engedélyezni kell a projektben: `gcloud services enable cloudbuild.googleapis.com`
9. Inicializáld az App Engine-t a projektben: `gcloud app create --project=YOUR-PROJECT-ID`
10. `gcloud app deploy` Megkapod a cél URL-t, amelyen az alkalmazás elérhető. A böngészőt az alkalmazáshoz nyithatod meg: `gcloud app browse`
11. A futásidejű hibaüzeneteket az App Engine webkonzolon tekintheted meg: App engine → Services → Diagnose: Logs.

    **GYAKORLAT:** javítsd a GCP alkalmazást azzal, hogy megjeleníted az esetleges hiba okát is — vagyis az alacsony egyenleget vagy az alacsony készletet, az alábbiak szerint. Ehhez a `new_order()` tárolt eljárás és az ügyfél HTML-sablonok kis módosítása szükséges. A módosított alkalmazást telepítsd: `gcloud app deploy`

[31] Ha már jól ismered a felhőprogramozási koncepciókat, kihagyhatod ezt a részt, és folytathatod az alkalmazással localhostról futtatva.

[32] Lásd: https://medium.com/@dmahugh_70618/deploying-a-flask-app-to-google-app-engine-faa883b5ffab

The order **failed**. ERROR: Balance too low

# Last 5 orders:

| Date | Customer | Country | Current balance | Product | Quantity | Value | Current stock |
|---|---|---|---|---|---|---|---|
| 2022-04-06 14:09:11 | Ana Trujillo Emparedados y helados | Mexico | $11,690.00 | Northwoods Cranberry Sauce | 5 | $200.00 | 1 |
| 2022-04-06 14:09:00 | Ana Trujillo Emparedados y helados | Mexico | $11,690.00 | Chef Anton's Cajun Seasoning | 5 | $110.00 | 48 |
| 2022-04-06 14:02:47 | Alfreds Futterkiste | Germany | $19,938.00 | Ikura | 1 | $31.00 | 21 |

Back

**MÉG TÖBB GYAKORLAT:** oszd fel a fenti táblázatot két táblára, amelyek az utolsó 5 rendelést és az érintett ügyfelek egyenlegét külön jelenítik meg.

**Mielőtt elmész:**

1. Állítsd le az alkalmazást: App engine → Settings → Disable application
2. Állítsd le a Postgres szervert: SQL → Overview → STOP
3. Kapcsold le a GCP-projektet.

---

## BigQuery áttekintés és bemutató

„A BigQuery standard SQL segítségével strukturált és félig strukturált adatok lekérdezésére lett tervezve.”[33]

A BigQuery denormalizált (ellapított) adattárházi táblákon futtatandó **csak olvasható** OLAP-jellegű lekérdezésekre lett tervezve döntéstámogatási és üzleti elemzési célokra. A natív tárolási modell egy tömörített (és opcionálisan particionált) oszloptár (neve: Dremel).[34] Az összes adat titkosított formában tárolódik. A BigQuery közvetlenül futtathat lekérdezéseket más adatforrásokon is, mint például Bigtable, Cloud storage vagy Google Drive — de a legjobb teljesítmény érdekében az ilyen külső forrásokból származó adatokat először érdemes importálni a Dremel oszloptárba.

A logikai adatmodell egy adatkészlet (Dataset), amely kapcsolódó táblákat tartalmaz rögzített sémával és típusos mezőkkel; hivatkozása: `project.dataset.table`

### A Northwind adatbázis bányászata

Ebben a demóban:

- Az SQL Server verzióját a Northwind adatbázisnak használjuk
- Létrehozunk egy denormalizált nézetet, és exportáljuk CSV-be
- Betöltjük a CSV-t egy BigQuery táblába

[33] Lásd: https://panoply.io/data-warehouse-guide/bigquery-architecture/
[34] Lásd: https://medium.com/google-cloud/bigquery-explained-storage-overview-70cac32251fa

- A táblát felhasználjuk egy lineáris regressziós BigQueryML modell létrehozásához
- Értékeljük a modell pontosságát

Lépések:

1. Hozz létre egy nézetet, amely a következő mezőket tartalmazza: `value_numeric`, `country`, `categoryid`, `p_unitprice`, `discount`, `pyear`
2. Exportáld a nézetet CSV-fájlba.
3. Chrome böngészőt használva hozz létre egy új projektet: GCP főmenü → IAM&admin → Manage resources.
4. GCP főmenü → Analytics → BigQuery, Enable API.
5. A projekt „...” menüjéből → Create dataset, válaszd a Multi-region ‘US’ lehetőséget. Meg kell adnod egy azonosítót. Nevezd el az adatkészletet ‘northwind’-nek. Az adatkészlet titkosított formában kerül tárolásra. Lépj az adatkészletre.
6. Válaszd a + create table lehetőséget, és add meg: Create table from upload → schema auto-detect. Töltsd be a denormalizált táblát, és ellenőrizd az adattípusokat. Nevezd el a táblát ‘nw’-nek. A money (pénz) típus problémát okozhat.
7. Tekintsd át a sémát és az adatok előnézetét.
8. Érdemes ellenőrizni a forrásadatok érvényességét és tartalmát elemzés előtt. A GCP Analytics → Dataplex szolgáltatás ezt támogatja:
   - a. A Dataplex-ből vagy közvetlenül a BigQuery Studióból (a Schema fül melletti utolsó fül) hozz létre és indíts el egy igény szerinti Profile task feladatot a Quick data profile → Details menüponton. Ez a feladat kb. 5 percig fut a háttérben, majd megjeleníti az oszlopstatisztikákat és hisztogramokat.
   - b. Indíts el egy Data quality scan feladatot is egy manuálisan hozzáadott minőségi szabállyal (pl. Freight >=0). A profil feladat eredménye alapján a ‘de facto’ szabályokat is áttekintheted és elfogadhatod (pl. a Freight mezőre levezetett szabály: “min: 0.02, max:1007.64”). Az ellenőrzéssel kapcsolatos adminisztratív adatok a Northwind adatkészletben tárolhatók. Az eredmények a Dataplex → Data quality oldalon az ellenőrzés nevére kattintva, a Jobs history fülön érhetők el.
9. A BigQuery-ben SQL lekérdezések futtathatók a konzolon: Compose new query → SQL console window.
10. Hozd létre azt a modellt, amely a `value_numeric` mezőt becsüli a `country`, `categoryid`, `p_unitprice`, `discount`, `pyear` mezők alapján (ha az adatkészletet ‘northwind’-nek és a táblát ‘nw’-nek nevezted el):[35]

```
CREATE MODEL `northwind.nw_model` OPTIONS (model_type='linear_reg',
↪  input_label_cols=['value_numeric']) AS
SELECT value_numeric, country, categoryid, p_unitprice, discount, pyear
FROM `northwind.nw`
WHERE value_numeric is not null
```

11. Építsd fel a `northwind.nw_model` modellt (kb. 30 másodperc), majd lépj a modelre. Ellenőrizd a Details és Evaluation füleket. Látható, hogy az adatkészlet automatikusan felosztásra kerül egy tanuló táblára (1702 rekord) és egy kiértékelő táblára (453 rekord). Az átlagos abszolút hiba 301.
12. A modellt a teljes adatkészlettel is kiértékelhetjük (bár ez adattudományos szempontból nem ajánlott bevált gyakorlat — csak az EVALUATE módszer bemutatása céljából tesszük). Használd a következő parancsot:

[35] Lásd: https://cloud.google.com/bigquery-ml/docs/reference/standard-sql/bigqueryml-syntax-create

```
SELECT * FROM ML.EVALUATE(MODEL `northwind.nw_model`, (
 SELECT value_numeric, country, categoryid, p_unitprice, discount, pyear
 FROM `northwind.nw`
))
```

13. Az eredmények azt mutatják, hogy az előrejelzés átlagos hibája 342, a magyarázott variancia 53%, tehát a modell teljesítménye gyenge: a függő változó és a független változók közötti kapcsolat gyenge. Ha újra létrehozzuk a modellt a `CALCULATE_P_VALUES = true`, `CATEGORY_ENCODING_METHOD='DUMMY_ENCODING'` opciókkal, ellenőrizhetjük, hogy az egyes változók hozzájárulása szignifikáns-e.[36] Azokat a rekordokat, ahol a hiba kisebb volt 40%-nál, a következő paranccsal kérdezhetjük le. Hasonlóképpen megállapítható, hogy a legrosszabb előrejelzés 17,0 helyett 517,9 volt.

```
SELECT * FROM ML.PREDICT(MODEL `northwind.nw_model`, (
 SELECT value_numeric, country, categoryid, p_unitprice, discount, pyear
 FROM `northwind.nw`
)) where abs((predicted_value_numeric-value_numeric))/value_numeric<0.4
```

**GYAKORLAT:** módosítsd a nézetet úgy, hogy tartalmazza a `value_nominal` mezőt, amely három kategóriát tartalmaz az alacsony, közepes és magas árú tételekhez. Próbáld meg a `value_nominal` mezőt a `BOOSTED_TREE_CLASSIFIER` vagy a `LOGISTIC_REG` módszerrel becsülni. Hány esetben/milyen százalékban tudod helyesen megbecsülni az osztálycímkét?

Segítség:

```
select od.unitprice*od.quantity*(1-od.discount) value,
case when (od.unitprice*od.quantity*(1-od.discount)) < 200 then 'L'
      when (od.unitprice*od.quantity*(1-od.discount)) < 1200 then 'M'
      when (od.unitprice*od.quantity*(1-od.discount)) >= 1200 then 'H'
else 'N/A' end value_nominal,
c.CustomerID, c.CompanyName, c.Country, c.Balance, o.OrderID,  o.OrderDate, o.RequiredDate,
year(o.orderdate) pyear,
o.ShippedDate, o.ShipVia, o.Freight, o.ShipName, o.ShipAddress, o.ShipCity, o.ShipRegion,
o.ShipPostalCode, o.ShipCountry, od.UnitPrice, od.Quantity, od.Discount,
p.ProductName, p.SupplierID, p.CategoryID,
p.QuantityPerUnit, p.UnitPrice p_unitprice, p.UnitsInStock, p.UnitsOnOrder, p.ReorderLevel,
p.Discontinued
from Customers AS c join Orders AS o ON c.CustomerID = o.CustomerID join
[Order Details] AS od ON o.OrderID = od.OrderID join
Products AS p ON od.ProductID = p.ProductID
```

**GYAKORLAT:** használd a nyilvános `bigquery-public-data.samples.natality`[37] teszt táblát egy csecsemő születési súlyának becslésére lineáris regresszióval. Ez a tábla sok rekordot tartalmaz, ezért mintát vehetsz belőle egy `WHERE RAND() < 0.0001` feltétellel.

Segítség:

Ha az adatkészleted nem az US régióban lett létrehozva, hozz létre egy új adatkészletet 'US' Multi-régióban `us_dataset` névvel, majd futtasd az alábbi lekérdezést — így nem kell manuálisan beállítanod a lekérdezés régióját (a MORE → Query settings → Advanced options menüben):

```
CREATE MODEL `us_dataset.natality_model`
OPTIONS
  (model_type='linear_reg',
    input_label_cols=['weight_pounds']) AS
SELECT
  weight_pounds,
  is_male,
  gestation_weeks,
  mother_age,
```

[36] Úgy tűnik, ez a funkció jelenleg nem működik.

[37] Lásd: https://cloud.google.com/bigquery-ml/docs/bigqueryml-natality

```
  CAST(mother_race AS string) AS mother_race
FROM
  `bigquery-public-data.samples.natality`
WHERE
  weight_pounds IS NOT NULL AND RAND() < 0.0001

SELECT  * FROM  ML.TRAINING_INFO(MODEL `dataset_nev.natality_model`)

SELECT * FROM ML.PREDICT(MODEL `dataset_nev.natality_model`,
(select true as is_male, 40 as gestation_weeks, 30 as mother_age,'38' as mother_race))
```

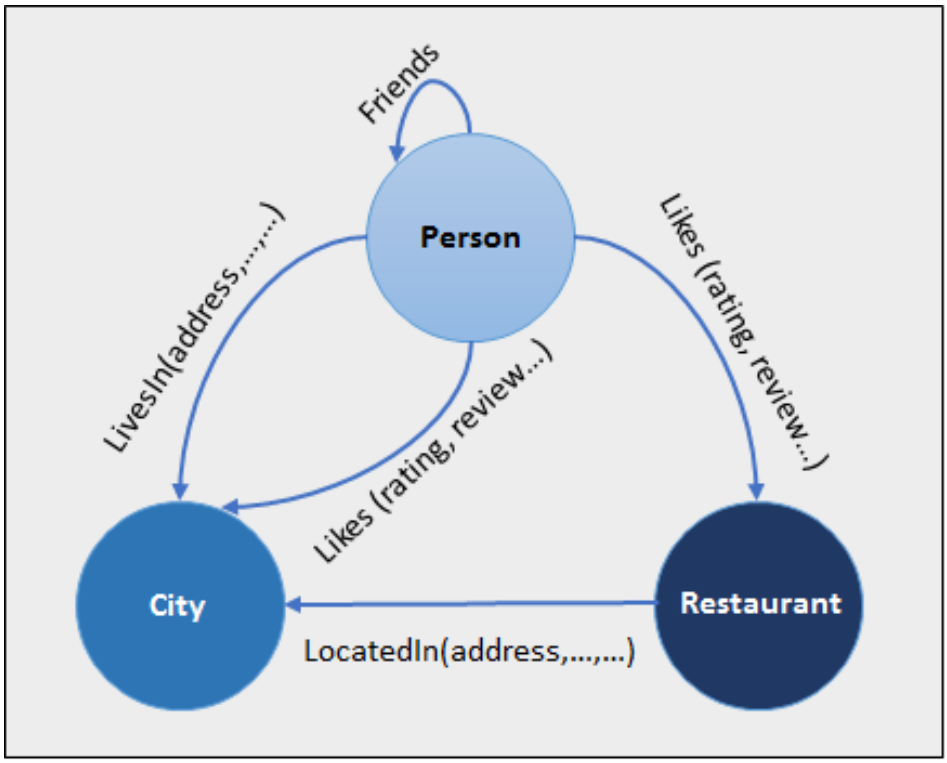
---

### További olvasnivaló

- Faculty training: https://edu.google.com/programs/faculty/training
- Learning paths: https://cloud.google.com/training
- Quickstarts (rövid oktatóanyagok a Cloud Platform termékekhez, szolgáltatásokhoz és API-khoz): https://cloud.google.com/gcp/getting-started
- Sample Projects: https://go.google-mkto.com/CPI00b01C3FT0A2j1tciC0V
- GCP docs: https://cloud.google.com/docs

---

## 8. Gráfmodellek és BLOB-tárolás




Az UML- és ERM-modellek az entitásokat és azok kapcsolatait írják le. A szokásos megközelítés szerint ezekből relációs modell készül. A **gráfmodell** (graph model) a csomópontokat — a gráfelméletben: **csúcsokat** (nodes) — és az éleket (edges) helyezi a középpontba; alternatívát kínál a relációs modellhez, és bizonyos lekérdezéseknél (pl. hálózati bejárás, legrövidebb út) hatékonyabb lehet. (A kurzus végig a Microsoft SQL Server-dokumentációban használt „cso-

mópont” kifejezést alkalmazza, de az „él” és a „kapcsolat” nem szinonimák: az él strukturális kapcsolat a gráfban, a kapcsolat pedig a kapcsolat szemantikai jelentése.)

---

## SQL Server gráftáblák

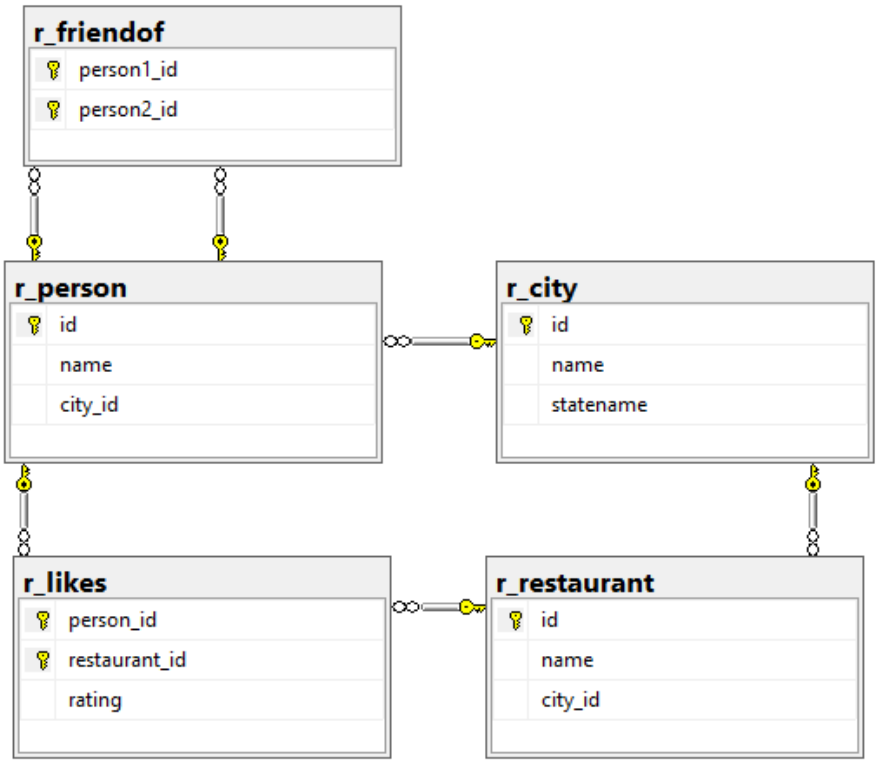
Az SQL Server 2017-től natív gráftábla-támogatással rendelkezik. A legfontosabb fogalmak:

- **Csomóponttábla** (Node table): `AS NODE` záradékkal hozható létre; a tábla sorai a gráf csomópontjait képviselik
- **Éltábla** (Edge table): `AS EDGE` záradékkal hozható létre; a tábla sorai az irányított éleket képviselik
- **`$node_id`**: a csomópontok rendszer által generált belső azonosítója (pszeudo-oszlop)
- **`$edge_id`**: az élek rendszer által generált belső azonosítója (pszeudo-oszlop)
- **`MATCH`**: kulcsszó gráf-mintaillesztéses lekérdezésekhez; a `-(él)->` szintaxis irányított élt jelöl

**DEMO:** hozz létre egy közösségi hálózatot modellező gráf-adatbázist.

```
use master
go
create database graphdemo
go
use graphdemo
go

-- Csomóponttáblák
create table person (id integer, name varchar(100)) as node;
create table restaurant (id integer, name varchar(100), city varchar(100)) as node;
create table city (id integer, name varchar(100)) as node;

-- Éltáblák
create table likes as edge;
create table friendof as edge;
create table livesin as edge;
create table locatedin as edge;

-- Személyek
insert into person values (1, 'john');
insert into person values (2, 'mary');
insert into person values (3, 'alice');
insert into person values (4, 'peter');

-- Városok
insert into city values (1, 'New York');
insert into city values (2, 'London');

-- Éttermek
insert into restaurant values (1, 'Ristorante Rosso', 'New York');
insert into restaurant values (2, 'Trattoria Verde', 'London');
insert into restaurant values (3, 'Café Bleu', 'New York');

-- Barátság-élek ($node_id alapján)
insert into friendof values
 ((select $node_id from person where name = 'john'),
 (select $node_id from person where name = 'mary'));
insert into friendof values
 ((select $node_id from person where name = 'john'),
 (select $node_id from person where name = 'alice'));
insert into friendof values
 ((select $node_id from person where name = 'mary'),
 (select $node_id from person where name = 'peter'));

-- Tetszik-élek
insert into likes values
```

```
 ((select $node_id from person where name = 'john'),
 (select $node_id from restaurant where name = 'Ristorante Rosso'));
insert into likes values
 ((select $node_id from person where name = 'mary'),
 (select $node_id from restaurant where name = 'Ristorante Rosso'));
insert into likes values
 ((select $node_id from person where name = 'alice'),
 (select $node_id from restaurant where name = 'Trattoria Verde'));

-- Lakóhely-élek
insert into livesin values
 ((select $node_id from person where name = 'john'),
 (select $node_id from city where name = 'New York'));
insert into livesin values
 ((select $node_id from person where name = 'mary'),
 (select $node_id from city where name = 'London'));
insert into livesin values
 ((select $node_id from person where name = 'alice'),
 (select $node_id from city where name = 'London'));
insert into livesin values
 ((select $node_id from person where name = 'peter'),
 (select $node_id from city where name = 'New York'));

-- Étterem-elhelyezkedési élek
insert into locatedin values
 ((select $node_id from restaurant where name = 'Ristorante Rosso'),
 (select $node_id from city where name = 'New York'));
insert into locatedin values
 ((select $node_id from restaurant where name = 'Trattoria Verde'),
 (select $node_id from city where name = 'London'));
insert into locatedin values
 ((select $node_id from restaurant where name = 'Café Bleu'),
 (select $node_id from city where name = 'New York'));
```

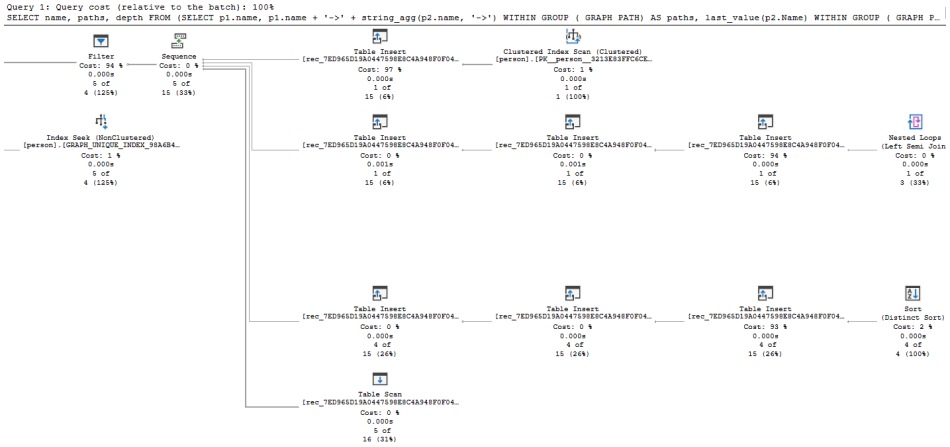



Gráf-mintaillesztéses lekérdezések a `MATCH` záradékkal:

```
-- John barátai
select p2.name
from person p1, friendof f, person p2
where match(p1-(f)->p2)
and p1.name = 'john';

-- John által kedvelt éttermek
select r.name
from person p, likes l, restaurant r
where match(p-(l)->r)
and p.name = 'john';

-- John barátai által kedvelt éttermek
select p2.name as friend, r.name as restaurant
from person p1, friendof f, person p2, likes l, restaurant r
where match(p1-(f)->p2-(l)->r)
and p1.name = 'john';

-- Azonos városban lakó személyek
select p1.name, p2.name
from person p1, livesin l1, city c, livesin l2, person p2
where match(p1-(l1)->c<-(l2)-p2)
and p1.name <> p2.name;
```

### SHORTEST_PATH (SQL Server 2019+)

```
select p1.name,
   string_agg(p2.name, '->') within group (graph path) as path,
   count(p2.name) within group (graph path) as path_length
from person p1,
  friendof for path fo,
  person for path p2
where match(shortest_path(p1(-(fo)->p2)+))
and p1.name = 'john';
```




### Hagyományos relációs megfelelő

Ugyanez a modell öt relációs táblával is megvalósítható:

```
create table r_person (id integer primary key, name varchar(100));
create table r_city (id integer primary key, name varchar(100));
create table r_restaurant (id integer primary key, name varchar(100),
 city_id integer references r_city(id));
create table r_likes (person_id integer references r_person(id),
 restaurant_id integer references r_restaurant(id));
create table r_friendof (person_id integer references r_person(id),
 friend_id integer references r_person(id));
```

Relációs sémával a bejárások több JOIN-t igényelnek — pl. „John barátainak barátai által kedvelt éttermek” négy JOIN-t kíván. A gráftáblás megközelítés olvashatóbb és természetesebb az ilyen lekérdezéstípusoknál.

**GYAKORLAT:**

1. Valósítsd meg az Orders / Order Details / Products táblakapcsolatokat gráfként az AdventureworksDW2019 adatbázisban, és futtass MATCH-lekérdezéseket.
2. Futtasd az alábbi T-SQL szkriptet, amely véletlenszerű gráfot generál 100 csomóponttal és 300 éllel, majd mérd meg az átmérőt (a leghosszabb legrövidebb út hosszát) SHORTEST_PATH segítségével.

```
-- Véletlenszerű gráf generálása
drop table if exists rnd_edge;
drop table if exists rnd_node;
create table rnd_node (id integer) as node;
create table rnd_edge as edge;

declare @i integer = 0;
while @i < 100
begin
 insert into rnd_node values (@i);
 set @i = @i + 1;
end;

declare @j integer = 0;
while @j < 300
begin
 insert into rnd_edge values (
  (select $node_id from rnd_node where id = abs(checksum(newid())) % 100),
  (select $node_id from rnd_node where id = abs(checksum(newid())) % 100)
 );
 set @j = @j + 1;
end;

-- Átmérő mérése
select max(path_len) as diameter
from (
 select n1.id,
    max(count(n2.id)) within group (graph path) as path_len
 from rnd_node n1,
   rnd_edge for path re,
   rnd_node for path n2
 where match(shortest_path(n1(-(re)->n2)+))
 group by n1.id
) t;
```

---

## GraphQL interfészek

A **GraphQL** egy lekérdező nyelv és futtatókörnyezet webes API-khoz (Facebook, 2015). Fő jellemzők:

- A kliens pontosan meghatározza, milyen adatokat kér — nincs over-fetching (felesleges adatok lekérése)
- Erősen típusos séma írja le az elérhető adatokat és műveletek típusait
- Egyetlen HTTP-végponton keresztül érhető el

Egy GraphQL interfész adatbázis-háttérrendszerrel a következőképpen működik:

1. A kliens GraphQL-lekérdezést küld a szervernek
2. A szerver feldolgozza (parse) a lekérdezést a séma alapján
3. A szerver végrehajtja a lekérdezést az adatbázison (SQL vagy gráf)
4. Az eredményt JSON formátumban adja vissza a kliensnek

---

## Neo4j natív gráfadatbázis

A **Neo4j** natív gráfadatbázis: a csomópontokat és éleket közvetlenül a lemezen tárolja, nem relációs táblákban. Ez hatékonyabb gráfbejárást tesz lehetővé nagy hálózatokon, szemben az SQL Server gráftábláival, amelyek relációs tárolón alapulnak.

### Technikai beállítások

- A Neo4j sok memóriát igényel; a `neo4j.conf` fájlban állítsd be: `dbms.memory.heap.max_size=2G`
- Jogosultságproblémák esetén futtasd a Neo4j Desktop-ot rendszergazdaként

### Adattárolás

A Neo4j adatait az úgynevezett Neostore fájlok tartalmazzák:

| Fájl | Tartalom |
|---|---|
| `neostore.nodestore.*` | Csomópontok |
| `neostore.labelscanstore.*` | Csomópontcímkék (labels) |
| `neostore.relationshipstore.*` | Kapcsolatok (gráf-élek) |
| `neostore.propertystore.*` | Tulajdonságok (properties) |

### Modell jellemzői

- **Nem kötelező séma**: a csomópontoknak és éleknek nem szükséges előre definiált sémának megfelelni (schema-on-read)
- **Irányított élek**: minden él irányított, de lekérdezéskor mindkét irányban bejárható
- **Egyetlen típus élenként**: egy él csak egyféle típusú lehet (pl. `LIKES`, `DIRECTED`)
- **Tulajdonságok**: csomópontokhoz és élekhez egyaránt rendelhetők kulcs-érték párok

### A Cypher lekérdező nyelv

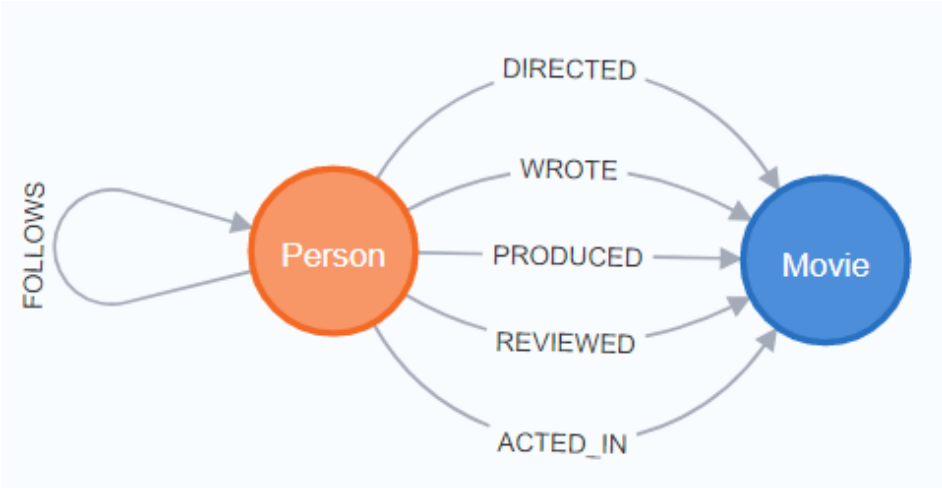
A Neo4j saját lekérdező nyelve a **Cypher**. Főbb parancsok:

| Parancs | Leírás |
|---|---|
| `MATCH` | Mintaillesztés a gráfban |
| `RETURN` | Visszaadandó értékek |
| `CREATE` | Csomópontok és élek létrehozása |
| `MERGE` | Létrehozás, ha még nem létezik |
| `SET` | Tulajdonság beállítása |
| `REMOVE` | Tulajdonság vagy cimke eltávolítása |
| `DELETE` | Csomópontok és élek törlése |
| `WHERE` | Szűrési feltétel |

### Movies adatbázis — Cypher-példák

A Neo4j Desktop alapértelmezett mintaadatbázisa a **Movies**. Csomópontok: `Person`, `Movie`. Élek: `DIRECTED`, `WROTE`, `PRODUCED`, `REVIEWED`, `ACTED_IN`, `FOLLOWS`.

```
// Séma vizualizáció
CALL db.schema.visualization()

// Minden film
MATCH (m:Movie) RETURN m.title, m.released ORDER BY m.released DESC

// Tom Hanks filmjei
MATCH (p:Person {name: 'Tom Hanks'})-[:ACTED_IN]->(m:Movie)
RETURN m.title, m.released

// Ki rendezte a The Matrix-ot?
MATCH (p:Person)-[:DIRECTED]->(m:Movie {title: 'The Matrix'})
RETURN p.name

// Filmek rendezőkkel és szereplőkkel
MATCH (director:Person)-[:DIRECTED]->(m:Movie)<-[:ACTED_IN]-(actor:Person)
RETURN m.title, director.name, collect(actor.name) AS actors

// Csoportosítás és aggregáció: szereplők filmszáma
MATCH (p:Person)-[:ACTED_IN]->(m:Movie)
RETURN p.name, count(*) AS filmCount, avg(m.released) AS avgYear
ORDER BY filmCount DESC LIMIT 10

// Filmek statisztikái
MATCH (m:Movie)
RETURN max(m.released), avg(m.released), min(m.released)

// Legrövidebb út két személy között a FOLLOWS élek mentén
MATCH p = shortestPath((p1:Person {name: 'Tom Hanks'})-[:FOLLOWS*]-(p2:Person {name: 'Meg Ryan'}))
RETURN p

// Gráf átmérője (leghosszabb legrövidebb út)
MATCH (p1:Person), (p2:Person)
WHERE p1 <> p2
WITH p1, p2, shortestPath((p1)-[:FOLLOWS*]-(p2)) AS sp
WHERE sp IS NOT NULL
RETURN max(length(sp)) AS diameter

// Séma lekérdezése (Movie csomópontok kulcsai)
MATCH (p:Movie) RETURN distinct keys(p), size(keys(p))

// Minden él törlése
MATCH ()-[r]-() DELETE r

// Minden csomópont törlése
MATCH (n) DELETE n
```

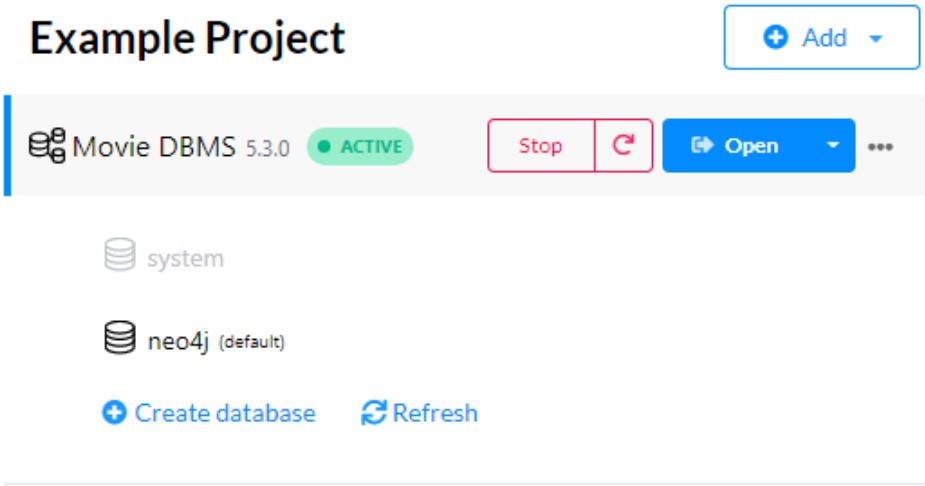



**GYAKORLAT:**

1. Nyisd meg a Neo4j Desktop alkalmazást, és hozz létre egy új projektet.
2. Importáld a Movies mintaadatbázist (Add → Example Project).
3. Futtasd a `CALL db.schema.visualization()` parancsot, és tekintsd meg a sémát.
4. Próbáld ki a fenti Cypher-lekérdezéseket.

---

## Adatok importálása Neo4j-be




Az SQL Server adatai **CSV-fájlokon keresztül** importálhatók a Neo4j-be. Első lépésként hozzuk létre a szükséges nézeteket a Northwind adatbázisban:

```
create view vi_products as
select ProductID, ProductName, CategoryID, UnitPrice, UnitsInStock
from Products;

create view vi_orders as
select OrderID, CustomerID,
   convert(varchar(10), OrderDate, 120) as OrderDate, ShipCountry
from Orders;

create view vi_orders_products as
select od.OrderID, od.ProductID, od.Quantity, od.UnitPrice
from [Order Details] od;
```

Exportáld a nézeteket CSV-fájlokba, helyezd el a Neo4j `import/` könyvtárában, majd Cypher `LOAD CSV` parancsokkal importáld:

```
// Termékek
LOAD CSV WITH HEADERS FROM 'file:///vi_products.csv' AS row
CREATE (:Product {
```

```
 productId: toInteger(row.ProductID),
 name: row.ProductName,
 price: toFloat(row.UnitPrice),
 unitsInStock: toInteger(row.UnitsInStock)
});

// Megrendelések
LOAD CSV WITH HEADERS FROM 'file:///vi_orders.csv' AS row
CREATE (:Order {
 orderId: toInteger(row.OrderID),
 customerId: row.CustomerID,
 orderDate: row.OrderDate,
 shipCountry: row.ShipCountry
});

// Kapcsolatok (rendelés–termék)
LOAD CSV WITH HEADERS FROM 'file:///vi_orders_products.csv' AS row
MATCH (o:Order {orderId: toInteger(row.OrderID)})
MATCH (p:Product {productId: toInteger(row.ProductID)})
MERGE (o)-[:CONTAINS {quantity: toInteger(row.Quantity), unitPrice: toFloat(row.UnitPrice)}]->(p);
```

**GYAKORLAT:**

1. Kérdezd le Cypher segítségével az éves bevételt terméknévenkénti bontásban, és hasonlítsd össze a relációs SQL-megoldással.
2. Generálj véletlenszerű gráfot T-SQL-lel az SQL Serverben, exportáld CSV-be, importáld Neo4j-be, és hasonlítsd meg a gráfátmérő mérésének futási idejét a két rendszerben.

---

## Képek és BLOB-ok tárolása

Az SQL Server két megközelítést kínál bináris nagy objektumok (**BLOB** – Binary Large Object) tárolásához.

### VARBINARY(MAX) — közepes méretű fájlokhoz

Az adatbázison belüli tárolás tranzakcióbiztos és egyszerűen lekérdezhető. Néhány MB méretig ajánlott.

```
create table my_photos (
 id integer identity primary key,
 image_name varchar(200),
 image_column varbinary(max)
);

-- Kép betöltése fájlból
insert into my_photos(image_name, image_column)
select 'my_photo.png', * from openrowset(bulk 'c:\my_path\my_photo.png', single_blob) as img;

-- Fájlméretek lekérdezése
select id, image_name, datalength(image_column) as size_bytes from my_photos;
```

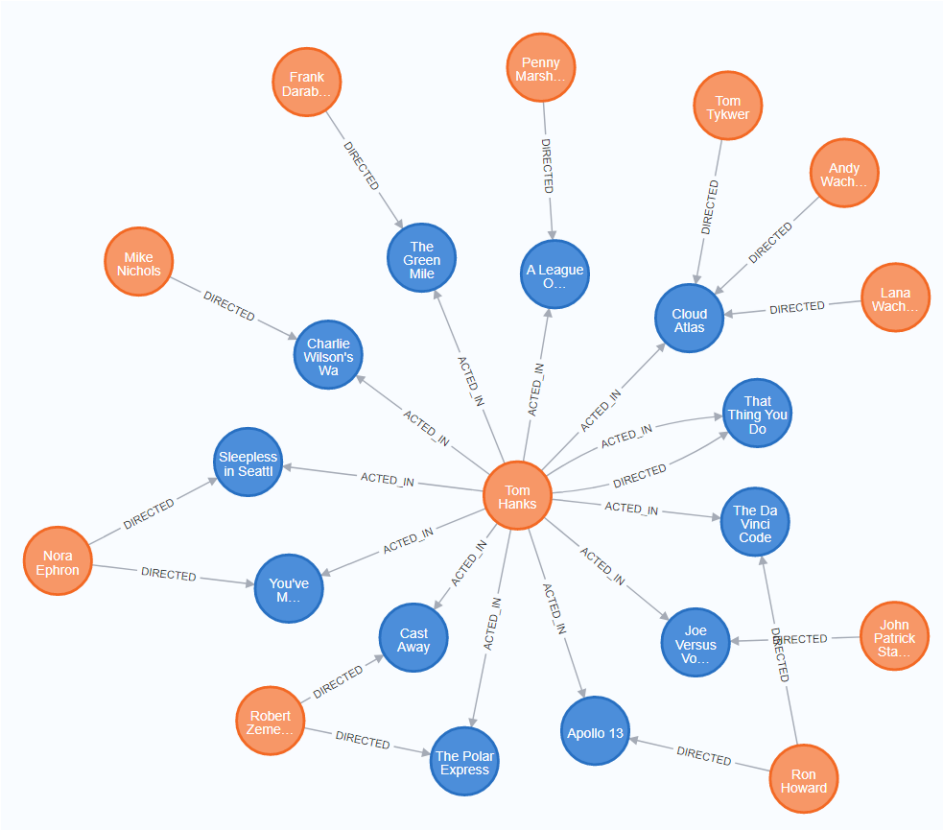
### FileTables — nagy méretű fájlokhoz

A **FileTable** az SQL Server **FILESTREAM** technológiájára épül. A fájlok a fájlrendszeren tárolódnak, de SQL-táblákén is elérhetők:

- Nagy méretű fájlokhoz (videó, hangfájl, dokumentum) ajánlott
- Tranzakcióbiztos fájlkezelés
- Windows fájlrendszer-API-val (UNC elérési út) és SQL-en keresztül egyaránt kezelhető
- Előfeltétel: a SQL Server-példányon és az adatbázison engedélyezni kell a FILESTREAM-et

**GYAKORLAT:** tervezz FileTable-alapú megoldást a kurzus képernyőfelvétel-videóinak tárolásához és lekérdezéséhez (cím, dátum és időtartam szerint kereshetően).

---




# 9. Memóriaoptimalizált táblák az SQL Serverben

A memóriaoptimalizált táblák (Memory-Optimized Tables, MOT) natívan lefordított tárolt eljárásokkal kombinálva az OLTP relációs adatbázis-technológia „Formula–1-je”,[38] amelyet a memória árának csökkenése tett lehetővé.

---

## Jellemzők

- **Párhuzamos adatfeldolgozás**, finomított memóriabeli adatstruktúrák, hash indexek és önálló, dedikált memóriában működő adatbázismotor → nagy teljesítmény.
  - Minden MOT-táblaszerkezethez és natívan lefordított eljáráshoz egy új DLL jön létre.
  - A zárolások elkerülése érdekében sorverziózást alkalmaz: egy sor frissítésekor a sor új verziója jön létre (= snapshot izoláció / pillanatkép-alapú izoláció). A törlések szintén logikai jellegűek: a törölt sorok mindaddig megőrződnek, amíg az adott verziót esetlegesen használó összes tranzakció véglegesítésre nem kerül.
- **Teljes ACID-támogatás.** A tartósságot a nem illékony (lemezalapú) tranzakciós napló biztosítja.

[38] https://learn.microsoft.com/en-us/sql/relational-databases/in-memory-oltp/overview-and-usage-scenarios?view=sql-server-ver16

- A MOT-ok **nem tartóssá (non-durable)** is tehetők, ha tranziens adatok tárolására használjuk őket — a tempdb alternatívájaként. Az ilyen táblákban végzett módosítások nem igényelnek lemez I/O-t. További előny, hogy a táblák szerkezete — ellentétben az ideiglenes táblákéval — megmarad. A nem tartós MOT-ok gyorsítótárként (cache-ként) használhatók.
- A MOT-ok hagyományos táblákkal **azonos adatbázisban** is elhelyezhetők.
- Ha egy tárolt eljárás nincs natívan lefordítva, de MOT-ot használ, **értelmezett módban** kell futnia (a MOT-ot egy másik motor kezeli).[39]

---

## Követelmények és korlátozások[40]

- Csak **hash** vagy **nem fürtözött (nonclustered) BTree index** támogatott.
- Elegendő memóriára van szükség:
  - a MOT és indexei számára,
  - a sorverziózáshoz,
  - táblaváltozókhoz és növekményes növekedéshez.

  Ökölszabály: „a memóriaoptimalizált táblák és indexek várható méretének kétszeresével kezdj”.[41]
- Egy MOT nem érhető el más adatbázisból (nincs adatbázisok közötti lekérdezés vagy tranzakció), és **nem particionálható**.
- Az oldal- és sortömörítés, valamint a DDL-triggerek **nem támogatottak**.
- MOT-előfizetővel csak **tranzakciós replikáció** támogatott.
- DDL (CREATE / ALTER / DROP) tranzakciókon belül **nem támogatott**.
- A natívan lefordított tárolt eljárások csak **memóriaoptimalizált táblákra** hivatkozhatnak.
- **Identity** elsődleges kulcs szükséges.

---

## Ajánlott alkalmazási területek

- **Nagy teljesítményű tranzakció-feldolgozás**, pl. nagyszámú párhuzamos, alacsony késleltetésű tőzsdei tranzakció kezelése. A megvalósítás memóriaoptimalizált tranzakciós táblákat és natívan lefordított tárolt eljárásokat alkalmaz. Különösen a CPU-idő által dominált eljárások profitálnak a natív fordításból.
- **Valós idejű adatgyűjtés és -transzformáció** sok forrásból (pl. IoT-érzékelők). Ha a tábla méretét korlátozni kell, egy ütemezett feladat segítségével az adatokat a memóriabeli táblából lemezalapú oszloptárba (column store) lehet áthelyezni. Ha sok frissítés van, az alaptábla egy temporális memóriaoptimalizált tábla lehet — a korábbi rekordok lemezen maradnak.

[39] https://learn.microsoft.com/en-us/sql/relational-databases/in-memory-oltp/a-guide-to-query-processing-for-memory-optimized-tables

[40] https://learn.microsoft.com/en-us/sql/relational-databases/in-memory-oltp/unsupported-sql-server-features-for-in-memory-oltp?view=sql-server-ver16

[41] https://learn.microsoft.com/en-us/sql/relational-databases/in-memory-oltp/estimate-memory-requirements-for-memory-optimized-tables?view=sql-server-ver16

- **ETL-folyamat gyorsítása**: az adatraktár Extract-Transform-Load folyamatának staging tábláiként nem tartós, memóriaoptimalizált táblák használhatók. Ezek nem igényelnek lemez I/O-t, így felgyorsítják az ETL-folyamatot.
- **Gyorsítótárazás (cache)**: egy nem tartós memóriabeli tábla kulcs-érték tárolóként (BLOB vagy JSON) is alkalmazható.
- A **táblatípusok** szintén memóriaoptimalizálttá tehetők. Az ilyen típusok tárolt eljárások táblaértékű paramétereiként használhatók.

---

## GYAKORLAT — Teljesítménymérés[42]

Egy új tesztadatbázisban mérjük a MOT teljesítménybeli előnyét a normál táblával szemben. Három forgatókönyvet hasonlítunk össze a táblabeszúrások tekintetében:

1. Lemezalapú tábla és értelmezett Transact-SQL
2. Memóriaoptimalizált tábla hash indexszel és értelmezett Transact-SQL
3. Memóriaoptimalizált tábla hash indexszel és natívan lefordított tárolt eljárás

---

## Migráció tervezése In-Memory megoldásra

A migráció jelentős erőfeszítést igényelhet. Az SQL Server beépített eszközöket kínál az OLTP → In-Memory átálláshoz.

**GYAKORLAT:**

- Töröld a Northwind adatbázist, és állítsd vissza a mentésből (dump).
- Nyisd meg a **Database → Reports → Standard reports → Transaction Performance Analysis Overview** riportot. Az eszköz ajánlásokat ad a jelölt táblákhoz és eljárásokhoz. Kattints a tábla/eljárás nevére a részletes statisztikákhoz. A már memóriaoptimalizált táblák/eljárások nem szerepelnek a listában.
- Az összesítő tábla létrehozásával ellenőrizheted a migrációs blokkolók (migration blockers) számát. A legrosszabb eset a Customers tábla 36 blokkolóval — ez a mentésfájl hibájára vezethető vissza (ugyanaz az idegen kulcs több példányban szerepel). Javítsd a mentésfájlt.
- **Database → Tasks → Generate In-Memory OLTP Migration Checklists** (ez a funkció jelenleg nem működik).
- Táblanév → **Table Memory Optimization Advisor**: első lépésként ellenőrzőlistát (checklist) generál. Az összes idegen kulcs megszorítást manuálisan kell eltávolítani a tábláról (és a táblára mutató megszorításokat is) az áttelepítés előtt, majd újra létre kell hozni.
- Az Advisor megtervezi és végrehajtja az áttelepítést. A régi tábla megmarad, és átnevezésre kerül.

---

## GYAKORLAT — Reális forgatókönyv párhuzamos munkamenetekkel

Reálisabb forgatókönyvben a Northwind adatbázis rendelésfeldolgozó tárolt eljárásának teljesítményét teszteljük párhuzamos munkamenetekben.

[42] SQL Server 2022 nevű példányon a fájl memóriaoptimalizált filegroup-hoz való hozzáadása sikertelen a következő hibaüzenettel: *„Could not process the operation. Always On Availability Groups replica manager is disabled on this instance... etc.”* Ez ismert probléma. Helyette SQL Server 2019 alapértelmezett példányát (nem nevezett példányt) használjuk.

1. Töröld az adatbázist, hozd létre újra a mentésből, és hozd létre az `sp_new_order()` eljárást.
2. Állítsd be a tesztelési paramétereket, majd futtasd az eljárást 10 000-szer 3 párhuzamos munkamenetben Read Committed izolációval, és 100-szor Repeatable Read izolációval. Mérd meg a futási időt.
3. Nézd át a **Transaction Performance Analysis Overview** riportot. A MOT-ra ajánlott táblák: Products, Orders, Order Details, Territories és Customers.
4. Telepítsd át a Products, Orders, Order Details és Customers táblákat:
   a. Hozz létre egy új adatbázist `Northwind_mot` névvel, és futtasd benne a javított dump szkriptet.
   b. Töröld az érintett táblákra mutató és az azoktól induló idegen kulcs megszorításokat.
   c. Telepítsd át a táblákat az Advisor segítségével. Ne felejdd belevenni az adatokat az új táblába.
5. Futtasd az alábbi parancsot:

```
alter database current set memory_optimized_elevate_to_snapshot = on
```

6. Futtasd a régi rendelésfeldolgozó eljárást, teszteld, és hasonlítsd össze a futási időket.
7. Készítsd el a tárolt eljárás natívan lefordított változatát, és ismételd meg a teszteket.
8. Értelmezd az eredményeket!

---

# 10. Köszönetnyilvánítás

Ezen kurzusjegyzet egyes példái Dejan Sarka, Matija Lah és Grega Jerkič könyvén alapulnak: *Exam 70-463: Implementing a Data Warehouse with Microsoft SQL Server 2012*, Microsoft Press, ISBN 978-0-7356-6609-2, 2015. A többi hivatkozott példa forrása a lábjegyzetekben található.

# 11. A függelék: SQL DML-példák önálló tanuláshoz

Az alábbi példák a NORTHWIND adatbázison alapulnak. A lekérdezések egyre összetettebbek; célszerű sorban haladni rajtuk.

---

## Alapvető SELECT lekérdezések

```
use NORTHWIND
select * from employees
select lastname, birthdate from employees

--a Londonban található ügyfelek neve
select companyname, city
from customers
--where city LIKE 'L%' and (city LIKE '%b%' or city LIKE '%n%') --részleges egyezés
where city IN ('London', 'Lander')
where city ='London' or city ='Lander'
where city IN ('London')
where city = 'London'
```

---

## Aggregáló függvények és beágyazott lekérdezések

```
--Ki a legfiatalabb alkalmazott? Mi a neve?
--1/2) a legnagyobb születési dátum
```

```sql
--aggregáló függvények: max, min, avg, std, sum, count
select max(birthdate) as max_year, min(birthdate) as min_year
--, lastname --hibát okozna
from employees

--2/2) ágyazd be ezt a lekérdezést
select lastname, birthdate from employees
where birthdate = ('1966-01-27 00:00:00.000')

select lastname, birthdate as "birth date" from employees
where birthdate = (
  select max(birthdate) as max_year
  from employees
)

--FELADAT: keresd meg az első rendelés ShipAddress mezőjét
select orderdate, shipaddress from orders
where orderdate = (
  select min(orderdate) as min_date
  from orders
)

--a legfiatalabb alkalmazott szállítási címei
--táblák összekapcsolása
select distinct lastname, shipaddress
from orders o inner join employees e on o.employeeid=e.employeeid
where e.employeeid = (
  select employeeid from employees
  where birthdate = (
    select max(birthdate) as max_year
    from employees
  )
)
order by shipaddress --desc
```

---

## JOIN-ok

```sql
--milyen termékeket rendeltek a legfiatalabb alkalmazottól
--megjegyzés: mindig a lekérdezés FROM részével kezdjünk
select distinct p.productname, e.lastname
from orders o inner join employees e on o.employeeid=e.employeeid
inner join [order details] od on od.orderid=o.orderid
inner join products p on p.productid=od.productid
where e.employeeid=9 --ő a legfiatalabb
order by productname

--FELADAT: mely városokba szállítják az 1-es kategóriájú termékeket?
select distinct o.shipcity
from orders o inner join [order details] od on od.orderid=o.orderid
inner join products p on p.productid=od.productid
where p.categoryid = 1 --a keresési feltételünk
order by shipcity
```

---

## Csoportosítás (GROUP BY)

Az alábbi sorozat bemutatja, hogyan érdemes felépíteni egy összetett GROUP BY lekérdezést. A számok a helyes írási sorrendet jelzik (FROM → GROUP BY → SELECT → ORDER BY).

```sql
--Hány rendelése van alkalmazottanként?
--1/5) CSOPORTOSÍTÁS
select employeeid, count(*)
from orders
group by employeeid
```

```sql
--mellékes megjegyzés: hogyan teszteljük a NULL értéket?
select * from orders where employeeid is null
delete from orders where employeeid is null

--2/5) NE CSINÁLD EZT:
select e.lastname, count(*)
from orders o inner join employees e on o.employeeid=e.employeeid
group by e.lastname
--logikai hibát okoz, ha két személynek azonos a vezetékneve!!!

--3/5)
select e.lastname, e.firstname, count(*)
from orders o inner join employees e on o.employeeid=e.employeeid
group by e.employeeid, e.lastname, e.firstname
--ez a lekérdezés kihagyja azt az alkalmazottat, akinek nincs rendelése

--FELADAT: listázd ki az egyes kategóriákban lévő termékek számát (a CategoryName-re is szükségünk
↪  van)
3. select c.categoryid, c.categoryname, count(*) as no_prod
1. from products p inner join categories c on p.categoryid=c.categoryid
2. group by c.categoryid, c.categoryname
4. order by no_prod desc

--4/5)
select e.employeeid, e.lastname, e.firstname, count(*)
from employees e left outer join orders o on o.employeeid=e.employeeid
group by e.employeeid, e.lastname, e.firstname
--probléma: a tétlen alkalmazott hamisan 1-es darabszámot kap

--5/5)
select e.employeeid, e.lastname, e.firstname, count(o.orderid) as no_ord
from employees e left outer join orders o on o.employeeid=e.employeeid
group by e.employeeid, e.lastname, e.firstname
order by no_ord desc
--minden probléma megoldva
```

---

## Önillesztés (hierarchia lekérdezése)

```sql
--ki kinek a főnöke
select e.lastname, boss.lastname as boss, bboss.lastname as boss_of_boss
from employees e left outer join employees boss on e.reportsto=boss.employeeid
left outer join employees bboss on boss.reportsto=bboss.employeeid

--Hány rendelése van alkalmazottanként?
select e.lastname, count(orderid)
--a count(*) hamisan egy rendelést adna Lamernek, akinek valójában nincs rendelése
from employees e left outer join orders o on
--from employees e inner join orders o on
e.employeeid = o.employeeid
group by e.employeeid, e.lastname
order by count(*) desc

--kinek nincs rendelése?
select e.*
from employees e left outer join orders o on
e.employeeid = o.employeeid
where o.orderid is null
```

---

## Aritmetika, STR() formázás

```sql
--melyik a legnagyobb rendelés?
--aritmetika
```

```
4. select o.orderid,
 cast(o.orderdate as varchar(50)) as order_date,
 str(sum((1-discount)*unitprice*quantity), 15, 2) as order_total,
 sum(quantity) as no_of_units,
 count(d.orderid) as no_of_items
1. from orders o inner join [order details] d on o.orderid=d.orderid
2. where...
3. group by o.orderid, o.orderdate
 --order by o.orderdate
5. order by sum((1-discount)*unitprice*quantity) desc
```

---

## HAVING, ISNULL, a legjobb értékesítő

```
--ki a legsikeresebb alkalmazott? hány rendeléssel?
-- figyeld meg: having
-- count distinct
-- számok formázása
select u.titleofcourtesy+' '+u.lastname+' '+ u.firstname +' ('+u.title +')' as name,
str(sum((1-discount)*unitprice*quantity), 15, 2) as cash_income,
count(distinct o.orderid) as no_of_orders, count(productid) as no_of_items
from orders o inner join [order details] d on o.orderid=d.orderid
inner join employees u on u.employeeid=o.employeeid
group by u.employeeid, u.titleofcourtesy, u.title, u.lastname, u.firstname
--having count(o.orderid)>200 --ha csak a több mint 200 rendeléssel rendelkező alkalmazottak
 ↪  érdekelnek
order by cash_income
--sum((1-discount)*unitprice*quantity) desc

--miért csak 9 van?
select count(*) from employees
--10-nek kellene lennie!
--szükségünk lenne azokra is, akiknek 0 rendelésük van
-- isnull függvény
select isnull(u.titleofcourtesy, '')+' '+isnull(u.lastname, '')+' '+ isnull(u.firstname,
'') +' ('+isnull(u.title, '') +')' as name,
isnull(str(sum((1-discount)*unitprice*quantity), 15, 2), 'N/A') as cash_income,
count(distinct o.orderid) as no_of_orders, COUNT(d.productid) as no_of_items
from employees u left outer join
(orders o inner join [order details] d on o.orderid=d.orderid)
on u.employeeid=o.employeeid
--where u.titleofcourtesy='Mr.' --ha csak a férfiak érdekelnek
group by u.employeeid, u.titleofcourtesy, u.title, u.lastname, u.firstname
order by sum((1-discount)*unitprice*quantity) desc
```

---

## TOP — a legnépszerűbb termék

```
--melyik a legnépszerűbb termék?
-- top 1
select top 1 p.productid, p.productname, count(*) as no_app,
sum(quantity) as total_pieces
from products p left outer join [order details] d on p.productid=d.productid
group by p.productid, p.productname
order by no_app desc

--melyik alkalmazott adta el a legtöbbet a legnépszerűbb termékből?
--első verzió
select top 1 u.titleofcourtesy+' '+u.lastname+' '+ u.firstname +' ('+u.title +')' as name,
sum(quantity) as no_pieces_sold
from orders o inner join [order details] d on o.orderid=d.orderid
inner join employees u on u.employeeid=o.employeeid
where d.productid = 59 --ezt már tudjuk
group by u.employeeid, u.titleofcourtesy, u.title, u.lastname, u.firstname
-- having....
order by sum(quantity) desc
```

```
/*********************************************************************
FELADAT
--melyik alkalmazott adta el a legtöbbet a legnépszerűbb termékből, és mi annak a
terméknek a neve?
--a pubs_access adatbázisban: melyik a leggyakrabban használt kiadó annál a szerzőnél, akinek
a legtöbb publikációja van?
*********************************************************************/
```

---

## Dátumfüggvények

```
--TÖBBSZINTŰ CSOPORTOSÍTÁS
--dátum- és időfüggvények
select 2
select getdate() --datetime adattípus
select DATEDIFF(s,'2013-10-10 12:13:50.370', '2013-10-10 14:16:50.370')
select DATEADD(s, 1000, '2013-10-10 14:16:50.370')
select YEAR(getdate()), MONTH(getdate())

--RENDELÉSEK HÓNAP ÉS ALKALMAZOTT SZERINT
select e.employeeid, lastname, year(orderdate) as year, month(orderdate) as month,
count(orderid) as no_of_orders
from employees e left outer join orders o on e.employeeid=o.employeeid
group by e.employeeid, lastname, year(orderdate), month(orderdate)
order by lastname, year, month

--ugyanez másképpen:
select e.employeeid, lastname,
cast(year(orderdate) as varchar(4)) +'_'+ cast(month(orderdate) as char(2)) as month,
count(orderid) as no_of_orders
from employees e left outer join orders o on e.employeeid=o.employeeid
group by e.employeeid, lastname, cast(year(orderdate) as varchar(4)) +'_'+
cast(month(orderdate) as char(2))
order by lastname, month
```

---

## CASE kifejezés

```
--select case szerkezet
select e.employeeid, lastname,
case
  when month(orderdate) < 10 then cast(year(orderdate) as varchar(4)) +'_0'+
    cast(month(orderdate) as char(2))
  when month(orderdate) >= 10 then cast(year(orderdate) as varchar(4)) +'_'+
    cast(month(orderdate) as char(2))
  else 'N.A'
end as month,
count(orderid) as no_of_orders
from employees e left outer join orders o on e.employeeid=o.employeeid
group by e.employeeid, lastname,
case
  when month(orderdate) < 10 then cast(year(orderdate) as varchar(4)) +'_0'+
    cast(month(orderdate) as char(2))
  when month(orderdate) >= 10 then cast(year(orderdate) as varchar(4)) +'_'+
    cast(month(orderdate) as char(2))
  else 'N.A'
end --egy függvény jobban szolgálná ezt a célt
order by lastname, month
```

---

### Ideiglenes táblák

```
--ideiglenes táblák használata
select GETDATE() as ido into #uj_tabla
select * from #uj_tabla
drop table #uj_tabla

select * into #uj_tabla from employees

--drop table #tt
select e.employeeid, lastname, year(orderdate) as ev, month(orderdate) as month,
count(orderid) as no_of_orders
into #tt
from employees e left outer join orders o on e.employeeid=o.employeeid
group by e.employeeid, lastname, year(orderdate), month(orderdate)
order by lastname, month

select * from #tt

--Figyelmeztetés: Null értéket kizár egy aggregáló vagy más SET művelet.
--ok: aggregáló függvény (max, sum, avg..) null értékeken működik
select * from #tt
select lastname, str(avg(cast(no_of_orders as float)), 15, 2) as avg_no_of_orders
--select lastname, avg(no_of_orders) as avg_no_of_orders
from #tt group by employeeid, lastname
order by avg_no_of_orders desc

--másik megoldás ugyanarra a problémára beágyazott lekérdezéssel
select forras.lastname, str(avg(cast(forras.no_of_orders as float)), 15, 2) as
avg_no_of_orders
from (
  select e.employeeid, lastname, year(orderdate) as ev, month(orderdate) as month,
  count(orderid) as no_of_orders
  from employees e left outer join orders o on e.employeeid=o.employeeid
  group by e.employeeid, lastname, year(orderdate), month(orderdate)
) as forras --alias használata kötelező
group by employeeid, lastname
order by avg_no_of_orders desc
```

---

### Házi feladatok

```
--HÁZI FELADATOK:
--Mi az átlagos havi rendelésszám minden termékre?
--Ki rendelt több mint kétszer annyit összesen, mint a főnöke?
```

---

# 12. B függelék: Adatbázis-adminisztráció és -karbantartás

A relációs adatbázis-adminisztráció leggyakoribb feladatai:

- Adatbázisfájlok kezelése
- Az adatbázis teljesítményének fenntartása (pl. töredezettség megszüntetése, táblastatisztikák frissítése)
- Felhasználók és biztonság kezelése (bejelentkezési szerepkörök, jogosultságok, házirendek, adatbázis-titkosítás)
- Biztonsági mentési stratégia megvalósítása
- Riasztások konfigurálása kritikus állapotokra

Ezek az elemek megvalósíthatók külön-külön, vagy egyetlen adatbázis-karbantartási tervbe is foglalhatók.

---

## Adatbázisfájlok

Minden SQL Server-adatbázishoz legalább két fájl tartozik: az adatbázis-objektumokat tartalmazó fájl (`.mdf`) és a tranzakciós naplófájl (`.ldf`). A naplófájl a feladatátvételi helyreállítás szempontjából a legkritikusabb, ezért redundáns adathordozón (pl. RAID-tömbön) érdemes tárolni.

A töredezettség elkerülése érdekében a fájlokat lehetőleg olyan kötetekre helyezd, amelyeket más programok nem használnak. Az adatbázis kezdeti mérete a tervezett alkalmazás alapján becsülhető; a túl kis automatikus növekedési lépés szintén töredezettséghez vezet.

Az adatbázis integritásának ellenőrzéséhez célszerű `DBCC CHECKDB`-t futtatni minden teljes biztonsági mentés előtt:

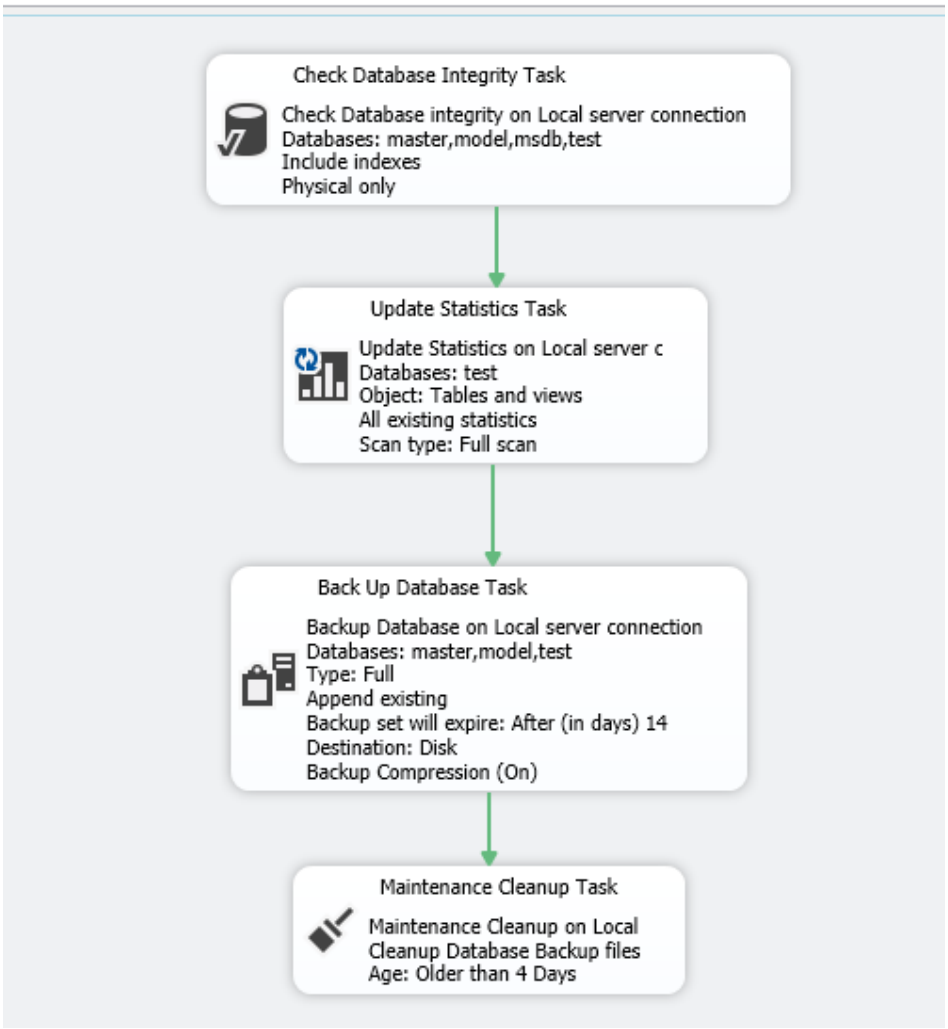
```
DBCC CHECKDB ('DB_name') WITH NO_INFOMSGS, ALL_ERRORMSGS
```

Database files:

| Logical Name | File Type | Filegroup | Initial Size (MB) |
|---|---|---|---|
| nw | ROWS Data | PRIMARY | 6 |
| nw_log | LOG | Not Applicable | 29 |

Bármilyen kimenet adatbázis-sérülést jelez.

| Subplan | Description | Schedule |
| --- | --- | --- |
| Backup test (round-robin) | | Occurs every day every 2 minute(s) between 12:... |




## Adatbázis-teljesítmény

Válaszd ki a megfelelő helyreállítási módot (recovery mode). A FULL módot csak akkor használd, ha az alkalmazás valóban megköveteli. Győződj meg arról, hogy az alábbi adatbázis-beállítások helyesek:

- **Auto update statistics** = TRUE
- **Auto create statistics** = TRUE
- **Auto shrink** = FALSE (lehetőség szerint a tábla- és adatbázis-zsugorítást egyáltalán ne használd)
- **Page verify** = CHECKSUM

Az adatbázis finomhangolása egy adott alkalmazáshoz nem tartozik ennek a kurzusnak a hatókörébe.

## Biztonsági mentések

Kliens-szerver adatbázis-alkalmazásban az előre-írás (write-ahead) tranzakciós naplóval védekezünk az adatveszteség ellen. Három típusú hibára kell felkészülni:

1. A klienssel megszakad a kapcsolat → a napló segítségével visszagörgetjük az összes nem véglegesített tranzakciót.
2. A szerver folyamata leáll (elvész a memóriabeli adat és naplópuffer) → újraindításkor a szerver először előre görget minden olyan tranzakciólépést, amelynek adatlapjai még nem kerültek a fájlba, majd visszagörgeti az összes nem véglegesített tranzakciót.
3. Az adatfájl elveszik → mentjük a tranzakciós naplót, a korábbi teljes és differenciális mentések, majd a naplómentés segítségével visszaállítjuk az adatbázist, végül visszagörgetjük a nem véglegesített tranzakciókat.

A rendszeres biztonsági mentés a legáltalánosabb katasztrófa-elhárítási módszer. A mentési fájlokat az adatbázis adatfájljaitól és naplófájljaitól eltérő adathordozón kell tárolni. A három leggyakoribb stratégia:

- **#1 – Minimum:** teljes mentések SIMPLE helyreállítási módban[43], pl. naponta egyszer, terhelésmentes időszakban. A mentéseket körforgásosan tárold (pl. heti ciklusban). Az adatveszteség így legfeljebb egy napra korlátozható. Érdemes a `master`, `model` és `msdb` adatbázisokat is belefoglalni.
- **#2:** Teljes mentések és differenciális mentések SIMPLE helyreállítási módban, pl. napi teljes mentés és 2 óránkénti differenciális mentés. Az adatveszteség legfeljebb 2 órára korlátozható.
- **#3:** Ha az alkalmazás minimális adatveszteséget követel meg, az adatbázist FULL helyreállítási módba kell állítani, és a 2. stratégiát tranzakciósnapló-mentésekkel kell kiegészíteni. FULL módban mentések nélkül a napló jelentősen megnőhet. Példastratégia: napi teljes mentés, 2 óránkénti differenciális mentés, 20 percenkénti tranzakciósnapló-mentés. Ha az adatfájl elveszik, az összes véglegesített tranzakció megőrzhető.

A mentések a `BACKUP DATABASE` / `RESTORE DATABASE` T-SQL utasításokkal végezhetők:[44]

```
USE [master]
RESTORE DATABASE [northwind] FROM DISK = N'C:\Program Files\Microsoft SQL
Server\MSSQL14.PRIM\MSSQL\Backup\nw' WITH FILE = 1, NOUNLOAD, REPLACE, STATS = 5
```

> **GYAKORLAT:** szimuláljuk a lemezhiba-helyreállítást. Hozz létre teljes biztonsági mentést a Northwind adatbázisról egy fájleszközre, állítsd le a PRIM SQL Server-szolgáltatást, töröld az adatbázisfájlt, indítsd el újra a szolgáltatást, majd állítsd vissza a mentésből a `WITH REPLACE` opcióval. Ellenőrizd a tartalmat.

[43] A SIMPLE helyreállítási mód azt jelenti, hogy a napló inaktív részei rendszeresen csonkolásra kerülnek.
[44] Az SQL-szkriptek az SSMS minden párbeszédpaneljéről generálhatók.

| Name | Date modified |
|---|---|
| master_backup_2019_04_04_113604_8418384.bak | 4/4/2019 11:36 AM |
| model_backup_2019_04_04_113604_9017994.bak | 4/4/2019 11:36 AM |
| test_backup_2019_04_04_113604_9486662.bak | 4/4/2019 11:36 AM |
| master_backup_2019_04_04_113404_9953875.bak | 4/4/2019 11:34 AM |
| model_backup_2019_04_04_113405_0431004.bak | 4/4/2019 11:34 AM |
| test_backup_2019_04_04_113405_1054666.bak | 4/4/2019 11:34 AM |
| master_backup_2019_04_04_113209_4809439.bak | 4/4/2019 11:32 AM |
| model_backup_2019_04_04_113209_5610171.bak | 4/4/2019 11:32 AM |
| test_backup_2019_04_04_113209_6200058.bak | 4/4/2019 11:32 AM |

Rendszeres mentéshez az egyik lehetőség a BACKUP T-SQL utasítást ütemezett jobban futtatni. A másik lehetőség karbantartási tervbe (maintenance plan) integrálni a mentési jobokat.

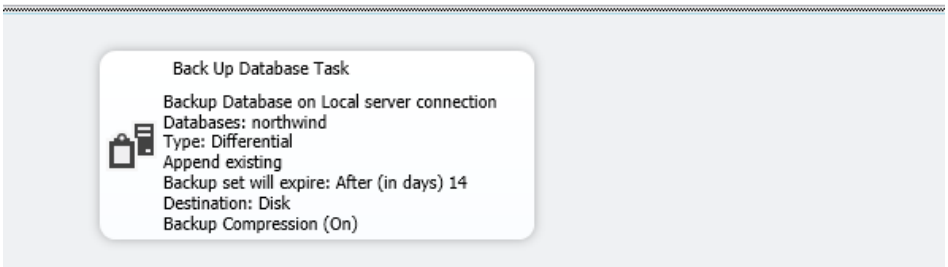
---

## Karbantartási tervek

A terv létrehozása előtt engedélyezni kell az SQL Server Agent kiterjesztett tárolt eljárásainak (XPs) használatát:

```
use master
sp_configure 'show advanced options', 1
go
reconfigure
go
sp_configure 'Agent XPs', 1
go
reconfigure
go
```

| Subplan | Descri... | Schedule |
|---|---|---|
| Full backup (NW) | | Occurs every day at 12:00:00 AM. Schedule will be used starting ... |
| Incremental backup (NW) | | Occurs every day every 2 hour(s) between 12:00:00 AM and 11:... |
| Log backup (NW) | | Occurs every day every 15 minute(s) between 12:00:00 AM and ... |




> **GYAKORLAT:** valósítsd meg az #1 biztonsági mentési stratégiát egy új karbantartási tervben egy új tesztadatbázishoz. Az ütemezést állítsd 2 percre (csak a bemutatóhoz). Foglalja magában az indexek újraszámítását és az integritás-ellenőrzést is napi feladatként. Használd a **Management → Maintenance Plans** menüpontot — ne a varázslót. Ellenőrizd, hogy 2 percenként jönnek-e létre a mentési fájlok.

A törlési (cleanup) feladatot nem tudjuk tesztelni, mert a minimális törlési kor 1 napnál kevesebbre nem állítható.

**GYAKORLAT:** valósítsd meg a #3 biztonsági mentési stratégiát egy új karbantartási tervben a Northwind adatbázishoz.

---

## Riasztások (Alerts)

Célszerű riasztást beállítani minden 24-es súlyosságú (severity 24) hibára. Egy másik klasszikus kritikus feltétel, ha az adatbázisfájlokat tartalmazó köteten kevés a szabad terület.

**DEMO:** beállítunk egy riasztást, amely akkor aktiválódik, ha a Northwind adatbázis mérete eléri a jelenlegi méret 150%-át. Előbb ellenőrizd az aktuális méretet a `C:\Program Files\Microsoft SQL Server\MSSQL14.PRIM\MSSQL\DATA` mappában: 8,2 MB.

Ha a méret lényegesen nagyobb, állítsd SIMPLE helyreállítási módba, majd zsugorítsd az adatbázist, 10%-os szabad helyet hagyva:

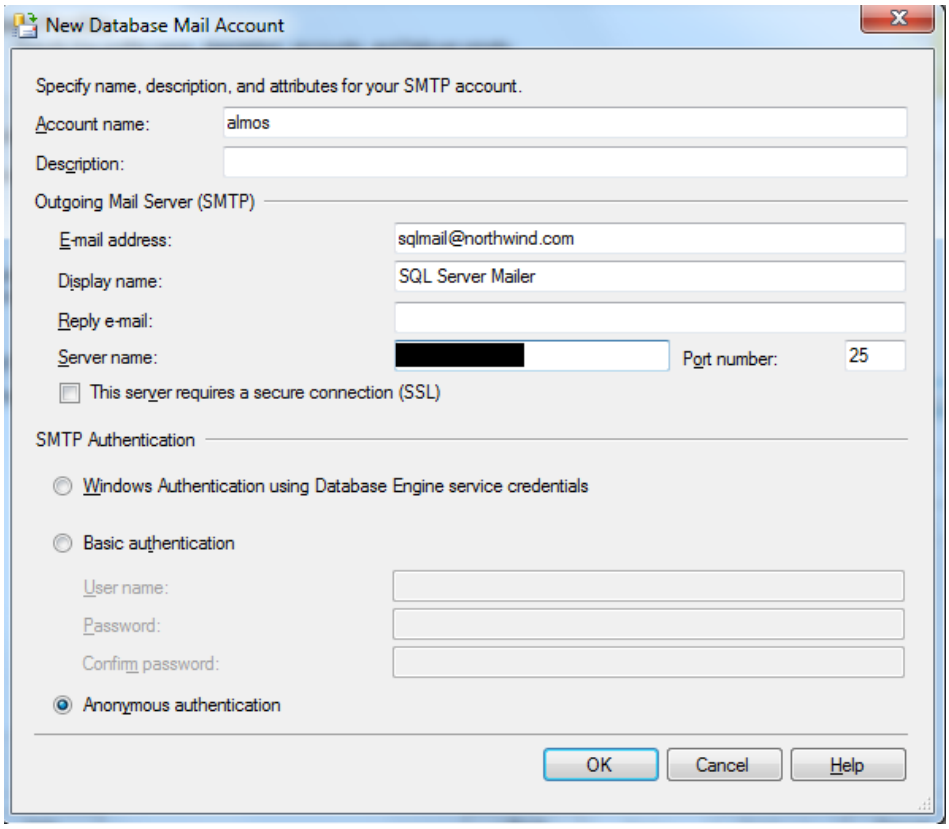
```
use master
dbcc shrinkdatabase(northwind, 10)
```

A riasztás beállítása 4 lépésből áll:

1. Adatbázis-levelezési profil (Database Mail profile) beállítása
2. A levelezési profil engedélyezése az SQL Server Agentben
3. Operátor létrehozása (az a személy, aki megkapja a riasztást és kezeli a problémát)
4. A riasztás létrehozása és tesztelése

### Adatbázis-levelezés beállítása

Válaszd a **Server → Management → Database Mail → Configure Database Mail** lehetőséget. Az új profil panelen írd be a profil nevét: `prim_mail`, majd kattints az **Add SMTP account** gombra, és add meg az SMTP-szerver nevét. Tedd a profilt nyilvánossá, hogy minden felhasználó küldhessen vele e-mailt.




Ellenőrizd a profil működését: **Database Mail → Send test E-mail**.

### Az operátor létrehozása

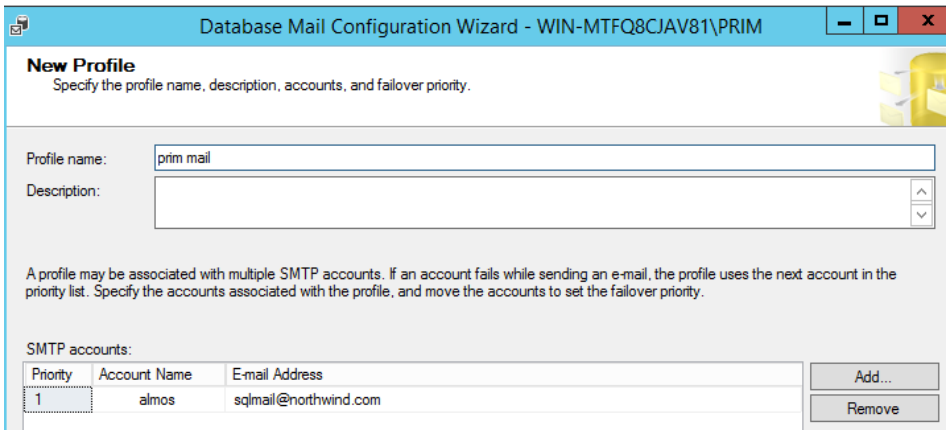
Válaszd az SQL Server Agent csomópont **Operators** elemét, adj hozzá egy új operátort, és az **E-mail name** mezőbe írd be a saját e-mail-címedet.

### A riasztás hozzáadása

Válaszd az SQL Server Agent csomópont **Alerts** elemét, és adj hozzá egy új riasztást. Állítsd be a választ: **Notify operator (NW system administrator) by email**. Az **Options** lapon adj hozzá egyéni üzenetet, és állítsd a késleltetést 1 percre (csak a bemutatóhoz).

Teszteld a riasztást az alábbi szkripttel:

```
select
  object_name(object_id) as 'tablename',
  count(*) as 'totalpages',
  sum(Case when is_allocated=0 then 1 else 0 end) as 'unusedPages',
  sum(Case when is_allocated=1 then 1 else 0 end) as 'usedPages'
from sys.dm_db_database_page_allocations(db_id(),null,null,null,'DETAILED')
group by
  object_name(object_id)

--létrehozunk egy nagy táblát
go
create table big_table (a char(4000))
declare @i int=0
while @i<1000 begin
  insert big_table values ('a')
  set @i=@i+1
end
--a big_table 500 lapos -> a riasztás elsül
```



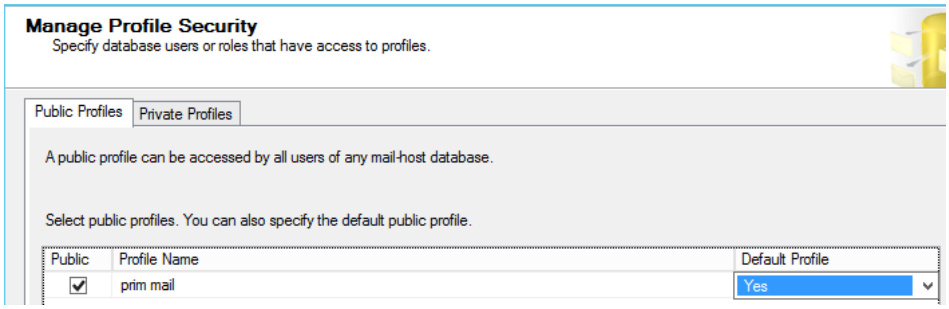

Ellenőrizd a riasztás előzményeit és a postaládádat. Ne felejtsd el törölni a `big_table`-t, és letiltani vagy törölni a riasztást.

---

## Rendszerbiztonsági tagok, jogosultságok, sémák, szerepkörök

A kiszolgálói bejelentkezések (login) és az adatbázis-felhasználók (database user) viszonyának áttekintése. A bejelentkezés adatbázis-felhasználókhoz kapcsolódik, és általában van alapértelmezett adatbázisa.

```
use master
create login nw_user with password='...', default_database=northwind
use northwind --context switch
create user nw_user for login nw_user
alter role db_datareader add member nw_user
```




> **GYAKORLAT:** hozz létre egy új adatbázist, egy új bejelentkezést és egy új adatbázis-felhasználót.

A jogosultságok áttekintése: `GRANT [jogosultság] ON [objektum] TO [kinek]`, `REVOKE`, `DENY`.

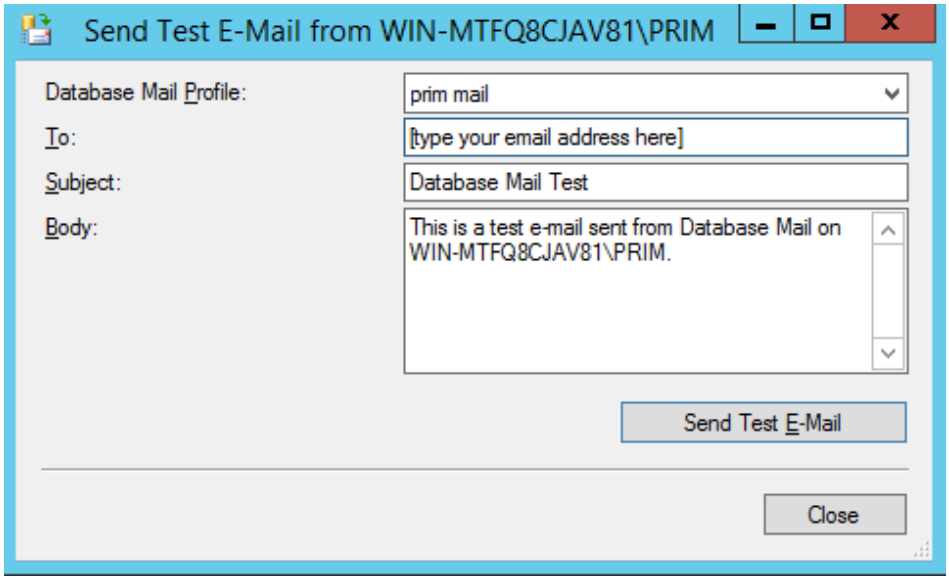
```
use northwind
alter role db_datareader drop member nw_user
go
create procedure sp_list_employees
  @city varchar(50) = 'London' --a paraméter alapértelmezett értéke
as
  select * from Employees where City=@city
go
exec sp_list_employees
go
exec sp_list_employees 'Seattle'
go
grant execute on sp_list_employees to nw_user
```

```
go
```

A szerepkörök (roles) jogosultságcsoportok, pl. olvasási hozzáférés minden táblához.

Fontos **kiszolgáló szintű szerepkörök**: `public`, `dbcreator`, `serveradmin`, `sysadmin`

Fontos **adatbázisszintű szerepkörök**: `db_owner`, `db_datareader`, `db_datawriter`; tiltó szerepkörök: `db_denydatawriter`, `db_denydatareader`

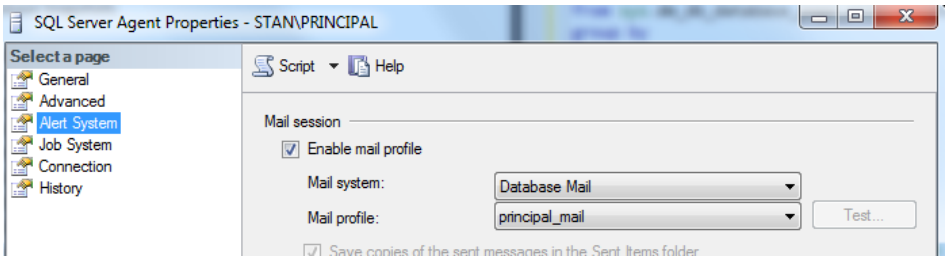



```
use adworks
alter role exec_all_sp drop member nw_user
alter role db_owner add member nw_user
alter role db_denydatawriter add member nw_user

--ellenőrzés egy nw_user kliens-kapcsolatban:
use adworks
select * from Sales.Store
--(701 sort érint)
exec Person.sp_DeletePerson_Temporal 2
--(1 sort érint)
--(1 sort érint)
create table test (id int)
--A parancsok sikeresen lefutottak.
insert test (id) values (25)
--Msg 229, Level 14, State 5, Line 10
--Az INSERT jogosultságot megtagadta a 'test' objektum, 'adworks' adatbázis, 'dbo' séma esetén.
drop table test
--A parancsok sikeresen lefutottak.
```

Saját szerepkörök is létrehozhatók:

```
use northwind
revoke execute on sp_list_employees from nw_user
go
--így adható végrehajtási jog az adatbázis ÖSSZES objektumára
grant execute to nw_user

--új szerepkör
use adworks
create role exec_all_sp
grant execute on schema::Person to exec_all_sp --a szerepkör rendszerbiztonsági tagként viselkedik
--alternatíva lehet: grant select, update on schema::Person to read_update_only_role stb.
create user nw_user for login nw_user
alter role exec_all_sp add member nw_user

--ellenőrzés egy kliens-kapcsolatban
```

```
use adworks
select * from Sales.Store
--Msg 229, Level 14, State 5, Line 8
--A SELECT jogosultságot megtagadta a 'Store' objektum, 'adworks' adatbázis, 'Sales' séma esetén.
exec Person.sp_DeletePerson_Temporal 2
--(1 sort érint)
--(1 sort érint)
```

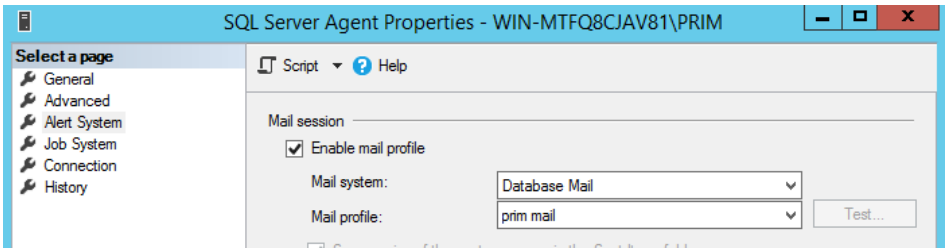



**GYAKORLAT:** hozz létre egy új szerepkört a `db_datareader` és `db_datawriter` szerepkörök összeolvasztásaként, és rendeld hozzá az új felhasználót. Teszteld.

Az adatbázis-objektumok egy sémához tartoznak, és a sémának egyetlen tulajdonosa van (ez különbözik a `db_owner` adatbázisszerepkör tagságától). A tulajdonos ejthet (DROP) sémát, és létrehozhat, illetve törölhet objektumokat a sémában. A jogosultságokat általában séma/szerepkör szinten, nem egyedi felhasználó/tábla szinten kezeljük.

Melyik rendszerbiztonsági tag melyik szerepkörben van?

```
select DP1.name as DatabaseRoleName, isnull (DP2.name, 'No members') as
DatabaseUserName
from sys.database_role_members as DRM right outer join
sys.database_principals as DP1 on DRM.role_principal_id = DP1.principal_id left outer
join sys.database_principals as DP2 on DRM.member_principal_id = DP2.principal_id
where DP1.type = 'R' --R: adatbázis-szerepkör
order by DP1.name
```




**GYAKORLAT:** tervezd meg egy nagy nemzeti könyvtár (pl. OSzK) sémáit, szerepköreit és jogosultságait.

---

## Adatbiztonság az SQL Serverben

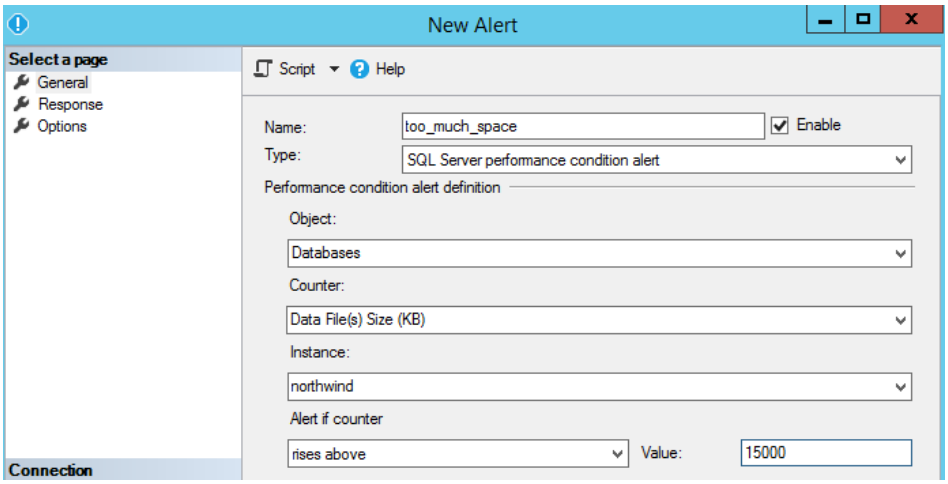
### Érzékeny adatok osztályozása

Az SQL Server 2017-től a mezők automatikusan vagy manuálisan osztályozhatók információtípus (pl. „Personal”) és érzékenységi szint (pl. „Confidential-GDPR”) szerint. Használd a **Database → Tasks → Classify data** eszközt. A rendszer sebezhetőségi elemzést (vulnerability analysis) is futtathat.

Az adatbiztonság területei:

- **Biztonságos ügyfélkapcsolatok** → SSL. A szerverhez telepíteni kell egy tanúsítványt.
- **Adatbázis- és mentési fájlok** (data at rest):

  - Fájlszint: mentési titkosítás (backup encryption) és transzparens adattitkosítás (TDE), amely az adatfájlokat is titkosítja.
  - Mezőszint: érzékeny oszlopok elrejtése még a `sysadmin` szerepkör elől is („Always Encrypted”), titkosítással és visszafejtéssel a kliensoldali.
- **A szerver által feldolgozott adatok** (data in motion):
  - **Dinamikus adatmaszkolás** (Dynamic Data Masking): érzékeny oszlopok részleges elrejtése a sémában definiált maszkkal, pl. `ALTER COLUMN [Social Security Number] <adattípus> MASKED WITH (FUNCTION = 'partial(0,"XXX-XX-",4)')` — csak az utolsó 4 jegy látható. Az `UNMASK` jogosultsággal rendelkező felhasználók az eredeti értéket olvashatják.[45]



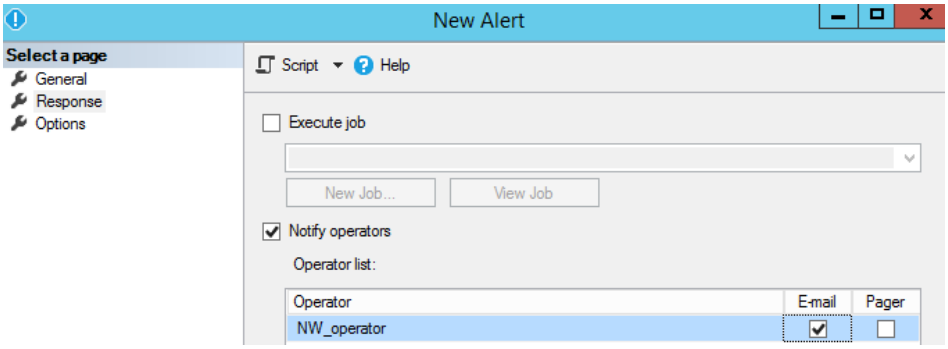

A javasolt legjobb gyakorlat: az érzékenynek minősített mezőkre mezőszintű védelmet kell alkalmazni.

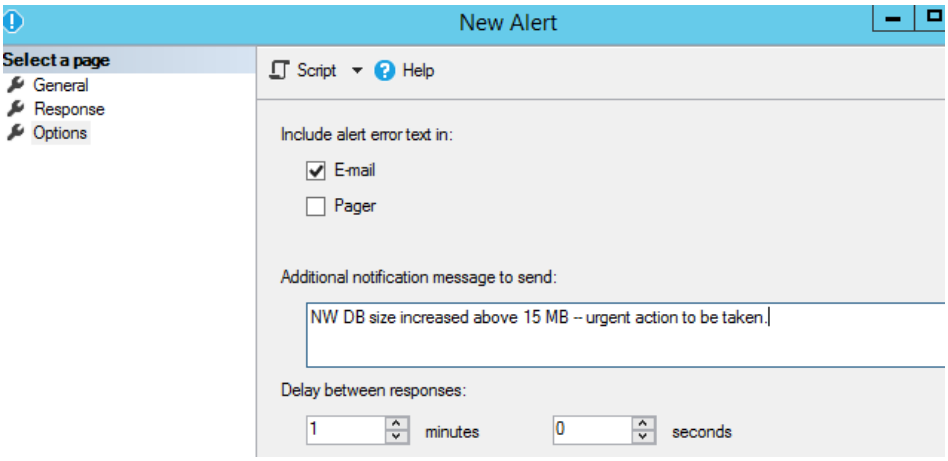
### DEMO — dinamikus adatmaszkolás

```
use northwind
--alter table employees drop column SSN
--alter table employees drop column email
alter table employees add ssn char(10) null, email varchar(200) --érzékeny oszlopok
alter table employees alter column ssn char(10) masked with (function =
'partial(1,".....",4)')
alter table employees alter column email char(200) masked with (function = 'email()')
go
update Employees set ssn ='1234567890'
update Employees set email ='sohase_mondd@citromail.hu'
select lastname, ssn, email from Employees --minden értéket látunk

--ha viszont létrehozunk egy írási/olvasási jogú felhasználót
use master
create login test with password='h6twqPNO', default_database=northwind
go
use northwind --context switch
create user test for login test
alter role db_datareader add member test
alter role db_datawriter add member test
go

--váltsunk át a test felhasználó kapcsolatára
execute as user = 'test'
select lastname, ssn, email from Employees
revert
go
--ezt látjuk:
```

[45] https://www.sqlshack.com/using-dynamic-data-masking-in-sql-server-2016-to-protect-sensitive-data/

```
--lastname     ssn           email
--Davolio      1.....7890    sXXX@XXXX.com
--Fuller       1.....7890    sXXX@XXXX.com

--most adjuk meg az UNMASK jogosultságot a test felhasználónak
grant UNMASK to test
go
execute as user = 'test'
select lastname, ssn, email from Employees
revert
go
--most ezt látjuk:
--lastname     ssn           email
--Davolio      1234567890    sohase_mondd@citromail.hu
--Fuller       1234567890    sohase_mondd@citromail.hu

--FIGYELEM: a felhasználó továbbra is módosíthatja az oszlopokat!!!
go
execute as user = 'test'
update Employees set email ='mondj_igent@citromail.hu'
select lastname, ssn, email from Employees
revert
go
revoke UNMASK from test
```




**GYAKORLAT:** próbáld ki a `default()` és `random()` maszkolási függvényeket az `AdventureworksDW2019.dbo.DimCustomer` táblán.

---

## SSL konfigurálása

- Hozz létre egy új önaláírt tanúsítványt: `certreq -new -f certparam.inf local.cer`, az `.inf` fájlban a paraméterek beállításával.

- Az mmc konzolon válaszd az **All tasks → Manage private keys** lehetőséget, és add hozzá az `MSSQL$PRIM` felhasználót teljes ellenőrzési joggal a tanúsítvány privát kulcsára.
- Az SQL Server Configuration Managerben a **Protocols → Properties** panelen állítsd be az **Enforce encryption** beállítást, majd indítsd újra a szolgáltatást.
- Az SSMS-ben futtasd a `select * from sys.dm_exec_connections` lekérdezést, és ellenőrizd: `encrypt_option=TRUE` → SSL, `auth_scheme=SQL`.

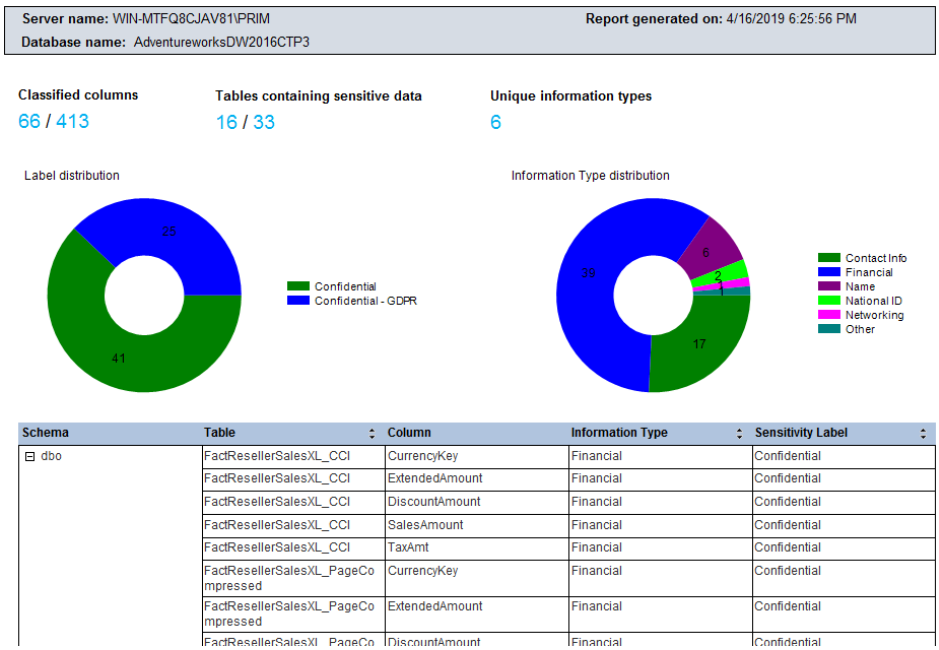
---

## Mentési titkosítás (Backup Encryption)

A Service Master Key (SMK) az SQL Server-példány telepítésekor automatikusan jön létre. A mentések titkosításához szükség van egy `master` adatbázisban tárolt főkulcsra (Master Key), amelyet a megadott jelszóval és az SMK-val is titkosítunk. Ez a főkulcs titkosítja a mentési tanúsítványok privát kulcsait.[46]

```
use master
CREATE MASTER KEY ENCRYPTION BY PASSWORD = 'h6twqPNO'
go
--mentés készítése az új MK-ról
OPEN MASTER KEY DECRYPTION BY PASSWORD = 'h6twqPNO'
go
--a mentéshez eltérő jelszót használunk
BACKUP MASTER KEY TO FILE = 'C:\install\exportedmasterkey'
ENCRYPTION BY PASSWORD = 'h6twqPNOh6twqPNO'
go
--létrehozunk egy privát-publikus kulcspárt és egy önaláírt tanúsítványt
CREATE CERTIFICATE backup_cert_master
--ezt az MK fogja titkosítani
WITH SUBJECT = 'NW DB backup',
EXPIRY_DATE = '20301031'
BACKUP CERTIFICATE backup_cert_master TO FILE = 'C:\install\exportedcert'
--visszaállítható a CREATE CERTIFICATE ... paranccsal
--mentést készítünk az adatbázisról
BACKUP DATABASE northwind TO DISK = 'C:\install\nw_enc.bak'
WITH COMPRESSION, ENCRYPTION (
  ALGORITHM = AES_256,
  SERVER CERTIFICATE = backup_cert_master
), STATS = 10
GO
```

[46] https://docs.microsoft.com/en-us/sql/relational-databases/backup-restore/backup-encryption?view=sql-server-2017

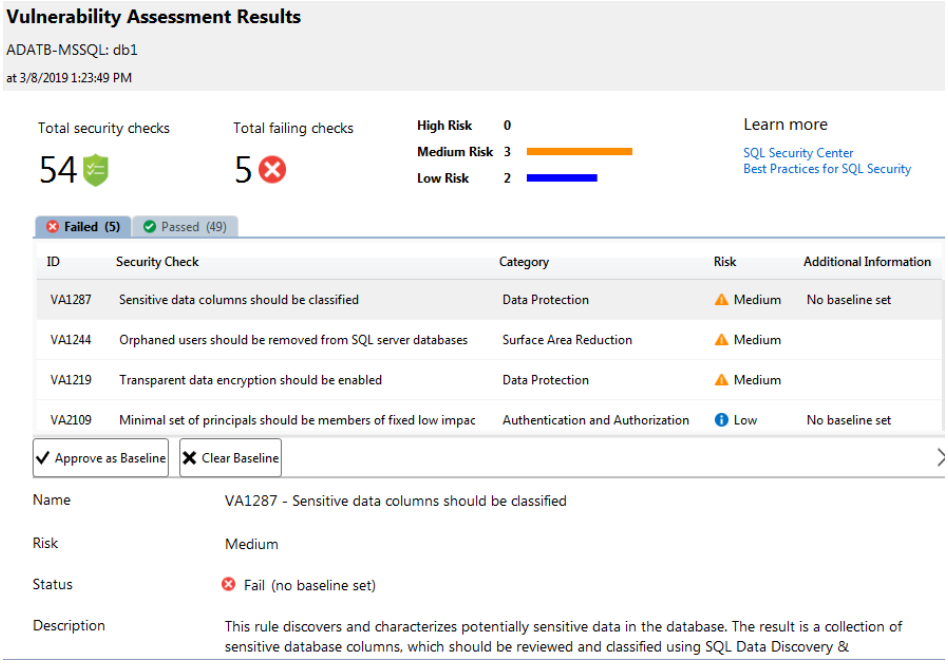



Próbáld meg a titkosított mentést visszaállítani a PRIM kiszolgálón egy új adatbázisba (a `backup_cert_master` segítségével automatikusan visszafejti). Majd próbálj meg ugyanezt elvégezni a SECOND kiszolgálón — a mentés tartalma ott olvashatatlan lesz.

> **GYAKORLAT:** titkosítsd a tesztadatbázis mentését a főkulccsal, majd próbáld meg visszaállítani a THIRD kiszolgálón.

---

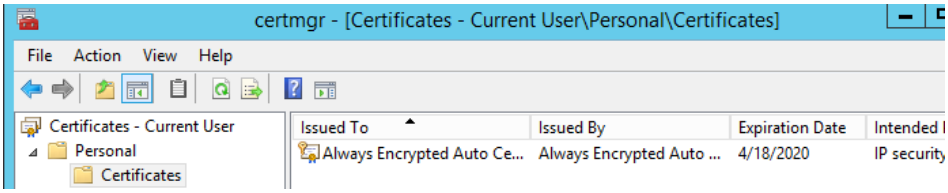
## Always Encrypted




A titkosítás és visszafejtés a kliensoldali alkalmazásban történik, a kulcsok az SQL Server-példányon kívül tárolódnak (helyi Windows tanúsítványtárban vagy Azure Key Vaultban).

**DEMO:** hozzáadjuk az Always Encrypted titkosítást az Employees tábla Address oszlopához.

1. Válaszd a **Northwind → Tables → Employees → Encrypt Columns** lehetőséget, és jelöld ki az Address mezőt.
2. Válaszd a **Randomized** titkosítási típust. Ez megakadályozza az ECB[47]-típusú támadást, de a titkosított oszlopon nem lehet csoportosítani, egyenlőség alapján szűrni vagy JOIN-t futtatni a szerveren.
3. Válaszd az új oszlopkulcs generálását.
4. Fogadd el, hogy az összevetési sorrend (collation) binárisra változik.
5. A következő panelen válaszd a **Windows certificate store** lehetőséget. Ez az SQL Server-példányon kívül van, így az `sysadmin` szerepkörű SQL Server-rendszergazdának nincs hozzáférése.
6. Ellenőrizd az új tanúsítványt a `certmgr.msc` MMC-alkalmazásban.
7. Futtass SELECT * lekérdezést a táblán, hogy lásd, mit lát a rendszergazda (titkosított értékek).
8. A visszafejtett eredmény megtekintéséhez nyiss új kapcsolatot, és az **Options → Additional Connection Parameters** mezőbe írd be: `Column Encryption Setting = Enabled`.
9. Lekérdezés futtatásakor engedélyezd a paraméterezést (parameterization), hogy a titkosított oszlopokba is lehessen szúrni, módosítani és szűrni.




**GYAKORLAT:** válaszd a **Deterministic** titkosítási típust a Title oszlophoz, és ellenőrizd, hogy azonos egyszerű szöveghez azonos titkosított szöveg generálódik — ez ECB-típusú támadást tesz lehetővé.

---

[47] https://searchsecurity.techtarget.com/definition/Electronic-Code-Book